SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

CSA1737 – ARTIFICIAL INTELLIGENCE

ASSIGNMENT 4 (CO4 – Unit IV: Learning)

Learning: Algorithm Design, Implementation, Test, Debug and Evaluate – Learning Protocols, Decision Trees, EM, Back-Propagation, Naive Bayes, Kernel Machines, Explanation-Based Learning and Reinforcement Learning

Item	Details
Student	**Saai Vardhan D** – Register No. **192525205**
Nature	Individual assignment, 50 marks, Tasks 1–6
Course	CSA1737 – Artificial Intelligence (L-T-P-C: 3-0-2-4)
Language / tools	Python 3 with NumPy, SciPy, scikit-learn (verification and comparison only) and Matplotlib; every learning algorithm is implemented from scratch; PYTHONHASHSEED=0 and fixed random seeds throughout
Date of submission	5 November 2026

Method followed in every task: Interpret the problem → identify the decision rule or state → design the logic → write pseudocode → trace manually → implement → instrument → challenge → refine. Each result follows the evidence chain *source code → test input → expected behaviour → actual output → performance metrics over several seeds or sizes → interpretation*. All metrics are measured on held-out data (a test split never used for fitting or for choosing hyper-parameters). Wherever the assignment says "predict", the prediction is printed by the program before the experiment it concerns, and the report states whether it was right; several predictions turned out wrong, and those are reported as they are.

Contents and Summary of Findings

Task	Title	Marks
1	Pseudocode review: diagnose and repair a poor learning protocol	5
2	From requirement to pseudocode to working model: decision trees	9
3	Pseudocode forensics: hidden variables and the EM algorithm	6
4	From recurrence to executable logic: back-propagation	9
5	Same dataset, different learning reasoning: Naive Bayes, kernel machines, explanation-based learning	8
6	Design challenge: reinforcement learning for an energy-aware smart controller	13
	Total	**50**

Key findings (all measured by the programs in the appendices)

Task 1. Scoring a model on its own training data gives 1.000 accuracy for an unpruned tree, 1-NN and a random forest, against 0.793, 0.851 and 0.871 on a held-out test set: optimism of **0.13–0.21** (5 seeds). On the 82:18 imbalanced data the do-nothing baseline already scores 0.82 accuracy with recall 0, so accuracy alone hides the failure. The repaired protocol (stratified split, 5-fold CV, test used once) gives a mean test F1 of 0.535 against a CV estimate of 0.533.

Task 2. The desk-check table has a three-way gain tie at the root (0.1887 each); the learner needs 6 leaves because Pass means "at least two of the three signals are good". The own tree equals scikit-learn on breast cancer (entropy: prediction agreement 1.0000, test F1 0.929). A unique ID attribute takes information gain to 1.0 and drops test accuracy from 0.905 to **0.501**; gain ratio only partly repairs it (0.862); the real repair is to exclude identifiers.

Task 3. The missing step is the normalisation of the responsibilities (rows must sum to 1). The faulty EM decreased the log-likelihood in **30 of 30** random initialisations and the weights collapsed to $\Sigma\pi \approx 10^{-19}$; the repair never decreased it (0 of 30), keeps $\Sigma\pi = 1$ and reaches the scikit-learn optimum −451.21 in 22 of 30 runs.

Task 4. Analytic gradients agree with finite differences to a maximum relative error of **6.1×10^-7**; the 2-2-1 hand trace matches the code (output 0.56429, loss 0.094922 → 0.090732). A perceptron stays at 0.50 on XOR, the network solves it in 20 of 20 runs. On digits the best cell was sigmoid, 32 hidden units, lr 1.0 (test 0.972). A six-layer sigmoid network does not learn (accuracy 0.10) with a first-to-last gradient-norm ratio of about 4000.

Task 5. On the non-linear substation data, a linear SVM scores 0.774 (F1 0.000), the RBF SVM 0.892, binned Naive Bayes 0.894 and the Task 2 tree 0.911 (5 splits). Laplace smoothing matters mostly for small data and log-loss (2.652 at $\alpha = 0$ versus 0.449 at $\alpha = 1$ with 60 training rows). Explanation-based learning produced a five-literal rule that is **8.0× cheaper** than proof from the 9-rule domain theory but misses 8 of the 14 hazardous test cases: it adds speed, not knowledge.

Task 6. The Q-learning controller (20 seeds, 1,500 training days) reduced comfort violations from 14.05 to 10.26 steps/day but spent 20 % more on energy cost, so its reward advantage over the fixed rule (−58.31 versus −59.60, 14 of 20 seeds better) is **not statistically significant** ($p = 0.42$). The prediction that it would save 10–40 % cost was wrong. The adversarial prediction was right: with the price peak moved the learner beats the baseline in only 1 of 20 seeds.

Task 1 – Pseudocode Review: Diagnose and Repair a Poor Learning Protocol (5 marks)

1.1 Diagnose the protocol [Design]

The junior engineer's algorithm is:

```
ALGORITHM TrainAndTest(Data)
BEGIN
  Read Data
  Train a suitable model on Data
  Test the model on Data
  PRINT accuracy
END
```

Problem interpretation. The goal is a student at-risk predictor: a classifier from student records to {at-risk, not-at-risk}. The quantity that matters to the college is how well the predictor will perform on *future* students, so every statement in the protocol must be judged by one question: does it estimate performance on unseen students honestly? The deficiencies below are listed with the reason each can mislead the engineer.

#	Deficiency	Why it misleads
1	**Evaluation on the training data** ("Test the model on Data")	A flexible learner can memorise its training rows. An unpruned tree, 1-nearest-neighbour and a random forest all reach training accuracy 1.000 whatever the signal is (Section 1.3), so the printed number measures memory, not prediction.
2	**No held-out test set**	Without data that the learning procedure never touched there is no unbiased estimate of generalisation error. Every number is conditioned on data that already shaped the model.
3	**No validation set or cross-validation for model selection**	"A suitable model" implies a choice (model family, depth, k, regularisation). If that choice is made by looking at the test score, the test score is no longer independent and becomes optimistic (data snooping, Section 1.3(c)).
4	**Accuracy alone on an imbalanced problem**	At-risk students are a minority (about 15–18 %). A rule "nobody is at risk" scores about 0.82 accuracy while detecting none. Recall, precision, F1 of the at-risk class and balanced accuracy are required.
5	**No baseline**	A number such as 0.85 means nothing until it is compared with the majority-class rule (0.81 here). A model that does not beat the baseline has learnt nothing.
6	**No control of randomness**	Splits, initialisation and tie-breaking use random numbers. One run can be lucky: the same unpruned tree gave test accuracies 0.78, 0.693, 0.727, 0.767, 0.773 for five seeds. Fixed seeds and repeated runs (mean ± sd) are needed.
7	**Ambiguous statements** ("a suitable model")	The statement hides every algorithmic decision (which model, which hyper-parameters, which loss, which stopping rule). Another engineer could not reproduce the experiment, and the author cannot defend it.
8	**No preprocessing or leakage policy**	Scaling, imputation or feature selection fitted on the whole of Data before splitting leaks information from test rows into training. The protocol must fit every transformation on the training part only.
9	**No uncertainty or report of the data**	Only a single number is printed: no class counts, no confusion matrix, no spread over repeats. A 0.01 difference between two models on 150 test rows is within noise.

1.2 Repair the protocol [Design]

The corrected pseudocode fixes every item above. It contains no "suitable model": the candidate models and the grid of hyper-parameters are named, the selection metric is stated, and the test set is touched exactly once.

```
ALGORITHM RepairedProtocol(Data, Labels, SEED, K, TEST_FRACTION, CANDIDATES)
INPUT : Data (n rows), Labels (at-risk = 1 / not at-risk = 0), SEED, K = 5, TEST_FRACTION = 0.25,
        CANDIDATES = every pair (max_depth in {1,2,3,4,5,8}, min_samples_leaf in {1,5,10})
OUTPUT: chosen candidate, test metrics, baseline metrics
BEGIN
  1   SET random generator seed to SEED
  2   (TrainPart, TestPart) <- STRATIFIED SPLIT of rows: TEST_FRACTION of every class goes to TestPart
  3   PRINT class counts of TrainPart and TestPart                    // report the imbalance
  4   FIT every preprocessing step (scaling, imputation) on TrainPart only; APPLY to TestPart
  5   baseline_label <- most frequent class in TrainPart              // majority-class baseline
  6   FOR every candidate c IN CANDIDATES DO
  7       FOLDS <- STRATIFIED K-FOLD split of TrainPart
  8       FOR f <- 1 TO K DO
  9           train model with parameters c on TrainPart minus fold f
 10           score_f <- F1 of the at-risk class on fold f
 11       END FOR
 12       cv_score[c] <- MEAN of score_1 .. score_K ; cv_sd[c] <- STANDARD DEVIATION
 13  END FOR
 14  best <- candidate with the largest cv_score (ties: the smaller max_depth)
 15  model <- train with parameters best on the whole TrainPart
 16  predictions <- model applied to TestPart                        // test set used ONCE
 17  COMPUTE accuracy, precision, recall, F1 (at-risk), balanced accuracy, confusion matrix from
predictions
 18  COMPUTE the same metrics for the constant prediction baseline_label on TestPart
 19  PRINT best, cv_score[best], test metrics, baseline metrics
 20  REPEAT lines 1-19 for 5 different SEED values and PRINT mean and standard deviation
END
```

Why each line is there. Lines 2–3 give an honest test set and expose the imbalance; line 4 stops leakage; line 5 and 18 give the baseline; lines 6–14 choose the model without touching the test set; line 16 uses the test set once; lines 17 and 20 report the metrics that expose imbalance and randomness.

1.3 Demonstrate the defect [Implement]

Test input. A synthetic at-risk dataset (scikit-learn `make_classification`: 600 students, 12 attributes of which 6 informative and 2 redundant, 15 % at-risk, 10 % label noise, class separation 1.0), generated for 5 seeds, and also the 569-case breast-cancer dataset as a second check. For each seed a stratified 75/25 split was made. **Expected behaviour:** training accuracy 1.000 for the three high-capacity models, with the CV and test accuracies clearly lower.

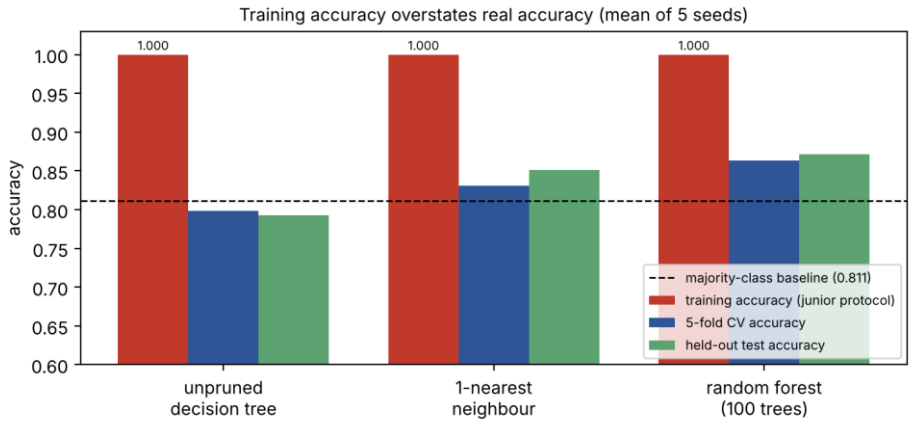


Figure 1.1 – Training accuracy (junior protocol) versus 5-fold CV and held-out test accuracy, mean of 5 seeds, with the majority-class baseline.

Model (mean of 5 seeds)	Train acc	5-fold CV acc	Test acc	Optimism (train – test)	Test recall (at-risk)
Unpruned decision tree	1.000	0.798	0.793	**0.207**	0.521
1-nearest-neighbour	1.000	0.831	0.851	**0.149**	0.533
Random forest (100 trees)	1.000	0.863	0.871	**0.129**	0.426
Majority-class baseline	–	–	0.811	–	0.000
Breast cancer, unpruned tree (1 split)	1.000	0.927	0.902	0.098	–

The per-seed values are in Appendix B.1: the optimism of the tree ranged from 0.173 to 0.233, that of 1-NN from 0.120 to 0.187 and that of the forest from 0.113 to 0.147; none of the 15 runs came close to zero. **Interpretation:** the junior protocol would report a perfect 100 % for every model, 13–21 percentage points above the true figure, and could not tell the forest (0.871) from the baseline (0.811) in terms of recall because it never looked at recall.

Other deficiencies made visible

- **(a) Accuracy on imbalanced data.** A constant "not at risk" predictor scores 0.820 accuracy on the test set (at-risk share 0.180) with recall 0.000 and F1 0.000.
- **(b) Randomness.** The same unpruned tree with five random seeds scored test accuracies [0.78, 0.693, 0.727, 0.767, 0.773], a spread of 0.087, larger than the difference between many models that engineers compare.
- **(c) Data snooping.** Choosing the depth with the best *test* accuracy gave a mean of 0.845 over five splits, whereas choosing the depth on a validation set and testing once gave 0.836: the snooped estimate is optimistic by +0.009. The effect is small for this one-parameter search but grows with the number of candidates tried.

The repaired protocol executed

The pseudocode of Section 1.2 was implemented (5 seeds, grid of 18 candidates, selection by cross-validated F1 of the at-risk class):

Seed	Chosen parameters	CV F1	Test acc	Precision	Recall	F1	Bal. acc	Baseline acc / F1
0	depth 8, leaf 5	0.639	0.833	0.550	0.407	0.468	0.667	0.820 / 0

Seed	Chosen parameters	CV F1	Test acc	Precision	Recall	F1	Bal. acc	Baseline acc / F1
1	depth 3, leaf 10	0.662	0.873	0.789	0.500	0.612	0.733	0.800 / 0
2	depth 8, leaf 10	0.449	0.887	0.789	0.536	0.638	0.751	0.813 / 0
3	depth 8, leaf 5	0.427	0.800	0.462	0.429	0.444	0.657	0.813 / 0
4	depth 8, leaf 10	0.487	0.833	0.591	0.448	0.510	0.687	0.807 / 0
Mean		0.533	0.845			**0.535**		0.811 / 0

Interpretation. The CV estimate of F1 (0.533) is almost identical to the test F1 (0.535), which is the property an honest protocol must have. The test accuracy of 0.845 beats the 0.811 baseline by only 3.4 points, and the at-risk recall is about 0.46: the model finds fewer than half of the at-risk students. The junior engineer's "1.000" would have hidden this entirely. The honest conclusion for the college is that attendance and marks in this synthetic data support a weak early-warning model that must not be the only safeguard.

Task 2 – From Requirement to Pseudocode to Working Model: Decision Trees (9 marks)

2.1 Model the requirement [Design]

Element	Specification
Input	One student record: Attendance ∈ {High, Low}, Assignments ∈ {Done, Missed}, Internal marks ∈ {Good, Poor}. (For the public dataset the inputs are 30 numeric features.)
Output	Result ∈ {Pass, Fail}, and, for explainability, the path of tests that led to it.
Objective	Find a tree that classifies the training examples and generalises to new students; among consistent trees prefer the smallest (Occam's razor).
Hypothesis space	All finite decision trees whose internal nodes test one attribute (one branch per value; for numeric attributes one threshold) and whose leaves carry a class label. This space can represent any Boolean function of the attributes, so it can overfit.
Constraints	Finite attribute set; each attribute is used at most once on a path (categorical); every example reaches exactly one leaf; training data may be noisy or inconsistent; the tree must be small enough to explain to staff.
Decision at each node	Which attribute to test next, or whether to stop and create a leaf.
Information needed to choose an attribute	The class distribution at the node (counts of Pass and Fail) and, for every candidate attribute, the class distribution in every branch it would create.

2.2 Splitting hypothesis [Design]

> **Rule proposed**
>
> **At every node, test the attribute A that maximises the information gain $Gain(S, A) = H(S) - \Sigma_{v} (|S_v|/|S|) \cdot H(S_v)$, where $H(S) = -\Sigma_{c} p_c \log_2 p_c$.**

Justification. Entropy measures how mixed the classes are in a node; the attribute with the largest gain produces branches that are, on average, the purest, so fewer further tests are needed beneath them. Greedy purity-maximising choices tend to produce short paths to leaves, hence a small tree, and small trees that fit the data are less likely to be coincidences. Two weaknesses are tested later: ties (Task 2.4) and a bias toward attributes with many values (Task 2.7).

2.3 Algorithm design [Design]

```
ALGORITHM BuildTree(Examples, Attributes, ParentExamples)
INPUT : Examples - list of (attribute values, class); Attributes - attributes still available;
        ParentExamples - examples of the parent node (used for the empty case)
OUTPUT: a decision tree (either a leaf or a node with one subtree per attribute value)
BEGIN
  1   IF Examples is empty THEN
  2       RETURN Leaf(MajorityClass(ParentExamples))                // stopping condition 1: no
examples
  3   ELSE IF all examples in Examples have the same class c THEN
  4       RETURN Leaf(c)                                             // stopping condition 2: pure node
  5   ELSE IF Attributes is empty THEN
  6       RETURN Leaf(MajorityClass(Examples))                       // stopping condition 3: no
attributes left
  7   ELSE
  8       parent_entropy <- Entropy(Examples)
  9       best_gain <- -1 ; best_attribute <- NONE
 10       FOR every attribute A IN Attributes (in the listed order) DO
 11           remainder <- 0
 12           FOR every value v of A DO
```

```
 13              S_v <- examples in Examples whose value of A equals v
 14              remainder <- remainder + ( |S_v| / |Examples| ) * Entropy(S_v)
 15          END FOR
 16          gain <- parent_entropy - remainder
 17          IF gain > best_gain THEN best_gain <- gain ; best_attribute <- A     // strict >, so ties
keep the first
 18      END FOR
 19      IF best_gain <= 0 THEN RETURN Leaf(MajorityClass(Examples))   // no attribute helps
 20      node <- new internal node testing best_attribute
 21      FOR every value v of best_attribute DO
 22          S_v <- examples in Examples whose value of best_attribute equals v
 23          subtree <- BuildTree(S_v, Attributes minus best_attribute, Examples)
 24          attach subtree to node under the branch labelled v
 25      END FOR
 26      RETURN node
 27  END IF
END

FUNCTION Entropy(S): n <- |S| ; total <- 0 ; FOR every class c: p <- count(c in S)/n ; IF p > 0 THEN
total <- total - p * log2(p) ; RETURN total
FUNCTION MajorityClass(S): RETURN the class with the largest count in S (ties: the class listed first)
FUNCTION Predict(tree, x): WHILE tree is not a leaf: tree <- subtree of tree for the value of x on the
attribute tested ; RETURN class of the leaf
```

For numeric attributes the inner loop on lines 12–15 is replaced by a search over midpoints between consecutive distinct sorted values: for every candidate threshold t the examples are divided into $\{x \le t\}$ and $\{x > t\}$ and the remainder is computed for the two parts; the best threshold is the one with the largest gain. Gini impurity $1 - \Sigma\, p_c^2$ can replace entropy in the two functions without changing anything else.

2.4 Desk-check [Design]

The eight-row table is traced by hand. Notation: [P, F] are Pass and Fail counts, $\log_2$ values are rounded to 4 decimals.

Root node (all 8 rows)

Counts [P, F] = [4, 4] so $H(S) = -(\tfrac{1}{2}\log_2\tfrac{1}{2} + \tfrac{1}{2}\log_2\tfrac{1}{2})$ = **1.0000**.

Attribute	Branch 1	Branch 2	Remainder	Gain
Attendance	High: rows 1–4 = [3,1], H = 0.8113	Low: rows 5–8 = [1,3], H = 0.8113	½(0.8113) + ½(0.8113) = 0.8113	1 − 0.8113 = **0.1887**
Assignments	Done: rows 1,2,5,6 = [3,1], H = 0.8113	Missed: rows 3,4,7,8 = [1,3], H = 0.8113	0.8113	**0.1887**
Internal	Good: rows 1,3,5,7 = [3,1], H = 0.8113	Poor: rows 2,4,6,8 = [1,3], H = 0.8113	0.8113	**0.1887**

Decision: all three gains are equal (0.1887); the strict ">" on line 17 keeps the first attribute listed, so the root is **Attendance**. This tie is itself a finding: with three symmetric signals no first split is better than another, which is why the program prints all three gains.

Second level

Attendance = High (rows 1–4: Pass, Pass, Pass, Fail; [3,1], H = 0.8113). Assignments: Done = rows 1,2 → [2,0], H = 0; Missed = rows 3,4 → [1,1], H = 1; remainder = ½·0 + ½·1 = 0.5; gain = 0.8113 − 0.5 = **0.3113**. Internal: Good = rows 1,3 → [2,0]; Poor = rows 2,4 → [1,1]; the same gain **0.3113**. Tie again → Assignments. Branch Done is pure → leaf **Pass**. Branch Missed (rows 3,4: [1,1], H = 1.0) has one attribute left, Internal: Good → [1,0], Poor → [0,1]; gain = 1 − 0 = **1.0000** → Good → Pass, Poor → Fail.

Attendance = Low (rows 5–8: Pass, Fail, Fail, Fail; [1,3], H = 0.8113). Assignments: Done = rows 5,6 → [1,1], H = 1; Missed = rows 7,8 → [0,2], H = 0; gain = 0.8113 − 0.5 = **0.3113**; Internal gives the same, tie → Assignments. Branch Missed is pure → **Fail**. Branch Done: Internal splits it perfectly (gain 1.0000): Good → Pass (row 5), Poor → Fail (row 6).

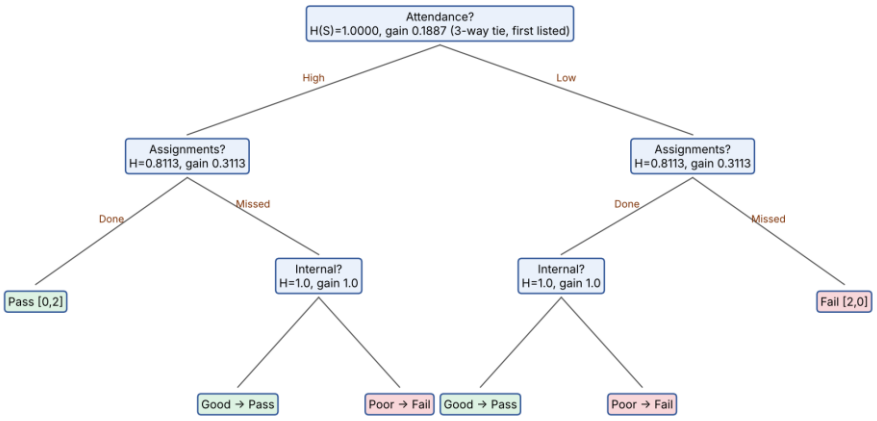


Figure 2.1 – The tree built by the desk-check; every number is a hand-computed entropy or gain.

Final tree: 6 leaves, depth 3, 8/8 training rows correct. Reading the table shows why it cannot be smaller: Pass holds exactly when at least two of the three signals are favourable (a majority-of-three function), and no single attribute decides a branch alone. The program of Section 2.5 printed identical gains (0.1887, 0.3113, 1.0000) and an identical tree (Appendix B.2).

2.5 Implement and verify [Implement]

The learner is the class `DecisionTree` in `dt.py`: entropy, Gini and gain ratio criteria; categorical attributes (one branch per value) and numeric attributes (midpoint thresholds, vectorised search); the three stopping conditions of Section 2.3; depth, minimum-split and minimum-leaf limits; reduced-error pruning; a `verbose` mode that prints every gain. **Public dataset:** Wisconsin breast cancer (569 cases, 30 numeric features), stratified 60/20/20 split (train 341, validation 114, test 114; malignant/benign in test 42/72), seed 0. **Expected behaviour:** the same split and criterion should give essentially the same tree as scikit-learn's `DecisionTreeClassifier`.

Model (test set, n = 114)	Train acc	Test acc	Precision	Recall	F1	Leaves	Depth	Confusion [[TN,FP],[FN,TP]]
Own tree (entropy)	1.000	0.912	0.956	0.903	0.929	10	6	[[39,3],[7,65]]
scikit-learn (entropy)	1.000	0.912	0.956	0.903	0.929	10	6	[[39,3],[7,65]]
Own tree (Gini)	1.000	0.912	0.943	0.917	0.930	13	6	[[38,4],[6,66]]
scikit-learn (Gini)	1.000	0.921	0.957	0.917	0.936	13	6	[[39,3],[6,66]]

Interpretation. For entropy the two trees agree on every test and training prediction (agreement 1.0000) and choose the same root (`worst perimeter`, thresholds 101.6 versus 101.5: a midpoint convention difference). For Gini one test prediction differs (agreement 0.9912): scikit-learn evaluates candidate thresholds with a different tie-breaking order when two splits have identical impurity, which is an implementation detail and not a logic error (the roots are identical, `mean concave points` at 0.04923). **Boundary tests of the own implementation:** a single class gives one leaf; a single example gives one leaf; constant features give the majority leaf; an unseen categorical value falls back to the parent majority (the

empty-examples stopping condition). Positive class is "benign" = 1, so recall is the fraction of benign cases found; the 7 false negatives are benign cases classified malignant.

2.6 Overfitting and pruning [Implement]

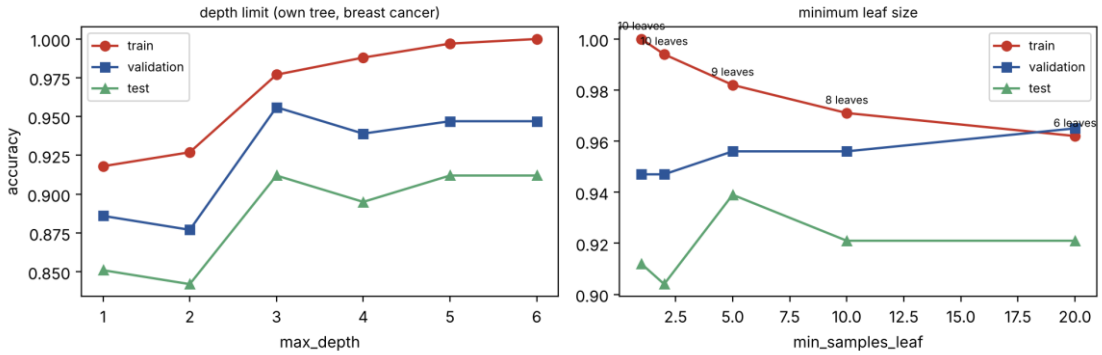


Figure 2.2 – Train, validation and test accuracy as the depth limit and the minimum leaf size are varied (own tree).

Setting	Depth	Leaves	Train	Validation	Test
Full tree (no limit)	6	10	1.000	0.947	0.912
max_depth = 1	1	2	0.918	0.886	0.851
max_depth = 2	2	4	0.927	0.877	0.842
max_depth = 3	3	6	0.977	0.956	0.912
max_depth = 4	4	8	0.988	0.939	0.895
max_depth = 5	5	9	0.997	0.947	0.912
min_samples_leaf = 2	6	10	0.994	0.947	0.904
min_samples_leaf = 5	5	9	0.982	0.956	0.939
min_samples_leaf = 10	4	8	0.971	0.956	0.921
min_samples_leaf = 20	3	6	0.962	0.965	0.921
Reduced-error pruning	3	4	0.962	0.974	0.939

Interpretation. The full tree has the highest training accuracy by construction, 1.000, and the gap to validation and test is the overfitting. Limiting depth to 3 (6 leaves) or pruning by validation error to 4 leaves keeps test accuracy at 0.912–0.939 with a smaller, more explainable tree. The differences on one 114-case test set are small (one case is 0.9 percentage points), so the tests were repeated on 10 random splits: full tree 0.931 ± 0.023, depth 3 0.938 ± 0.019, reduced-error pruning 0.936 ± 0.014. Pruning therefore gives a slightly better and clearly more stable tree (lower standard deviation), not a dramatic gain. The learned depth-3 tree (Appendix B.2) first tests `worst perimeter ≤ 101.6` and then `worst smoothness`, which is consistent with how malignant nuclei are larger and rougher.

2.7 Challenge your design

(a) Change one value

Change: row 4 (High, Missed, Poor) from Fail to Pass. **Prediction written before the run (from hand calculation):** the class counts become [5 Pass, 3 Fail], H(S) = 0.9544. Attendance: High = [4,0] → 0; Low = [1,3] → 0.8113; remainder 0.4057; gain **0.5488**. Assignments: Done [3,1] 0.8113, Missed [2,2] 1.0; remainder 0.9056, gain **0.0488**. Internal gives 0.0488 too. So the root remains Attendance but now by a

clear margin, not by a tie-break; the High branch becomes a pure Pass leaf and the tree shrinks from 6 to 4 leaves.

Verification by the code: root = Attendance, gains 0.5488 / 0.0488 / 0.0488, 4 leaves, exactly as predicted. The pattern is now "Pass unless Attendance is Low and (Assignments Missed, or Internal Poor)". **Lesson:** one noisy label turned a symmetric problem into one with a dominant attribute, which shows how sensitive greedy gain is to single examples on tiny data.

(b) Identifier-like attribute

A column StudentID (S1…S8, unique per row) was added to the eight-row table. Splitting on it gives eight pure branches, so the remainder is 0 and its gain equals H(S) = **1.0000**, larger than 0.1887, so it becomes the root and the tree is an eight-leaf lookup table that cannot classify any new student (an unseen ID falls to the parent majority). The same happens at scale: on a 300/200 synthetic table with a unique ID column:

Variant (mean of 5 seeds)	Train acc	Test acc	Leaves	Root
No ID column	0.912	0.905	8	Internal
ID column, information gain	1.000	**0.501**	300	StudentID
ID column, gain ratio	1.000	0.862	281	Internal

Repair 1 – gain ratio (C4.5): divide the gain by SplitInfo(A) = $-\Sigma$ ($|S_v|/|S|$) $\log_2(|S_v|/|S|)$, which penalises many-valued attributes. For the eight-row table SplitInfo(StudentID) = $\log_2 8 = 3$, so the ratio is 1.0/3 = 0.3333, **still larger** than 0.1887/1 = 0.1887 for Attendance, so StudentID is still chosen (confirmed by the program). On the larger table gain ratio recovers part of the accuracy (0.862 against 0.905 without the ID) but the tree still has 281 leaves. **Repair 2 – the real fix:** an identifier carries no generalisable information, so it must be excluded from the attribute set before learning (and, in general, attributes with more distinct values than a fraction of the rows should be flagged); minimum-leaf limits and pruning also prevent one-row leaves. The experiment is a good warning that a "repair" can fail and must itself be tested.

Task 3 – Pseudocode Forensics: Hidden Variables and the EM Algorithm (6 marks)

3.1 Locate the defect [Design]

Problem interpretation. Unlabelled data are assumed to come from K Gaussians. Which component produced each point is a hidden variable z_i. EM alternates an E-step, which computes for each point and component the posterior probability (responsibility) γ_ik = P(z_i = k | x_i), and an M-step, which re-estimates the parameters as responsibility-weighted averages. The M-step formulas are correct only if the responsibilities are genuine probabilities, that is, if Σ_k γ_ik = 1 for every point.

> **The defect**
>
> **The incomplete step is the E-step line** `r[i][k] <- pi[k] * Normal(X[i] | mu[k], var[k])`. It computes the *joint* density π_k N(x_i | μ_k, σ_k²) but never divides by its sum over k. Required: r[i][k] = π_k N(x_i | μ_k, σ_k²) / Σ_j π_j N(x_i | μ_j, σ_j²).
>
> **Quantity that loses its meaning:** the responsibility r[i][k]. It is no longer a probability (its row sum is the marginal density p(x_i), typically 0.1–0.2 instead of 1). Consequently N_k = Σ_i r[i][k] is no longer the expected number of points of component k, and π_k = N_k / n no longer sums to 1.

3.2 Failing input and trace [Design]

Designed data: X = [0, 1, 4, 5] (n = 4, two obvious clusters), K = 2, initial π = (0.5, 0.5), μ = (1, 4), σ^2 = (1, 1). The Normal density is N(x | μ, σ^2) = exp(−(x−μ)²/(2σ^2)) / √(2πσ^2), with 1/√(2π) = 0.39894. **E-step, row i = 1 (x = 0):** N(0 | 1, 1) = 0.39894·$e^{-0.5}$ = 0.24197, so r[1][1] = 0.5 × 0.24197 = **0.12099**; N(0 | 4, 1) = 0.39894·e^{-8} = 0.000134, so r[1][2] = 0.5 × 0.000134 = 0.00007. The row sum is 0.12105, not 1. **This is the first incorrect value.**

i	x	faulty r[i][1]	faulty r[i][2]	row sum	repaired r[i][1]	repaired r[i][2]	row sum
1	0	0.12099	0.00007	0.12105	0.99945	0.00055	1.00000
2	1	0.19947	0.00222	0.20169	0.98901	0.01099	1.00000
3	4	0.00222	0.19947	0.20169	0.01099	0.98901	1.00000
4	5	0.00007	0.12099	0.12105	0.00055	0.99945	1.00000

M-step with the faulty responsibilities: N_1 = 0.12099 + 0.19947 + 0.00222 + 0.00007 = 0.32274 (and N_2 the same), so ΣN_k = 0.64548 while n = 4; π = (0.08068, 0.08068) with Σπ = **0.16137** instead of 1. The means 0.64656 and 4.35344 look plausible because the weights cancel in the ratio, which makes the bug hard to see; the variances 0.31506 and the weights are what reveal it. **With the repaired responsibilities** N = (2.0, 2.0), ΣN = 4.0, π = (0.5, 0.5), μ = (0.51786, 4.48214), σ^2 = 0.32113.

3.3 Repair [Design]

Only one line is added; nothing else changes. After the inner loop over k for each i, insert:

```
    FOR i <- 1 TO n DO
        FOR k <- 1 TO K DO   r[i][k] <- pi[k] * Normal(X[i] | mu[k], var[k])   END FOR
        total <- SUM over k of r[i][k]                    // NEW: marginal density p(x_i)
        FOR k <- 1 TO K DO   r[i][k] <- r[i][k] / total   END FOR     // NEW: normalise the row
    END FOR
```

Implementation notes (boundary handling, not a change to the logic): if `total` is 0 because of underflow the row is set to 1/K, and each variance is kept above a floor of 10^-6 to prevent a component collapsing onto one point.

3.4 Implement faulty and repaired versions [Implement]

Test input. 200 points drawn from 0.4·N(−2, 1) + 0.6·N(3, 1.5²) (seed 1), K = 2, T = 60 iterations, 30 random initialisations (random distinct data points as means, data variance as variance, Dirichlet weights). **Expected behaviour:** the repaired version never decreases the log-likelihood (EM's monotonicity guarantee) and keeps Σπ = 1; the faulty version has no such guarantee. Reference: scikit-learn `GaussianMixture` with 10 restarts.

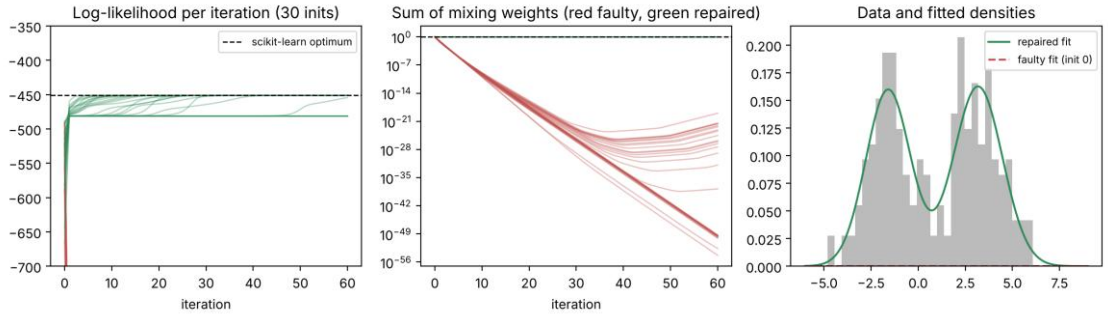


Figure 3.1 – Log-likelihood per iteration and sum of mixing weights for the faulty and the repaired EM over the random initialisations.

Version	Runs with an LL decrease	Largest decrease	Final Σπ (min … max)	Final LL (min / median / max)	Runs within 0.01 of optimum
Faulty	**30 of 30**	9940.87	4.8×10^-55 … 1.0×10^-19	−135339 / −39685 / −32132	0 of 30
Repaired	**0 of 30**	0.0000	1 … 1 (|Σπ−1| ≤ 8.9×10^-16)	−481.33 / −451.21 / −451.21	22 of 30

Interpretation. In the faulty version every iteration multiplies the weights by the row-sum factor (about 0.1–0.2), so Σπ shrinks geometrically (0.0922, 0.0094, 0.0010, … in the first iterations of the traced run) and the log-likelihood falls from −521 to −3478 in six iterations: the model assigns ever less probability to the data. The repaired version reaches the scikit-learn optimum (LL −451.21, weights (0.527, 0.473), means (3.178, −1.59)) in 22 of 30 runs. The other 8 runs end at the local optima −454.45, −481.32 and −481.33: EM is guaranteed to be monotone, not to be global, which is why restarts are used. The best run found weights [0.527, 0.473], means [3.178, −1.59] and variances [1.675, 1.397] against true values (0.6, 0.4), (3, −2) and (2.25, 1).

3.5 Regression check

An input on which the original already worked: K = 1 with two points at ±0.21557 and variance 0.1, chosen so that the density at both points is exactly 1.000000. Then the unnormalised r[i][1] = 1 × 1 = 1, which equals the correct responsibility (with a single component it must be 1). Original: r = [1, 1], N = 2, μ = 0, σ² = 0.046471, π = 1. Repaired: identical. Both agree with the closed-form MLE (μ = 0, σ² = a² = 0.046471, π = 1), so the repair preserves correct behaviour.

Contrast input (original fails): K = 2 with tight, well-separated clusters started near the truth. The original ends with $\pi = (0.2684, 0.3554)$ ($\Sigma = 0.6239$), variances 0.0042 and 0.008 and LL −12.13; the repaired version gives $\pi = (0.4, 0.6)$, $\mu = (-1.95, 3.05)$, $\sigma^2 = (0.0125, 0.0292)$, LL −1.5513, identical to scikit-learn. So the one-line repair matters exactly when the densities differ from 1, which is nearly always.

Task 4 – From Recurrence to Executable Logic: Neural-Network Back-Propagation (9 marks)

4.1 Define the state [Design]

Network with layers l = 1…L; layer 0 is the input. n_l is the number of units of layer l. Column-vector convention.

Symbol	Meaning	Dimension
a^(0) = x	input vector	n_0 × 1
W^(l)	weight matrix from layer l−1 to layer l	n_l × n_{l-1}
b^(l)	bias vector	n_l × 1
z^(l) = W^(l) a^(l−1) + b^(l)	pre-activation (weighted input) of layer l	n_l × 1
a^(l) = f_l(z^(l))	activation (output) of layer l; a^(L) = ŷ	n_l × 1
δ^(l) = ∂E/∂z^(l)	error term: sensitivity of the loss E to the pre-activation of layer l	n_l × 1

4.2 Forward and backward recurrences [Design]

Forward recurrence (for l = 1 … L): z^(l) = W^(l) a^(l−1) + b^(l); a^(l) = f_l(z^(l)); then the loss E(a^(L), y) is evaluated. Every layer needs only the previous layer's output, so it runs from the input to the output.

Backward recurrence by the chain rule. E depends on z^(l) only through a^(l) and then through z^(l+1) = W^(l+1) a^(l) + b^(l+1). Therefore

$$\delta^{(l)} = ((W^{(l+1)})^{T}\,\delta^{(l+1)}) \odot f'_l(z^{(l)}) \qquad for\ l = L-1, \ldots, 1$$

Meaning of each term. (W^(l+1))^T δ^(l+1) carries the error of layer l+1 back along the same weights that carried the signal forward: it is the sum, over the units fed by unit j, of (weight × that unit's error). f'_l(z^(l)) is the local slope: a saturated unit (slope ≈ 0) blocks the error; ⊙ is element-wise product. The gradients then follow in one line each:

$$\partial E/\partial W^{(l)} = \delta^{(l)}\,(a^{(l-1)})^{T}, \qquad \partial E/\partial b^{(l)} = \delta^{(l)}$$

Boundary condition at the output layer. δ^(L) = ∂E/∂a^(L) ⊙ f'_L(z^(L)). For squared error E = ½‖a − y‖² with a sigmoid output this is (a − y) ⊙ a ⊙ (1 − a). For softmax with cross-entropy the two factors cancel and δ^(L) = a^(L) − y, which removes the saturation problem at the output. **Why last to first:** δ^(l) needs δ^(l+1) (the recurrence runs backwards), and the first quantity that can be computed directly from the loss is δ^(L). Computing each δ once and re-using it gives total cost proportional to the number of weights, whereas differentiating each weight separately would repeat the shared products.

4.3 Pseudocode for mini-batch training [Design]

```
ALGORITHM TrainNetwork(X, Y, LAYER_SIZES, ETA, BATCH, MAX_EPOCHS, TOL, SEED)
INPUT : X (n x d), Y (n x c, one-hot), LAYER_SIZES = [n_0, n_1, ..., n_L], ETA learning rate,
        BATCH batch size, MAX_EPOCHS, TOL loss-change tolerance, SEED
OUTPUT: weights W[1..L], biases b[1..L], loss history
BEGIN
  1   SET random generator seed to SEED
  2   FOR l <- 1 TO L DO
  3       W[l] <- matrix n_l x n_{l-1} of random numbers with variance 1/n_{l-1} (sigmoid, tanh) or 2/n_{l-1}
(ReLU)
  4       b[l] <- zero vector of length n_l
  5   END FOR
```

```
  6   previous_loss <- INFINITY
  7   FOR epoch <- 1 TO MAX_EPOCHS DO
  8       shuffle the row order of (X, Y)
  9       epoch_loss <- 0
 10       FOR every mini-batch (Xb, Yb) of BATCH consecutive rows DO
 11           m <- number of rows in the batch
 12           a[0] <- Xb transposed                                  // forward pass
 13           FOR l <- 1 TO L DO  z[l] <- W[l] * a[l-1] + b[l] ; a[l] <- f_l(z[l])  END FOR
 14           batch_loss <- (1/m) * SUM over rows of E(a[L], Yb)      // loss
 15           delta[L] <- output boundary term (a[L] - Yb for softmax+cross-entropy;
                          (a[L] - Yb) * a[L] * (1 - a[L]) for sigmoid+squared error)     // backward pass
 16           FOR l <- L - 1 DOWN TO 1 DO
 17               delta[l] <- ( transpose(W[l+1]) * delta[l+1] ) * f'_l(z[l])     // element-wise product
 18           END FOR
 19           FOR l <- 1 TO L DO                                     // update
 20               W[l] <- W[l] - ETA * (1/m) * delta[l] * transpose(a[l-1])
 21               b[l] <- b[l] - ETA * (1/m) * SUM over the batch columns of delta[l]
 22           END FOR
 23           epoch_loss <- epoch_loss + batch_loss * m
 24       END FOR
 25       epoch_loss <- epoch_loss / n
 26       APPEND epoch_loss to the history
 27       IF epoch_loss is not a finite number THEN STOP and REPORT divergence   // stopping criteria
 28       IF ABS(previous_loss - epoch_loss) < TOL THEN STOP
 29       previous_loss <- epoch_loss
 30   END FOR
 31   RETURN W, b, history
END
```

4.4 Desk-check [Design]

2-2-1 sigmoid network, $x = (1, 0)^T$, $y = 1$, $E = \tfrac{1}{2}(a - y)^2$, $\eta = 0.5$, biases 0. Hidden unit 1 has weights (0.5, −0.4), unit 2 has (0.3, 0.8); the output unit has weights (0.6, −0.2). So $W^{(1)} = [[0.5, -0.4], [0.3, 0.8]]$, $W^{(2)} = [[0.6, -0.2]]$, $\sigma(z) = 1/(1 + e^{-z})$, $\sigma' = \sigma(1 - \sigma)$.

Forward pass

- $z^{(1)} = W^{(1)} x = (0.5 \cdot 1 - 0.4 \cdot 0,\ 0.3 \cdot 1 + 0.8 \cdot 0) =$ **(0.5, 0.3)**
- $a^{(1)} = (\sigma(0.5), \sigma(0.3)) =$ **(0.62246, 0.57444)**
- $z^{(2)} = 0.6 \cdot 0.62246 - 0.2 \cdot 0.57444 = 0.37348 - 0.11489 =$ **0.25859**; $a^{(2)} = \sigma(0.25859) =$ **0.56429**
- $E = \tfrac{1}{2}(0.56429 - 1)^2 = \tfrac{1}{2}(0.18999) =$ **0.094922**

Backward pass

- $\delta^{(2)} = (a - y) \cdot a \cdot (1 - a) = (-0.43571)(0.56429)(0.43571) =$ **−0.10713**
- $(W^{(2)})^T \delta^{(2)} = (0.6 \cdot (-0.10713),\ -0.2 \cdot (-0.10713)) = (-0.06428,\ 0.02143)$; $f'(z^{(1)}) = (0.62246 \cdot 0.37754,\ 0.57444 \cdot 0.42556) = (0.23500,\ 0.24445)$
- $\delta^{(1)} = (-0.06428 \cdot 0.23500,\ 0.02143 \cdot 0.24445) =$ **(−0.01511, 0.00524)**
- Gradients: $\partial E/\partial W^{(2)} = \delta^{(2)}(a^{(1)})^T = (-0.06668, -0.06154)$; $\partial E/\partial W^{(1)} = \delta^{(1)} x^T = [[-0.01511, 0], [0.00524, 0]]$ (the second column is 0 because $x_2 = 0$); biases: $\partial E/\partial b^{(2)} = -0.10713$, $\partial E/\partial b^{(1)} = (-0.01511,\ 0.00524)$.

Update ($\eta = 0.5$)

- $W^{(2)} = (0.6 + 0.5 \cdot 0.06668,\ -0.2 + 0.5 \cdot 0.06154) =$ **(0.63334, −0.16923)**
- $W^{(1)} = [[0.5 + 0.00755, -0.4], [0.3 - 0.00262, 0.8]] =$ **[[0.50755, −0.4], [0.29738, 0.8]]**

With the weights alone updated the output rises to 0.57401 and the loss falls from 0.094922 to 0.090732, as gradient descent on a small step should do.

4.5 Implement and verify [Implement]

`nn.py` implements the class `MLP` (any layer sizes; sigmoid, tanh or ReLU hidden units; sigmoid output with squared error or softmax output with cross-entropy), with `forward`, `backward` (δ recurrence), `numerical_gradient` and a mini-batch `train`. **The desk-check was reproduced by the code** (Appendix B.4): z^(1) = [0.5 0.3], a^(1) = [0.62246 0.57444], a^(2) = 0.56429, loss 0.094922, δ^(2) = −0.10713, δ^(1) = [−0.01511 0.00524], W^(2) after the update [0.63334, −0.16923], loss afterwards 0.090732, matching the hand values digit for digit.

Gradient check

Each parameter was perturbed by ±10^-6 (central difference) and the relative error |g_a − g_n| / max(|g_a|, |g_n|, 10^-12) was recorded. **Expected:** below 10^-6 if the recurrence is right.

Network	Parameters	Max relative error	Max absolute difference
3-4-2 sigmoid, squared error	26	3.124×10^-7	2.0×10^-11
3-5-4-3 tanh hidden, softmax + cross-entropy	59	2.113×10^-8	1.9×10^-10
3-6-3 ReLU hidden, softmax + cross-entropy	45	6.107×10^-7	4.2×10^-10
4-5-5-5-2 sigmoid (deep), squared error	97	5.297×10^-7	1.8×10^-11
Deliberate bug: f′(z) omitted in one layer	–	**1.000**	–

Interpretation. All four correct networks are below 10^-6, the largest being 6.1×10^-7; the planted bug gives an error of 1.0, so the check can detect a wrong recurrence. The check is therefore evidence, not a formality.

XOR

A single-layer perceptron (learning rule, 1000 epochs, 20 random starts) reached accuracy 0.50 in every start, never classifying all four points: XOR is not linearly separable. The 2-3-1 sigmoid network (back-propagation, 6000 epochs, η = 1.0, 20 random starts) classified all four points correctly in **20 of 20** runs, median final loss 0.00078; for seed 0 the outputs for (0,0), (0,1), (1,0), (1,1) were 0.021, 0.958, 0.954, 0.054.

4.6 Stress the design [Implement]

Data: 8×8 digits (1,797 images), pixels scaled to [0, 1], stratified 60/20/20 split (1077/360/360), mini-batch 32, 60 epochs, softmax + cross-entropy, Xavier initialisation (sigmoid) or He initialisation (ReLU). 32 cells: hidden {2, 8, 32, 64} × learning rate {0.001, 0.01, 0.1, 1.0} × activation {sigmoid, ReLU}.

> **Predictions written before the runs (printed by the program)**
>
> **P1:** hidden = 2 limits accuracy to roughly 0.5–0.75 in every setting. **P2:** lr = 0.001 underfits (validation < 0.5 for sigmoid, better for ReLU). **P3:** lr = 1.0 with ReLU is unstable (spikes or NaN) while sigmoid still trains. **P4:** the best cell is hidden 64 or 32 with lr 0.1, test accuracy above 0.95.

Hidden	lr	Sigmoid: loss	val	test	F1	ReLU: loss	val	test	F1
2	0.001	2.3115	0.097	0.097	0.018	2.2326	0.150	0.206	0.109
2	0.01	2.2458	0.175	0.189	0.148	1.7967	0.217	0.250	0.173
2	0.1	1.3723	0.444	0.425	0.343	0.8527	0.658	0.653	0.625

Hidden	lr	Sigmoid: loss	val	test	F1	ReLU: loss	val	test	F1
2	1.0	1.0302	0.547	0.575	0.522	2.3103	0.100	0.100	0.018
8	0.001	2.2818	0.100	0.081	0.058	2.1212	0.203	0.183	0.162
8	0.01	2.0022	0.581	0.542	0.477	0.6007	0.886	0.844	0.843
8	0.1	0.3825	0.925	0.908	0.907	0.0639	0.961	0.950	0.950
8	1.0	0.0362	0.956	0.958	0.958	2.3103	0.100	0.100	0.018
32	0.001	2.2634	0.164	0.161	0.155	1.9298	0.439	0.414	0.379
32	0.01	1.7656	0.783	0.775	0.748	0.3338	0.933	0.925	0.925
32	0.1	0.2018	0.950	0.950	0.950	0.0445	0.975	0.964	0.964
32	1.0	0.0187	0.978	0.972	0.972	0.0017	0.967	0.969	0.970
64	0.001	2.2656	0.214	0.222	0.172	1.8465	0.542	0.492	0.454
64	0.01	1.6450	0.833	0.822	0.805	0.3063	0.925	0.922	0.923
64	0.1	0.1759	0.950	0.947	0.947	0.0405	0.972	0.975	0.975
64	1.0	0.0183	0.972	0.972	0.972	0.0014	0.975	0.969	0.970

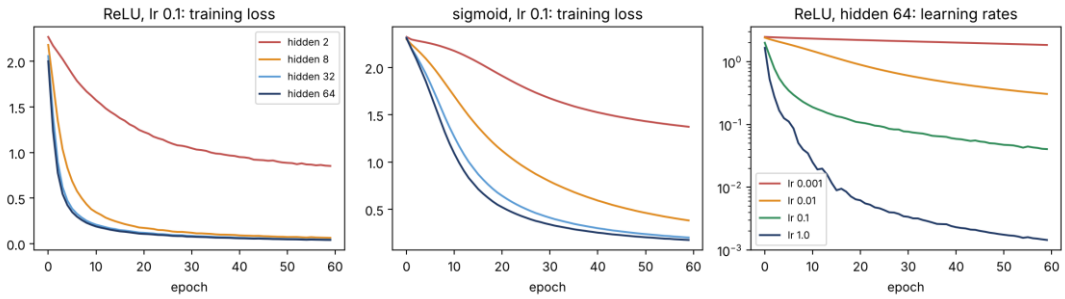


Figure 4.1 – Training-loss curves for selected settings of the digits experiment.

Prediction check (honest). P1 partly wrong: the 2-unit networks are even worse than predicted at small learning rates (test 0.097–0.25) and reach only 0.425–0.653 at lr 0.1 (ReLU 0.653 is inside the predicted range; sigmoid at lr 1.0 reached 0.575). **P2 right:** at lr = 0.001 and 64 hidden units, sigmoid validation accuracy was 0.214 and ReLU 0.542 (ReLU better). **P3 wrong:** ReLU with lr 1.0 and 64 hidden units did not diverge (test 0.969, final loss 0.0014); the instability appears only with 2 or 8 hidden units, where ReLU at lr 1.0 collapsed to chance (0.100, loss 2.3103, dead units). **P4 half right:** the best cell by validation accuracy was sigmoid, 32 hidden, lr 1.0 (test 0.972, macro-F1 0.972), which is the predicted size but a larger learning rate than predicted; lr 0.1 gave 0.950–0.975. The wrong predictions are informative: with a mini-batch of 32 and softmax-cross-entropy the gradients are well scaled, so large steps are tolerated by wide networks.

Confusion matrix of the best cell (test, rows = true digit): the diagonal counts are 35, 35, 34, 37, 34, 37, 35, 36, 32, 35 (350 of 360 correct). The errors are isolated: digit 8 was mistaken for 1 twice and for 5 once; digits 0, 1, 2, 6 and 9 each lost one case and digit 4 lost two (to 1 and to 9).

Provoked failures and diagnosis by per-layer gradient norms

Prediction before the run: a six-hidden-layer sigmoid network (32 units each, lr 0.1) will not learn in 60 epochs because each backward step multiplies by (W)^T and by f′(z) ≤ 0.25; the gradient norm of the first layers will be orders of magnitude smaller than the last layer's.

Network	Epoch	‖∂E/∂W‖ for layers 1 … 7	Ratio last/first	Result
Deep sigmoid, 6 hidden layers	1	1.4e-4, 4.0e-4, 1.5e-3, 6.5e-3, 2.9e-2, 1.4e-1, 5.6e-1	**4080**	Final loss 2.3087 (= ln 10), test accuracy **0.100**

Network	Epoch	‖∂E/∂W‖ for layers 1 … 7	Ratio last/first	Result
	60	1.3e-4, 3.4e-4, 1.3e-3, 5.9e-3, 2.5e-2, 1.1e-1, 3.6e-1	2776	
Deep ReLU, 6 hidden layers, He init	1	0.85, 0.71, 0.66, 0.68, 0.78, 0.69, 0.57	0.7	Final loss 0.0004, test accuracy 0.956
Sigmoid, hidden 64, lr 50	1–5	loss 17.85, 15.8, 18.78, 21.47 (does not fall)	–	Divergence: test accuracy 0.097
ReLU, hidden 64, lr 5.0	60	loss 2.35	–	100 % of hidden units dead (never active), test accuracy 0.103

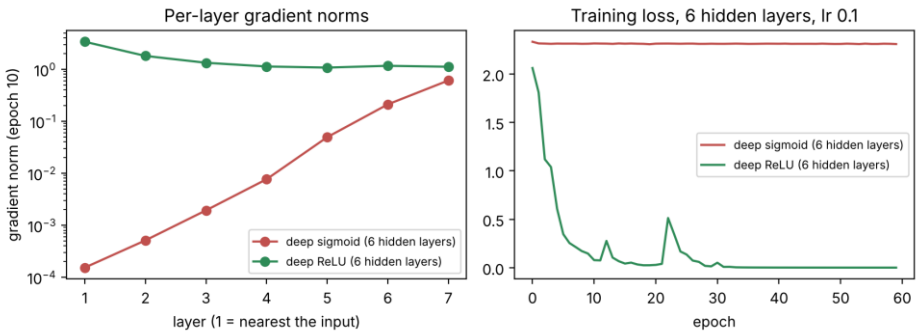


Figure 4.2 – Per-layer gradient norms: vanishing in the deep sigmoid network, balanced in the deep ReLU network.

Diagnosis. The prediction was right. In the deep sigmoid network the gradient norm falls by a factor of about 3–5 per layer going backwards (0.56 → 0.14 → 0.029 → 0.0065 → 0.0015 → 0.0004 → 0.00014), a ratio of roughly 4000 between output and input layers, so the early layers hardly change and the network stays at chance (loss ln 10 = 2.3026, accuracy 0.100). The same depth with ReLU and He initialisation has a ratio of 0.3–0.7 and trains to 0.956: ReLU's slope is 1 for active units and He scaling keeps the signal variance constant. The two divergence cases show different mechanisms: lr 50 with sigmoid makes the loss grow (overshoot), and lr 5 with ReLU drives all hidden units to negative pre-activations (**dead ReLU**: zero slope, zero gradient, no recovery), which is why the loss freezes at 2.35. **Repairs:** ReLU or other non-saturating units with He initialisation, smaller learning rate or gradient clipping, and monitoring the per-layer gradient norm as a routine diagnostic.

Task 5 – Same Dataset, Different Learning Reasoning (8 marks)

Scenario. A smart-grid operator has labelled records of substation states and wants to detect hazardous overload conditions. Three learners are compared that make three different assumptions: Naive Bayes (features independent given the class), a kernel SVM (a margin in an implicit feature space) and explanation-based learning (a domain theory plus one explained example).

5.1 Statistical learning with complete data: Naive Bayes

[Design] Pseudocode

```
ALGORITHM NaiveBayesTrain(Rows, Labels, FEATURES, VALUES, CLASSES, ALPHA)
INPUT : Rows - n rows of categorical feature values; Labels; FEATURES = list of j features;
        VALUES[j] = number of distinct values of feature j; CLASSES; ALPHA >= 0 (Laplace constant)
OUTPUT: prior[c], likelihood[c][j][v]
BEGIN
  1   FOR every class c IN CLASSES DO  count_c[c] <- number of rows with label c  END FOR
  2   FOR every class c, feature j, value v DO  count[c][j][v] <- number of rows with label c AND value
v for feature j  END FOR
  3   FOR every class c DO
  4       prior[c] <- ( count_c[c] + ALPHA ) / ( n + ALPHA * |CLASSES| )
  5       FOR every feature j, value v DO
  6           likelihood[c][j][v] <- ( count[c][j][v] + ALPHA ) / ( count_c[c] + ALPHA * VALUES[j] )
  7       END FOR
  8   END FOR
  9   RETURN prior, likelihood
END

ALGORITHM NaiveBayesClassify(x, prior, likelihood)
BEGIN
  1   FOR every class c IN CLASSES DO
  2       score[c] <- LOG( prior[c] ) + SUM over features j of LOG( likelihood[c][j][ x[j] ] )     //
LOG(0) = minus infinity
  3   END FOR
  4   IF every score[c] is minus infinity THEN RETURN the class with the largest prior          //
undefined case
  5   m <- MAX of score ; p[c] <- EXP( score[c] - m ) / SUM over c' of EXP( score[c'] - m )
  6   RETURN the class with the largest p[c], and the probabilities p
END
```

With ALPHA = 0 these are the maximum-likelihood counts; the log-sum form avoids underflow when there are many features; the "every score is −∞" rule is the explicit handling of the zero-count failure shown next.

[Design] Three-example toy table: why a zero count breaks classification

Example	Load	Temperature	Class
1	High	High	Hazard
2	High	Low	Normal
3	Low	Low	Normal

Query: (Load = Low, Temperature = High) – a combination never seen. **Maximum likelihood (α = 0):** P(Hazard) = 1/3, P(Normal) = 2/3; P(Low | Hazard) = 0/1 = **0**, P(High temp | Hazard) = 1/1; P(Low | Normal) = 1/2, P(High temp | Normal) = 0/2 = **0**. score(Hazard) = 1/3 × 0 × 1 = 0 and score(Normal) = 2/3 × 1/2 × 0 = 0: **both classes score zero, so the classification is undefined**; one unseen value vetoes a class however strong the other evidence is.

Laplace α = 1: P(Hazard) = (1+1)/(3+2) = 0.4, P(Normal) = 0.6. P(Low | Hazard) = (0+1)/(1+2) = 1/3, P(High temp | Hazard) = (1+1)/(1+2) = 2/3 → score(Hazard) = 0.4 × 1/3 × 2/3 = **0.0889**. P(Low | Normal) =

(1+1)/(2+2) = 1/2, P(High temp | Normal) = (0+1)/(2+2) = 1/4 → score(Normal) = 0.6 × 1/2 × 1/4 = **0.0750**. Hazard wins (normalised probability 0.54): a defined answer. The smoothing adds α pseudo-observations to every value, trading a small bias for the removal of impossible zeros.

[Implement] Experiment

Data. A synthetic substation dataset (`grid_data` in `t5_run.py`): six features (load ratio, transformer temperature, voltage deviation, harmonic distortion, ambient temperature, age); a state is hazardous when the pair (load, temperature) lies inside an elliptical overload band (about 23 % of states), with 4 % label noise; the other four features are weak or irrelevant. This data is deliberately **not linearly separable** (it is used again in 5.2). Each feature is discretised into 8 equal-frequency bins with edges computed from the training part only. A fixed 400-state test set (seed 11) is used for every training size. **Expected behaviour:** α = 0 can be as good as α = 1 when data are plentiful and bad when data are few; a very large α (10) should over-smooth and lower recall.

Train rows	α	Accuracy	Precision	Recall	F1	Log-loss	Test rows with no defined class
60	0	0.802	0.639	0.520	0.573	2.652	0
60	0.1	0.797	0.627	0.510	0.562	0.518	0
60	1	0.790	0.598	0.539	0.567	0.449	0
60	10	0.795	0.750	0.294	0.423	0.501	0
200	0	0.853	0.821	0.539	0.651	0.885	0
200	0.1	0.853	0.821	0.539	0.651	0.389	0
200	1	0.853	0.821	0.539	0.651	0.365	0
200	10	0.833	0.949	0.363	0.525	0.424	0
1000	0	0.910	0.867	0.765	0.812	0.326	0
1000	0.1	0.910	0.867	0.765	0.812	0.326	0
1000	1	0.912	0.876	0.765	0.817	0.325	0
1000	10	0.905	0.890	0.716	0.793	0.344	0

Interpretation. (1) Accuracy hardly depends on α (0.790–0.802 at 60 rows), but **log-loss does**: 2.652 at α = 0 against 0.449 at α = 1 for 60 rows. With α = 0 the model is wrong with probability 0 or 1 in some bins and is punished heavily when such a bin is misleading; the clipping at 10^-15 used in the log-loss calculation shows how large the penalty is. (2) The gap closes as data grow: at 1000 rows α = 0, 0.1 and 1 give the same log-loss (0.326, 0.326, 0.325) because every bin has been seen. (3) α = 10 raises precision (0.750, 0.949, 0.890) but lowers recall (0.294, 0.363, 0.716): the heavy pseudo-counts pull the likelihoods toward uniform, and the model becomes reluctant to say “hazard”. (4) In this test no query ended with both scores zero (the last column is 0) because with 8 quantile bins every bin was seen in training; the failure of the toy table therefore appears for sparse categorical data, and the implementation handles it by falling back to the prior. (5) Even at 1000 rows the F1 of 0.817 is limited: the hazard region depends on the *product* of two features, and Naive Bayes assumes the features contribute independently.

5.2 Change the assumption: kernel machines

[Design] Which assumption becomes unsafe

Naive Bayes assumes conditional independence of the features given the class; a linear classifier assumes that the classes can be separated by a hyperplane in the given feature space. For the substation data the

hazard is the *interior of a region in (load, temperature)*: each feature alone is uninformative at the edges (a high load is hazardous only when the temperature is also in the band), so the independence assumption is unsafe, and no line separates an ellipse from its surroundings, so the linearity assumption is unsafe too.

> **Margin, support vectors and kernel**
>
> **Margin.** The SVM chooses the separating surface with the largest distance to the nearest training points, because a wide margin bounds the generalisation error independently of the number of features. With noise it is a **soft margin**: C weights the slack penalty; large C punishes every violation (narrow margin, risk of overfitting), small C allows violations (wide margin, risk of underfitting).
>
> **Support vectors.** Only the training points on or inside the margin have non-zero dual weight α_i; the decision function $f(x) = \Sigma_i \alpha_i y_i K(x_i, x) + b$ depends on these points alone, so their number (a) measures model complexity and (b) bounds the leave-one-out error (error ≤ #SV / n).
>
> **Kernel function.** $K(x, x') = \phi(x)\cdot\phi(x')$ is an inner product in a feature space that is never constructed. The RBF kernel $K = \exp(-\gamma\|x - x'\|^2)$ corresponds to infinitely many features: γ sets how local the influence of a point is (large γ → narrow bumps, wiggly boundary).

[Implement] Experiment

Protocol. Five different splits of the same generator (train 600, test 400 each; seeds 100–104 / 200–204); every model sees an identical split within a split. All six features are standardised (fitted on training data) for the SVMs. Hyper-parameters by 5-fold stratified CV on the training part with F1 of the hazard class: linear SVM $C \in \{0.01, 0.1, 1, 10, 100\}$; RBF SVM $C \in \{0.1, 1, 10, 100\} \times \gamma \in \{0.01, 0.1, 1, 10\}$; Naive Bayes $\alpha \in \{0.1, 1, 10\}$; Task 2 tree max_depth $\in \{2, 3, 4, 5, 6, 8\}$. **Expected behaviour:** linear SVM near the majority rate (0.75), the RBF SVM and the tree well above it.

Model	Accuracy (mean ± sd)	F1 (hazard)	Support vectors	Fit time (ms)	Grid search (s)	Chosen hyper-parameters (5 splits)
Linear SVM	0.774 ± 0.016	**0.000**	302	4.2	0.39	C = 0.01 in all 5
RBF SVM	0.892 ± 0.014	0.762	235	11.0	0.87	(C, γ) = (100, .01), (10, .1) ×3, (100, .1)
Naive Bayes (binned)	0.894 ± 0.018	0.735	–	0.2	–	α = 0.1 in all 5
Decision tree (Task 2 learner)	0.911 ± 0.015	0.803	–	7.8	–	max_depth = 4 in all 5

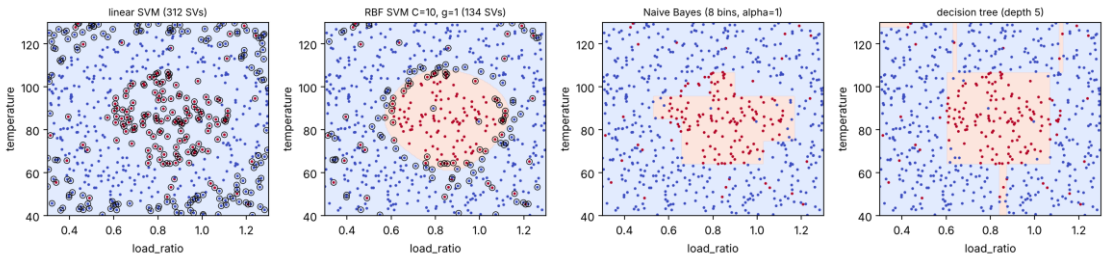


Figure 5.1 – Decision boundaries on (load ratio, temperature) only: linear SVM, RBF SVM, Naive Bayes and the decision tree; circled points are support vectors.

Interpretation. (1) The linear SVM chose the smallest C and **never predicts a hazard** (F1 = 0.000, accuracy 0.774 ≈ the share of normal states): the best line is "always normal". This confirms the design reasoning. (2) The RBF SVM reaches 0.892 with 235 support vectors (39 % of the training rows): a model that needs many support vectors to wrap a region and is therefore sensitive to the noise labels. (3) Naive Bayes on bins is as accurate (0.894) though its F1 is lower (0.735): binning lets it approximate the band marginally,

but it cannot represent the interaction. (4) The tree is best (0.911, F1 0.803) because the hazard is an axis-aligned-friendly region (a depth-4 tree approximates an ellipse by rectangles) and trees model feature interactions directly. The ranking depends on the data, which was generated to be an ellipse; the RBF SVM would be expected to win on smoother boundaries and the tree on rectangles. (5) Costs: Naive Bayes trains in 0.2 ms, the tree in 7.8 ms and the RBF SVM in 11 ms plus 0.87 s of grid search (16 settings × 5 folds), so the kernel machine is the most expensive and has the most hyper-parameters. The differences between RBF, NB and the tree (0.892–0.911) are of the order of one standard deviation (0.014–0.018) and should not be over-read; the decisive finding is the failure of the linear model.

5.3 Change the requirement: explanation-based learning

[Design] Domain theory (9 first-order Horn rules)

Rule	Statement
R1	Hazardous(s) ← FireHazard(s)
R2	Hazardous(s) ← Overheating(s) ∧ NearResidential(s)
R3	FireHazard(s) ← ArcRisk(s) ∧ OilFilled(s)
R4	ArcRisk(s) ← InsulationStress(s) ∧ Humid(s)
R5	InsulationStress(s) ← Overheating(s) ∧ AgedInsulation(s)
R6	Overheating(s) ← HighLoad(s) ∧ CoolingFault(s)
R7	Overheating(s) ← HighTemp(s) ∧ PoorVentilation(s)
R8	CoolingFault(s) ← FanFailed(s)
R9	CoolingFault(s) ← PumpFailed(s)

Operationality criterion: a predicate is operational if it can be observed directly on a substation: FanFailed, PumpFailed, HighLoad, HighTemp, PoorVentilation, AgedInsulation, Humid, OilFilled, NearResidential. The intermediate predicates (Overheating, CoolingFault, InsulationStress, ArcRisk, FireHazard) are not operational.

Training example (one explained hazardous case): Hazardous(T7) with the observed facts FanFailed(T7), HighLoad(T7), AgedInsulation(T7), Humid(T7), OilFilled(T7).

Explanation tree (proof of the example from the domain theory)

```
Hazardous(T7)                        by R1
  FireHazard(T7)                     by R3
    ArcRisk(T7)                      by R4
      InsulationStress(T7)           by R5
        Overheating(T7)              by R6
          HighLoad(T7)               fact
          CoolingFault(T7)           by R8
            FanFailed(T7)            fact
        AgedInsulation(T7)           fact
      Humid(T7)                      fact
    OilFilled(T7)                    fact
```

Generalisation (regression through the proof). The constant T7 is replaced by a variable x in the goal Hazardous(x). The goal is unified with the head of R1 (s := x); the body FireHazard(x) is expanded by R3 with a freshly renamed s, and so on down the same rules R3, R4, R5, R6, R8. Because every rule head is unified with the already-generalised goal, all the variables are bound to the same x. Every operational leaf

is kept and the constants that were only coincidentally equal are not retained: here only T7 appears, and it becomes x everywhere. Result:

> **Learned rule**
>
> **Hazardous(x) ← HighLoad(x) ∧ FanFailed(x) ∧ AgedInsulation(x) ∧ Humid(x) ∧ OilFilled(x)**

This is a single rule that contains only operational predicates and applies to every substation. Pseudocode of the procedure:

```
ALGORITHM EBL(Goal, Facts, Theory, Operational)
BEGIN
  1   tree <- PROVE(Goal, Facts, Theory)                          // explanation of the training
example
  2   IF tree = FAILURE THEN RETURN NO RULE
  3   general_goal <- Goal with its constants replaced by new variables
  4   leaves <- REGRESS(tree, general_goal, empty substitution)
  5   RETURN rule  general_goal <- conjunction of leaves after applying the final substitution
END
FUNCTION REGRESS(node, goal, theta)
  1   IF node is a fact leaf THEN RETURN [goal], theta
  2   rename the variables of the rule used at node to fresh variables
  3   theta <- UNIFY(goal, head of the renamed rule, theta)
  4   FOR every body literal b of the renamed rule with its sub-explanation child DO
  5       (sub_leaves, theta) <- REGRESS(child, b, theta) ; APPEND sub_leaves to leaves
  6   END FOR
  7   RETURN leaves, theta
```

[Implement] Test on 20 designed cases

`ebl.py` contains the domain theory, a depth-first prover that counts steps (every fact look-up and every rule-head unification attempt counts as one step) and the regression procedure; the learned rule is applied by looking up its five literals one after another (each look-up is one step, stopping at the first failure). The 20 cases: **S1–S6** repeat the training pattern; **S7–S10** have PumpFailed instead of FanFailed (hazardous by the theory via R9); **S11–S12** are hazardous by R2 (HighTemp, PoorVentilation, NearResidential); **S13–S14** are hazardous by R7 (HighTemp, PoorVentilation, AgedInsulation, Humid, OilFilled); **S15–S20** are random small fact sets, none hazardous.

Cases	Theory proves Hazardous	Theory steps (per case)	Learned rule fires	Rule steps (per case)
S1–S6 (training pattern)	yes ×6	17	yes ×6	5
S7–S10 (PumpFailed)	yes ×4	19	no ×4	2
S11–S12 (route R2)	yes ×2	23	no ×2	1
S13–S14 (route R7)	yes ×2	17	no ×2	1
S15–S20 (random, non-hazardous)	no ×6	29, 19, 19, 21, 21, 31	no ×6	2, 1, 1, 1, 1, 2
Total	14 hazardous	**398**	6 fired	**50**

Results. Agreement between the learned rule and the theory: **12 of 20**; false positives 0; **false negatives 8** (all the hazardous cases reached by another route). Total inference effort: 398 steps for the domain theory versus 50 steps for the learned rule, a speed-up of **8.0×** over all 20 cases; on the six training-pattern cases 102 steps versus 30 (3.4×). **Interpretation:** for the pattern it covers, the rule is much cheaper: five look-ups instead of a nine-rule proof that backtracks through alternatives (R2 before R1, R7 before R6, R9 after R8).

What was learned that the domain theory did not already entail?

Nothing. The learned rule is a logical consequence of the domain theory (it was derived by regressing a proof through the theory), so every case it classifies as hazardous is hazardous by the theory (0 false positives); no new fact about the world was acquired.

What was gained is operationality and speed: a direct test on observable facts (8.0× fewer steps overall), obtained from a single example. In addition, the learned rule records *which combination of facts was sufficient* (a cached, reusable shortcut), a form of knowledge compilation.

What was lost is coverage: the rule is one conjunction from a single explanation, so it misses the alternative routes (PumpFailed, R2, R7): 8 of 14 hazardous cases. EBL is therefore *incomplete* with respect to the theory; to recover them more examples (one per route) or the theory itself would be needed. If the theory is wrong, EBL faithfully compiles the error as well.

Task 6 – Design Challenge: Reinforcement Learning for an Energy-Aware Smart Controller (13 marks)

Stage 1 – Interpret the problem (2 marks)

Element	Specification
Input (what the controller observes)	Time of day (15-minute step 0…95), indoor temperature, occupancy (occupied or empty) and, through the clock, the price slot. Outdoor temperature and the occupancy schedule are not observed directly; they influence the indoor temperature and the clock.
Output (action)	One of Off, Low, High every 15 minutes (electrical power 0, 1, 2 kW; cooling effect 0, 0.45, 0.9 °C per step in the simulator).
Objective	Minimise the total electricity cost over the day while keeping the hall comfortable whenever it is occupied: indoor temperature within 22–26 °C.
Constraints	Only three discrete actions; the effect of an action is delayed (temperature changes slowly, so cooling now is worth something later); occupancy starts at about 18:00 (± 1 h) and ends at about 08:00 (± 45 min); the price is 3× higher in the 17:00–21:00 peak; the controller must learn from interaction, with no complete model of the building given.
Decisions to be made	At each step: whether to cool, how hard, and whether to pre-cool the empty hall before it fills or during cheap hours.
Episode	Begins at 00:00 with a random indoor temperature (25–29 °C) and a random outdoor level (28–34 °C base); ends after 96 steps (24 h). Episodes are independent days.

Stage 2 – Propose the strategy (2 marks)

Alternative	Fits the problem?	Verdict
Fixed-rule thermostat	Simple, safe and price-blind: it reacts after the temperature leaves the band and cannot pre-cool before arrival or avoid the peak.	Kept as the **baseline**, not the solution.
Supervised imitation of past operators	Needs a log of good decisions; it copies the operators' habits, including their mistakes, and has no notion of cost or consequence; it cannot exceed the operators.	Rejected.
Q-learning (off-policy TD control)	Decisions have delayed consequences (cooling now helps at 18:00) and the building model is unknown; Q-learning learns the value of each (state, action) pair directly from experience; discrete small state and action sets make a table feasible.	**Chosen.**
SARSA (on-policy TD control)	Same family; learns the value of the policy actually followed including exploration; safer when exploration is dangerous.	Implemented as the comparison.
Model-based (learn the transition model, then plan)	More sample-efficient if the model is accurate, but needs a model of the thermal dynamics, occupancy and weather and a planner; an inaccurate model is a new source of error.	Rejected for this prototype; named as the next step.

Reasoning for Q-learning. (i) Sequential decisions with delayed reward need a method that propagates future cost backwards (temporal-difference learning), which supervised learning and the thermostat do not do. (ii) The environment dynamics are not given and a day can be simulated or replayed cheaply, so model-free learning from episodes is applicable. (iii) The state is small (216 states, 3 actions), so a table converges and can be inspected line by line: an engineer can read why the controller acts, which matters for building operators. (iv) Exploration is cheap in simulation, so the off-policy advantage (learning the

greedy policy while exploring) is useful; SARSA is also run to test whether on-policy learning changes the result.

Stage 3 – Critical information (2 marks)

Design item	Choice and reason
State	s = (time block, temperature bin, occupancy). Time block = step ÷ 8 (12 blocks of 2 h); temperature bins from the edges 22, 23, 24, 25, 26, 27, 28, 30 °C (9 bins, finer around the comfort band); occupancy ∈ {0,1}. **216 states. Markov enough:** the next temperature depends on the current temperature, the action and the outdoor temperature, which is a smooth function of the clock; the next price and occupancy depend on the clock. The block, the bin and the occupancy therefore carry nearly all the information needed to predict the next state and reward; the day's outdoor level and the exact arrival jitter are hidden, which makes the problem only approximately Markov (tested in the adversarial runs).
Actions	0 = Off, 1 = Low (1 kW), 2 = High (2 kW).
Reward	r_t = −(price_t × kWh_t) − λ × V_t, where kWh_t = kW × 0.25 h, price = 1 (3 in 17:00–21:00), V_t = 1 if the hall is occupied and the temperature is outside 22–26 °C, and **λ = 2**. Cost and discomfort are on one scale; since the largest cost per step is 0.5 × 3 = 1.5, λ = 2 makes one violation worse than the most expensive cooling step, even in the peak.
Exploration	ε-greedy: ε_t = max(0.05, 0.995^episode): about 1 at the start, 0.05 after 600 days; with probability ε a uniformly random action. Q is initialised to 0, which is optimistic because every reward is ≤ 0 and so also encourages trying every action.
Discount	γ = 0.99 (15-minute steps; the effective horizon of about 100 steps is one day, so the effect of pre-cooling hours before the occupants arrive is still visible). Learning rate α = 0.1.

Stage 4 – Complete pseudocode (3 marks)

```
ALGORITHM QLearningController(ENV, EPISODES, ALPHA, GAMMA, EPS_START, EPS_DECAY, EPS_MIN, SEED)
INPUT : ENV - the environment (reset, step); EPISODES - number of training days; ALPHA - learning rate (0 <
ALPHA <= 1);
        GAMMA - discount (0 <= GAMMA < 1); EPS_START = 1.0; EPS_DECAY = 0.995; EPS_MIN = 0.05; SEED
OUTPUT: table Q[s][a] and the greedy policy POLICY[s]
BEGIN
  1   SET random generator seed to SEED
  2   FOR every state s IN STATES AND every action a IN {Off, Low, High} DO  Q[s][a] <- 0  END FOR      //
initialisation
  3   FOR episode <- 0 TO EPISODES - 1 DO                                                              //
interaction loop
  4       epsilon <- MAX( EPS_MIN , EPS_START * EPS_DECAY ^ episode )
  5       s <- ENCODE( ENV.reset(episode) )                       // new day: random start temperature, outdoor
level, jitter
  6       done <- FALSE
  7       WHILE NOT done DO
  8           IF RANDOM() < epsilon THEN                          // action selection with exploration
  9               a <- a random action chosen uniformly
 10           ELSE
 11               a <- the action with the largest Q[s][a] (ties: the lowest action number)
 12           END IF
 13           (observation, reward, done) <- ENV.step(a)          // 15 minutes pass
 14           s_next <- ENCODE(observation)
 15           IF done THEN  target <- reward                      // terminal step: no future value
 16           ELSE          target <- reward + GAMMA * MAX over a' of Q[s_next][a']
 17           END IF
 18           Q[s][a] <- Q[s][a] + ALPHA * ( target - Q[s][a] )   // value update
 19           s <- s_next
 20       END WHILE
 21   END FOR                                                      // termination: fixed number of episodes
 22   FOR every state s DO  POLICY[s] <- the action with the largest Q[s][a]  END FOR     // extraction of the
final policy
 23   RETURN Q, POLICY
END


FUNCTION ENCODE(obs): block <- obs.step DIV 8 ; bin <- number of edges in (22,23,24,25,26,27,28,30) that are <=
obs.temperature ;
                      RETURN (block * 9 + bin) * 2 + obs.occupied
FUNCTION Evaluate(POLICY, ENV, DAYS): FOR each of DAYS fresh days run the greedy policy (epsilon = 0, no
update) and RETURN the mean of
                      reward, kWh, cost and comfort violations per day
```

SARSA differs only on lines 9–16: the next action a_next is chosen (with exploration) before the update and the target is reward + GAMMA × Q[s_next][a_next]. The fixed-rule baseline is: if occupied then (High if T > 26, Low if T > 24.5, else Off); if empty then (Low if T > 30, else Off).

Stage 5 – Desk-check (1 mark)

Toy environment: states s0 (hot, empty), s1 (hot, occupied), s2 (comfortable, occupied); actions 0 = Off, 1 = Cool. α = 0.5, γ = 0.9, all Q = 0. A sequence of seven transitions is executed by hand and was reproduced by the program (Appendix B.6). The update is $Q(s,a) \leftarrow Q(s,a) + \alpha\,[\, r + \gamma \max_{a'} Q(s',a') - Q(s,a)\,]$.

k	State s	Action a	Reward r	Next s'	Old Q(s,a)	max Q(s',·)	Target r + 0.9·max	TD error	New Q(s,a)
1	s0	Cool	−1.0	s1	0	0	−1.0000	−1.0000	**−0.5000**
2	s1	Cool	−0.5	s2	0	0	−0.5000	−0.5000	**−0.2500**
3	s2	Off	+1.0	s2	0	0	1.0000	1.0000	**0.5000**
4	s2	Cool	−0.5	s2	0	0.5	−0.0500	−0.0500	**−0.0250**
5	s1	Off	−3.0	s0	0	0	−3.0000	−3.0000	**−1.5000**
6	s0	Cool	−1.0	s1	−0.5	−0.25	−1.2250	−0.7250	**−0.8625**
7	s1	Cool	−0.5	s2	−0.25	0.5	−0.0500	0.2000	**−0.1500**

Check of update 6 by hand: the state s0, action Cool was updated in step 1 to −0.5. Now r = −1.0 and max Q(s1,·) = max(Q(s1,Off) = −1.5, Q(s1,Cool) = −0.25) = −0.25. Target = −1.0 + 0.9 × (−0.25) = −1.225; TD error = −1.225 − (−0.5) = −0.725; new value = −0.5 + 0.5 × (−0.725) = **−0.8625**. **Final Q table:** s0: (Off 0, Cool −0.8625); s1: (Off −1.5, Cool −0.15); s2: (Off 0.5, Cool −0.025). **Greedy policy:** s0 → Off, s1 → Cool, s2 → Off. It is sensible: in s1 (hot and occupied) Off led to a −3.0 penalty, so Cool is preferred; in s2 (comfortable) doing nothing is best. Updates 4 and 7 show **bootstrapping**: the target uses the estimate Q(s2, Off) = 0.5 learned in update 3.

Stage 6 – Implement, instrument and attack the design (3 marks)

Simulator and set-up

`rl.py` implements the environment `ACEnv` (96 steps per day): indoor temperature $T_{t+1} = T_t + 0.08\,(T_{out,t} - T_t) - \text{cooling}(a) + N(0, 0.05^2)$; T_{out} = base + 5 sin(2π (t − 40) / 96) with base uniform in 28–34 °C; occupancy from about 18:00 (arrival 72 ± 4 steps) until about 08:00 (departure 32 ± 3 steps); price 1.0, and 3.0 in steps 68–83 (17:00–21:00). Training: 1,500 days per seed, 20 seeds (0–19) for each of Q-learning and SARSA; evaluation: the greedy policy on 40 held-out days (day seeds 900000–900039, never used in training), the same days for the baseline and both learners. Hyper-parameters (8 temperature bins instead of 5, γ = 0.99, ε-decay 0.995, 1,500 days) were chosen on three separate tuning seeds (100–102), after a first configuration (400 days, γ = 0.97) failed to beat the baseline; the 20 evaluation seeds were not used for tuning.

> **Predictions written before the 20-seed run (printed by the program)**
>
> **P1:** the Q-learning controller will use less cost than the fixed rule by moving cooling away from the 17:00–21:00 peak, with similar or slightly higher comfort violations; expected saving 10–40 %. **P2:** SARSA will converge to a similar level but a little lower reward than Q-learning. **P3 (adversarial):** the policy keys on clock-time blocks: if the peak is moved to 13:00–17:00 the learner keeps cooling at the new peak and loses most of its advantage; with a +6 °C heat wave it enters bins above 30 °C that were rare in training and its comfort violations will rise by more than the baseline's.

Results over 20 seeds

Controller	Reward / day	kWh / day	Cost / day	Comfort violations / day	Episodes to converge (median)
Fixed-rule baseline (40 days)	−59.60	18.54	31.50	14.05	–
Q-learning (20 seeds)	−58.31 ± 6.81	26.75 ± 4.73	37.79 ± 7.37	**10.26 ± 5.36**	175
SARSA (20 seeds)	−57.00 ± 4.75	27.91 ± 2.39	39.73 ± 3.62	8.64 ± 3.01	200

Mean ± standard deviation across seeds (each seed's value is the mean over the 40 held-out days); "episodes to converge" is the first evaluation point (every 50 training days) at which the held-out reward is within 5 % of that seed's final value. **Statistics:** Q-learning versus baseline reward: mean difference +1.29 (better), one-sample t-test over the 20 seeds p = 0.42, **14 of 20 seeds** beat the baseline; Q-learning versus SARSA (paired by seed): difference −1.31, p = 0.48. Cost change against the baseline +20.0 %, energy +44.3 %, comfort violations −3.79 steps/day.

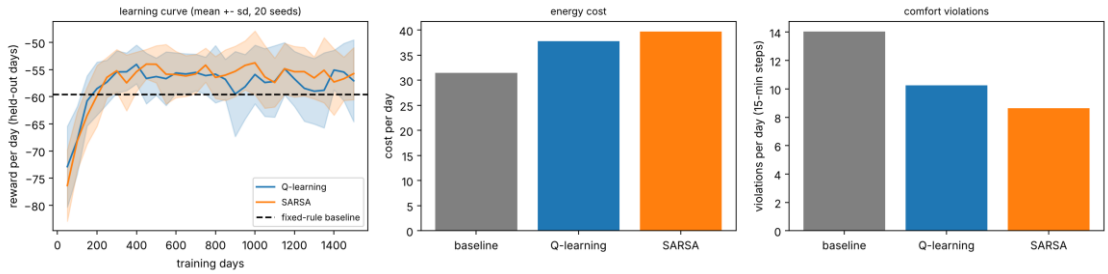


Figure 6.1 – Left: held-out reward per day against training days (mean ± sd over 20 seeds) against the baseline. Middle and right: cost and comfort violations.

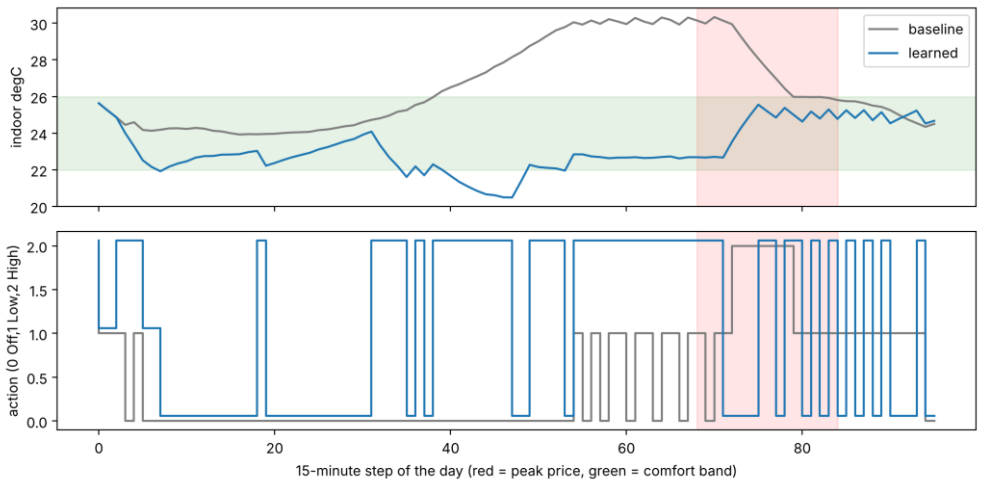


Figure 6.2 – One held-out day: indoor temperature and action of the baseline and of a learned policy (seed 0). Red = peak price; green = comfort band.

Interpretation of the baseline comparison

- **P1 was wrong.** The learner did not save cost: it spent 20 % **more** (and 44 % more energy) than the fixed rule. It reduced violations from 14.05 to 10.26 steps per day, so the learner's reward is better by 1.29 only because comfort was bought with energy, and with a spread of 6.8 across seeds the

improvement is **not statistically significant** ($p = 0.42$). The honest conclusion is that the learner found a different trade-off, not a clearly better one.

- **Why it behaves like this (Figure 6.2).** The baseline lets the empty hall heat up to 30 °C and then cools hard after the occupants arrive; because the room warms $0.08 \times (T_{out} - T)$ per step and High cooling removes 0.9 °C per step, it needs several steps to bring the room back, so it violates comfort for the first hour or more. The learner pre-cools the empty hall (it reached 20.5 °C around step 45, below the band), which avoids most arrival violations but uses energy while nobody is there. It also does **not** avoid the peak: it keeps cooling at 17:00–21:00, since the hall fills at about 18:00 and one violation ($\lambda = 2$) costs more than one High step in the peak ($0.5 \text{ kWh} \times 3 = 1.5$). With this reward the high price cannot force load shifting; a weaker λ or a higher peak price would.
- **P2 was wrong** in direction: SARSA's mean reward (−57.00) is slightly higher than Q-learning's (−58.31), but the paired difference is not significant ($p = 0.48$); its violations are lower (8.64) and its cost higher (39.73), a more cautious, comfort-first policy, as an on-policy learner that experiences its own exploration would be. Both converge in about 175–200 training days.

Adversarial test (designed by me)

Attack. The learner's table is indexed by the clock. Three changes were made to the world after training, using the same 20 trained Q-learning tables and 40 evaluation days each: (a) a **+6 °C heat wave**; (b) the price **peak moved from 17:00–21:00 to 13:00–17:00** (an unusual price pattern); (c) both. The fixed-rule baseline is evaluated in the same scenarios. **Prediction (P3, above, written before the run):** (b) removes the learner's cost advantage; (a) raises the learner's violations by more than the baseline's.

Scenario	Baseline cost	Learner cost	Baseline violations	Learner violations	Baseline reward	Learner reward	Seeds learner better
Normal	31.50	37.79	14.05	10.26	−59.60	−58.31	14 / 20
Heat wave +6 °C	44.77	28.58*	35.02	44.84	−114.83	−109.82	16 / 20
Peak moved to 13–17 h	25.01	40.65	14.05	10.26	−53.11	−61.18	**1 / 20**
Both	38.80	26.70*	35.02	44.84	−108.85	−116.38	3 / 20

* In the heat-wave runs the learner's cost is lower than the baseline's, but its comfort violations are higher (44.84 against 35.02 steps/day); the reward combines both and is better for the learner in 16 of 20 seeds only because comfort is penalised at $\lambda = 2$ per step while the baseline's hard cooling is expensive in the heat. Cost, violations and reward must be read together.

What happened and which assumption failed. (b) confirmed: with the peak moved the learner's cost rose to 40.65 against 25.01 for the baseline and the learner beat the baseline in **1 of 20** seeds (reward −61.18 against −53.11). The baseline is price-blind, so moving the peak cost it less than before (31.50 → 25.01), because its cooling happens to fall mostly outside 13–17 h; the learner's cooling pattern was tuned to the old peak and is now concentrated at the new, expensive hours. The **failed assumption is stationarity of the price pattern encoded in the clock-based state**: the time block stands in for the price, and the Markov property is lost when the mapping from clock to price changes. **(a) confirmed:** in the heat wave the learner's violations rose from 10.26 to 44.84 (+34.6) while the baseline's rose from 14.05 to 35.02 (+21.0): the temperature bins above 30 °C were rarely visited in training, so those Q-values are essentially the initial zeros and the controller does not learn to use High cooling there. The **failed assumption is coverage of the state space by the training distribution**. **Repairs:** include the price (and a forecast of arrival time) as explicit state variables so the policy generalises across tariff schedules; randomise the weather and tariff

during training (domain randomisation); add a safety layer that overrides the learner with the thermostat rule if the temperature leaves a safe range; monitor violations online.

Limitations of this prototype (stated honestly). One simulated hall, hand-set physics and λ; a tabular learner on a coarse state; a single attack family (price and weather shifts). The "switch off before arrival" exploit suggested in the assignment was not seen: the learned policy did the opposite and pre-cooled the empty hall; the reward charges comfort only when occupied, so a policy that leaves the hall hot until 17:59 and then cools hard would be penalised by the arrival-time violations in the simulator, which is exactly what the baseline does.

References

[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2021 (chapters 19–22: learning from examples, knowledge in learning, probabilistic models, reinforcement learning).

[2] J. R. Quinlan, "Induction of decision trees," *Machine Learning*, vol. 1, pp. 81–106, 1986; and *C4.5: Programs for Machine Learning*, Morgan Kaufmann, 1993 (gain ratio).

[3] A. P. Dempster, N. M. Laird and D. B. Rubin, "Maximum likelihood from incomplete data via the EM algorithm," *J. Royal Statistical Society B*, vol. 39, pp. 1–38, 1977.

[4] D. E. Rumelhart, G. E. Hinton and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, pp. 533–536, 1986.

[5] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, pp. 273–297, 1995.

[6] T. M. Mitchell, R. M. Keller and S. T. Kedar-Cabelli, "Explanation-based generalization: a unifying view," *Machine Learning*, vol. 1, pp. 47–80, 1986.

[7] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, pp. 279–292, 1992; R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.

[8] F. Pedregosa et al., "Scikit-learn: machine learning in Python," *JMLR*, vol. 12, pp. 2825–2830, 2011 (used for verification and comparison only).

Appendix A – Source Code

All sources, the raw result files (`t*_results.json`), the tuning script `tune6.py` and the figures are in the accompanying zip file. Every experiment was run with `PYTHONHASHSEED=0` and fixed seeds.

A.1 Decision-tree learner from scratch: entropy, Gini, gain ratio, numeric splits, pruning (Tasks 2, 5.2) (dt.py)

```
"""Decision-tree learner written from scratch: ID3/C4.5 style, categorical (multiway) and numeric (binary threshold) attributes.
criterion: 'entropy' (information gain), 'gini' (Gini decrease), 'gainratio' (C4.5 repair for many-valued attributes)."""
import numpy as np
from collections import Counter


def entropy_from_counts(c):
    c = np.asarray(c, float); n = c.sum()
    if n == 0: return 0.0
    p = c[c > 0] / n
    return float(-(p * np.log2(p)).sum())
def gini_from_counts(c):
    c = np.asarray(c, float); n = c.sum()
    if n == 0: return 0.0
    return float(1.0 - ((c / n) ** 2).sum())
IMP = {'entropy': entropy_from_counts, 'gainratio': entropy_from_counts, 'gini': gini_from_counts}


class Node:
    __slots__ = ('attr', 'thr', 'children', 'label', 'counts', 'depth', 'is_leaf', 'default')
    def __init__(s): s.attr = None; s.thr = None; s.children = {}; s.label = None; s.counts = None; s.depth = 0; s.is_leaf = True;
s.default = None


class DecisionTree:
    def __init__(self, criterion='entropy', max_depth=None, min_samples_split=2, min_samples_leaf=1, types=None, verbose=False,
names=None):
        self.criterion = criterion; self.max_depth = max_depth; self.min_samples_split = min_samples_split; self.min_samples_leaf =
min_samples_leaf
        self.types = types; self.verbose = verbose; self.names = names; self.root = None; self.log = []
    # ------------------------------------------------------------------ training
    def fit(self, X, y, classes=None):
        X = np.asarray(X, dtype=object); y = np.asarray(y)
        self.classes_ = np.array(sorted(set(y.tolist()))) if classes is None else np.array(classes)
        self.cidx = {c: i for i, c in enumerate(self.classes_)}
        yi = np.array([self.cidx[c] for c in y]); d = X.shape[1]
        if self.types is None: self.types = ['num'] * d
        if self.names is None: self.names = [f'x{j}' for j in range(d)]
        self.root = self._build(X, yi, list(range(d)), 0, parent_label=int(np.bincount(yi, minlength=len(self.classes_)).argmax()))
        return self
    def _counts(self, yi): return np.bincount(yi, minlength=len(self.classes_))
    def _build(self, X, yi, attrs, depth, parent_label):
        node = Node(); node.depth = depth
        if len(yi) == 0:                                          # stopping condition 1: no examples -> parent's majority
            node.label = parent_label; node.counts = np.zeros(len(self.classes_), int); return node
        cnt = self._counts(yi); node.counts = cnt; node.label = int(cnt.argmax()); node.default = node.label
        imp = IMP[self.criterion]
        if (cnt > 0).sum() == 1: return node                      # stopping condition 2: pure
        cat_left = [a for a in attrs if self.types[a] == 'cat']; num_attrs = [a for a in attrs if self.types[a] == 'num']
        if not cat_left and not num_attrs: return node            # stopping condition 3: no attributes
        if (self.max_depth is not None and depth >= self.max_depth) or len(yi) < self.min_samples_split: return node
        best = None; H = imp(cnt); n = len(yi); cand = []
        for a in attrs:
            if self.types[a] == 'cat':
                vals = {}
                for i in range(n): vals.setdefault(X[i, a], []).append(i)
                if len(vals) < 2: continue
                rem = 0.0; si = 0.0
                for v, idx in vals.items():
                    w = len(idx) / n; rem += w * imp(self._counts(yi[idx])); si -= w * np.log2(w)
                gain = H - rem; score = gain / si if (self.criterion == 'gainratio' and si > 1e-12) else gain
                cand.append((a, None, gain, score, len(vals)))
            else:
                col = np.array([float(v) for v in X[:, a]]); order = np.argsort(col, kind='mergesort'); cs = col[order]; ys = yi[order]
                k = len(self.classes_); onehot = np.zeros((n, k)); onehot[np.arange(n), ys] = 1; left = np.cumsum(onehot, axis=0)
                tot = left[-1]
                valid = np.where(cs[:-1] < cs[1:])[0]
                valid = valid[(valid + 1 >= self.min_samples_leaf) & (n - valid - 1 >= self.min_samples_leaf)]
                if len(valid):
                    lc = left[valid]; rc = tot - lc; nl = (valid + 1).astype(float); nr = n - nl
                    def vimp(c, m):
                        p = c / m[:, None]
                        if self.criterion == 'gini': return 1.0 - (p ** 2).sum(axis=1)
                        with np.errstate(divide='ignore', invalid='ignore'): return -np.where(p > 0, p * np.log2(p), 0.0).sum(axis=1)
                    rem = nl / n * vimp(lc, nl) + nr / n * vimp(rc, nr); gains = H - rem
                    sis = -(nl / n * np.log2(nl / n) + nr / n * np.log2(nr / n))
                    scores = gains / np.where(sis > 1e-12, sis, 1) if self.criterion == 'gainratio' else gains
                    j = int(np.argmax(scores))                                  # first best threshold
                    cand.append((a, (cs[valid[j]] + cs[valid[j] + 1]) / 2.0, float(gains[j]), float(scores[j]), 2))
        if not cand: return node
        for c in cand:
            if best is None or c[3] > best[3] + 1e-12: best = c      # strict improvement: ties go to the first attribute listed
        if self.verbose:
            by = {}
            for c in cand:
                if c[0] not in by or c[3] > by[c[0]][3] + 1e-12: by[c[0]] = c
            self.log.append((depth, n, cnt.tolist(), H, [(self.names[a], by[a][2], by[a][3]) for a in by], self.names[best[0]]))
        if best[2] <= 1e-12: return node                              # no information gain -> leaf
        a, thr = best[0], best[1]; node.attr = a; node.thr = thr; node.is_leaf = False
        if thr is None:
            rest = [b for b in attrs if b != a]
            for v in sorted({X[i, a] for i in range(n)}, key=str):
```

```
                idx = [i for i in range(n) if X[i, a] == v]
                node.children[v] = self._build(X[idx], yi[idx], rest, depth + 1, node.label)
        else:
            col = np.array([float(v) for v in X[:, a]]); l = np.where(col <= thr)[0]; r = np.where(col > thr)[0]
            node.children['<='] = self._build(X[l], yi[l], attrs, depth + 1, node.label); node.children['>'] = self._build(X[r], yi[r],
attrs, depth + 1, node.label)
        return node
    # ------------------------------------------------------------------ prediction and inspection
    def _leaf(self, node, x):
        while not node.is_leaf:
            if node.thr is None:
                nxt = node.children.get(x[node.attr]);
                if nxt is None: return node
                node = nxt
            else: node = node.children['<='] if float(x[node.attr]) <= node.thr else node.children['>']
        return node
    def predict(self, X):
        X = np.asarray(X, dtype=object); return np.array([self.classes_[self._leaf(self.root, x).label] for x in X])
    def predict_proba(self, X):
        X = np.asarray(X, dtype=object); out = []
        for x in X:
            c = self._leaf(self.root, x).counts.astype(float); out.append(c / c.sum() if c.sum() else np.ones(len(c)) / len(c))
        return np.array(out)
    def depth(self, node=None):
        node = node or self.root
        return 0 if node.is_leaf else 1 + max(self.depth(c) for c in node.children.values())
    def n_leaves(self, node=None):
        node = node or self.root
        return 1 if node.is_leaf else sum(self.n_leaves(c) for c in node.children.values())
    def text(self, node=None, indent=''):
        node = node or self.root
        if node.is_leaf: return f"{indent}-> {self.classes_[node.label]}  {node.counts.tolist()}\n"
        s = ''
        for k, c in node.children.items():
            cond = f"{self.names[node.attr]} = {k}" if node.thr is None else f"{self.names[node.attr]} {k} {node.thr:.4g}"
            s += f"{indent}[{cond}]\n" + self.text(c, indent + '    ')
        return s
    def reduced_error_prune(self, Xv, yv):
        """bottom-up: replace a subtree by its majority leaf when this does not lower the validation accuracy"""
        Xv = np.asarray(Xv, dtype=object); yi = np.array([self.cidx[c] for c in yv])
        def acc(): return float((np.array([self._leaf(self.root, x).label for x in Xv]) == yi).mean())
        def rec(node):
            if node.is_leaf: return
            for c in node.children.values(): rec(c)
            before = acc(); saved = (node.is_leaf, node.children, node.attr, node.thr)
            node.is_leaf = True; node.children = {}
            if acc() < before: node.is_leaf, node.children, node.attr, node.thr = saved
        rec(self.root); return self
```

A.2 EM for a 1-D Gaussian mixture, faulty and repaired (Task 3) (em.py)

```
"""1-D Gaussian mixture EM: the faulty pseudocode (no normalisation of the responsibilities) and the repaired version"""
import numpy as np
def normal(x, mu, var): return np.exp(-(x - mu) ** 2 / (2 * var)) / np.sqrt(2 * np.pi * var)
def loglik(X, pi, mu, var):
    dens = sum(pi[k] * normal(X, mu[k], var[k]) for k in range(len(pi)))
    return float(np.log(np.maximum(dens, 1e-300)).sum())
def em(X, pi, mu, var, T, repaired, var_floor=1e-6, trace=False):
    X = np.asarray(X, float); n = len(X); K = len(pi); pi = np.array(pi, float); mu = np.array(mu, float); var = np.array(var, float)
    hist = dict(ll=[loglik(X, pi, mu, var)], spi=[pi.sum()], mu=[mu.copy()], var=[var.copy()], log=[])
    for t in range(T):
        r = np.zeros((n, K))
        for i in range(n):
            for k in range(K): r[i, k] = pi[k] * normal(X[i], mu[k], var[k])
        if repaired: r = r / r.sum(axis=1, keepdims=True)           # THE REPAIR: normalise over k so that sum_k r[i][k] = 1 for every
point i
        Nk = r.sum(axis=0); mu_new = (r * X[:, None]).sum(axis=0) / Nk
        var_new = np.maximum((r * (X[:, None] - mu_new[None, :]) ** 2).sum(axis=0) / Nk, var_floor); pi_new = Nk / n
        if trace: hist['log'].append(dict(r=r.copy(), Nk=Nk.copy(), mu=mu_new.copy(), var=var_new.copy(), pi=pi_new.copy()))
        mu, var, pi = mu_new, var_new, pi_new
        hist['ll'].append(loglik(X, pi, mu, var)); hist['spi'].append(pi.sum()); hist['mu'].append(mu.copy());
hist['var'].append(var.copy())
    hist['final'] = (pi, mu, var); return hist
```

A.3 Back-propagation network, numerical gradients, training (Task 4) (nn.py)

```
"""Fully connected neural network in NumPy (column convention: W[l] has shape (n_l, n_{l-1}), a batch is a matrix with one COLUMN per
example)"""
import numpy as np
ACT = {'sigmoid': (lambda z: 1 / (1 + np.exp(-np.clip(z, -500, 500))), lambda a, z: a * (1 - a)),
       'relu': (lambda z: np.maximum(z, 0), lambda a, z: (z > 0).astype(float)),
       'tanh': (lambda z: np.tanh(z), lambda a, z: 1 - a ** 2),
       'linear': (lambda z: z, lambda a, z: np.ones_like(z))}
def softmax(z): e = np.exp(z - z.max(axis=0, keepdims=True)); return e / e.sum(axis=0, keepdims=True)
class MLP:
    def __init__(self, sizes, hidden='sigmoid', out='sigmoid', loss='mse', seed=0, init=None):
        self.sizes = sizes; self.hidden = hidden; self.out = out; self.loss = loss; rng = np.random.RandomState(seed); self.W = []; self.b
= []
        for l in range(1, len(sizes)):
            fan_in, fan_out = sizes[l - 1], sizes[l]
            scale = np.sqrt(2.0 / fan_in) if (init or ('he' if hidden == 'relu' else 'xavier')) == 'he' else np.sqrt(2.0 / (fan_in +
fan_out))
            self.W.append(rng.randn(fan_out, fan_in) * scale); self.b.append(np.zeros((fan_out, 1)))
    def forward(self, X):
        """X: (n_0, m).  returns lists z[l], a[l] with a[0] = X"""
        a = [X]; z = [None]; L = len(self.W)
        for l in range(L):
            zl = self.W[l] @ a[-1] + self.b[l]; z.append(zl)
            if l == L - 1: al = softmax(zl) if self.out == 'softmax' else ACT[self.out][0](zl)
            else: al = ACT[self.hidden][0](zl)
            a.append(al)
        return z, a
```

```
    def loss_value(self, a_out, Y):
        m = Y.shape[1]
        if self.loss == 'mse': return float(0.5 * ((a_out - Y) ** 2).sum() / m)
        return float(-(Y * np.log(np.clip(a_out, 1e-12, 1))).sum() / m)
    def backward(self, z, a, Y):
        """returns dW, db and the error terms delta[l]  (delta^(l) = dLoss/dz^(l), loss averaged over the batch)"""
        L = len(self.W); m = Y.shape[1]; delta = [None] * (L + 1)
        if self.loss == 'ce' and self.out == 'softmax': delta[L] = (a[L] - Y)                       # boundary condition: softmax + cross-
entropy
        elif self.loss == 'mse': delta[L] = (a[L] - Y) * ACT[self.out][1](a[L], z[L])               # boundary condition: (a - y) . f'(z)
        else: raise ValueError('unsupported loss/output pair')
        for l in range(L - 1, 0, -1): delta[l] = (self.W[l].T @ delta[l + 1]) * ACT[self.hidden][1](a[l], z[l])   # recurrence, last layer
to first
        dW = [delta[l + 1] @ a[l].T / m for l in range(L)]; db = [delta[l + 1].sum(axis=1, keepdims=True) / m for l in range(L)]
        return dW, db, delta
    def predict(self, X): return self.forward(X)[1][-1]
    def get_params(self): return np.concatenate([p.ravel() for p in self.W + self.b])
    def set_params(self, v):
        i = 0
        for p in self.W + self.b: p[...] = v[i:i + p.size].reshape(p.shape); i += p.size
def grad_vector(dW, db): return np.concatenate([g.ravel() for g in dW + db])
def numerical_gradient(net, X, Y, eps=1e-6):
    v = net.get_params(); g = np.zeros_like(v)
    for i in range(len(v)):
        old = v[i]; v[i] = old + eps; net.set_params(v); lp = net.loss_value(net.forward(X)[1][-1], Y)
        v[i] = old - eps; net.set_params(v); lm = net.loss_value(net.forward(X)[1][-1], Y); v[i] = old; g[i] = (lp - lm) / (2 * eps)
    net.set_params(v); return g
def train(net, X, Y, epochs, lr, batch=32, seed=0, Xv=None, Yv=None, record_grads=(), stop_loss=None):
    """mini-batch gradient descent.  X: (n_0, N).  returns history"""
    rng = np.random.RandomState(seed); N = X.shape[1]; hist = dict(loss=[], val_loss=[], val_acc=[], gnorm={})
    for ep in range(epochs):
        order = rng.permutation(N); tot = 0.0
        for s in range(0, N, batch):
            idx = order[s:s + batch]; z, a = net.forward(X[:, idx]); tot += net.loss_value(a[-1], Y[:, idx]) * len(idx)
            dW, db, delta = net.backward(z, a, Y[:, idx])
            if ep in record_grads and s == 0: hist['gnorm'][ep] = [float(np.linalg.norm(g)) for g in dW]
            for l in range(len(net.W)): net.W[l] -= lr * dW[l]; net.b[l] -= lr * db[l]
        hist['loss'].append(tot / N)
        if Xv is not None:
            av = net.predict(Xv); hist['val_loss'].append(net.loss_value(av, Yv)); hist['val_acc'].append(float((av.argmax(0) ==
Yv.argmax(0)).mean()) if Yv.shape[0] > 1 else float(((av > .5) == (Yv > .5)).mean()))
        if stop_loss is not None and hist['loss'][-1] < stop_loss: break
        if not np.isfinite(hist['loss'][-1]): break
    return hist
```

A.4 First-order logic engine: unification and substitution used by EBL (Task 5.3) (fol.py)

```
"""
CSA17 AI Assignment 3 - first-order logic engine (Tasks 3-7)
Terms : variable '?x' (string starting with '?'), constant (any other str/int), compound ('f', t1, ..., tn)
Atoms : ('Pred', t1, ..., tn).  Rule = (head, [body literals]); a body literal is an atom or ('not', atom) (negation as failure).
"""
import itertools, time, sys, itertools
sys.setrecursionlimit(20000)


def is_var(t): return isinstance(t, str) and t.startswith('?')
def is_compound(t): return isinstance(t, tuple)


# ---------------------------------------------------------------- substitution and unification
def walk(t, th):
    while is_var(t) and t in th: t = th[t]
    return t
def subst(t, th):
    t = walk(t, th)
    if is_compound(t): return (t[0],) + tuple(subst(a, th) for a in t[1:])
    return t
def occurs(v, t, th):
    t = walk(t, th)
    if t == v: return True
    if is_compound(t): return any(occurs(v, a, th) for a in t[1:])
    return False
class UStat:
    def __init__(self): self.calls = 0; self.bindings = 0; self.occurs_checks = 0
def unify(x, y, th, occurs_check=True, stat=None):
    """returns an (extended) substitution dict or None.  th is not modified."""
    if stat is not None: stat.calls += 1
    return _unify(x, y, dict(th), occurs_check, stat)
def _unify(x, y, th, oc, stat):
    if th is None: return None
    x = walk(x, th); y = walk(y, th)
    if x == y: return th
    if is_var(x): return _bind(x, y, th, oc, stat)
    if is_var(y): return _bind(y, x, th, oc, stat)
    if is_compound(x) and is_compound(y):
        if x[0] != y[0] or len(x) != len(y): return None
        for a, b in zip(x[1:], y[1:]):
            th = _unify(a, b, th, oc, stat)
            if th is None: return None
        return th
    return None
def _bind(v, t, th, oc, stat):
    if oc:
        if stat is not None: stat.occurs_checks += 1
        if occurs(v, t, th): return None
    th[v] = t
    if stat is not None: stat.bindings += 1
    return th
def compose(s1, s2):
    """s1 then s2:  apply(apply(t, s1), s2) == apply(t, compose(s1, s2))"""
    r = {v: subst(t, s2) for v, t in s1.items()}
    for v, t in s2.items():
        if v not in r: r[v] = t
```

```
    return {v: t for v, t in r.items() if v != t}
def mgu_full(x, y, occurs_check=True):
    th = unify(x, y, {}, occurs_check)
    if th is None: return None
    try: return {v: subst(v, th) for v in th}
    except RecursionError: return 'CYCLIC'                    # fully resolved (idempotent) form
def tstr(t):
    if is_compound(t): return t[0] + '(' + ', '.join(tstr(a) for a in t[1:]) + ')'
    return str(t)
def sstr(th):
    if th == 'CYCLIC': return 'CYCLIC binding (infinite term, e.g. ?x = f(f(f(...))))'
    return '{' + ', '.join(f'{v}/{tstr(t)}' for v, t in sorted(th.items())) + '}' if th else '{}  (empty)'
def variables(t, acc=None):
    acc = [] if acc is None else acc
    if is_var(t):
        if t not in acc: acc.append(t)
    elif is_compound(t):
        for a in t[1:]: variables(a, acc)
    return acc
_cnt = itertools.count(1)
def rename_rule(rule):
    """standardise apart: fresh variable names for a rule (head, body)"""
    head, body = rule; m = {}
    def ren(t):
        if is_var(t):
            if t not in m: m[t] = f'{t}_{next(_cnt)}'
            return m[t]
        if is_compound(t): return (t[0],) + tuple(ren(a) for a in t[1:])
        return t
    return ren(head), [(ren(b) if b[0] != 'not' else ('not', ren(b[1]))) for b in body]


# ---------------------------------------------------------------- knowledge base with indexes
class KB:
    def __init__(self, rules=(), facts=(), index2=False):
        self.index2 = index2; self.facts_by_key2 = {}
        self.rules = []; self.rules_by_pred = {}; self.facts = set(); self.facts_by_pred = {}; self.facts_by_key = {}
        for r in rules: self.add_rule(r)
        for f in facts: self.add_fact(f)
    def add_rule(self, r):
        self.rules.append(r); self.rules_by_pred.setdefault(r[0][0], []).append(r)
    def add_fact(self, f):
        if f in self.facts: return False
        self.facts.add(f); self.facts_by_pred.setdefault(f[0], []).append(f)
        if len(f) > 1: self.facts_by_key.setdefault((f[0], f[1]), []).append(f)
        if self.index2 and len(f) > 2: self.facts_by_key2.setdefault((f[0], f[2]), []).append(f)
        return True
    def candidates(self, goal):
        """index on predicate and, when bound, first argument"""
        if len(goal) > 1 and not is_var(goal[1]) and not is_compound(goal[1]): return self.facts_by_key.get((goal[0], goal[1]), [])
        if self.index2 and len(goal) > 2 and not is_var(goal[2]) and not is_compound(goal[2]): return self.facts_by_key2.get((goal[0],
goal[2]), [])
        return self.facts_by_pred.get(goal[0], [])


# ---------------------------------------------------------------- backward chaining with substitutions
class BCStat:
    def __init__(self): self.goals = 0; self.unifications = 0; self.rule_tries = 0; self.max_depth = 0; self.repeated = 0; self.solutions =
0; self.facts_matched = 0; self.cut = 0; self.time = 0.0; self.exhausted = False


def variant_key(goal, th):
    g = subst(goal, th); m = {}
    def f(t):
        if is_var(t): return m.setdefault(t, f'_{len(m)}')
        if is_compound(t): return (t[0],) + tuple(f(a) for a in t[1:])
        return t
    return f(g)


class Prover:
    """Backward chaining (SLD resolution with negation as failure on ground literals).
       mode: 'plain' | 'loopcheck' | 'depth' (iterative/limit) ; goal-count budget stops runaway searches."""
    def __init__(self, kb, mode='plain', depth_limit=None, budget=2_000_000, occurs_check=False, index=True, trace=False):
        self.kb = kb; self.mode = mode; self.depth_limit = depth_limit; self.budget = budget; self.oc = occurs_check
        self.index = index; self.stat = BCStat(); self.seen_goals = {}; self.trace = trace
    def prove_all(self, goals, th, depth, ancestors):
        if not goals: yield th, []; return
        first, rest = goals[0], goals[1:]
        for th1, p1 in self.prove(first, th, depth, ancestors):
            for th2, p2 in self.prove_all(rest, th1, depth, ancestors): yield th2, [p1] + p2
    def prove(self, goal, th, depth, ancestors):
        st = self.stat
        if st.goals >= self.budget: st.exhausted = True; return
        if goal[0] == 'not':
            g = subst(goal[1], th)
            if any(is_var(a) for a in g[1:]): raise ValueError('floundering: non-ground negative literal ' + str(g))
            sub = Prover(self.kb, self.mode, self.depth_limit, 200000, self.oc, self.index)
            ok = any(True for _ in sub.prove(g, {}, 0, ()))
            st.goals += sub.stat.goals; st.unifications += sub.stat.unifications
            if not ok: yield th, ('naf', g)
            return
        if goal[0] == 'Ge':                                   # built-in arithmetic comparison (both arguments must be numbers)
            a, b = subst(goal[1], th), subst(goal[2], th)
            if isinstance(a, (int, float)) and isinstance(b, (int, float)):
                st.goals += 1
                if a >= b: yield th, ('fact', ('Ge', a, b))
                return
            raise ValueError('floundering: Ge needs bound numbers: ' + str((a, b)))
        st.goals += 1; st.max_depth = max(st.max_depth, depth)
        key = variant_key(goal, th)
        if key in self.seen_goals: st.repeated += 1
        self.seen_goals[key] = self.seen_goals.get(key, 0) + 1
        if self.depth_limit is not None and depth >= self.depth_limit: st.cut += 1; return
        if self.mode == 'loopcheck' and key in ancestors: st.cut += 1; return
        anc = ancestors | {key} if self.mode == 'loopcheck' else ancestors
        g = subst(goal, th)
        facts = self.kb.candidates(g) if self.index else self.kb.facts_by_pred.get(g[0], [])
        for f in facts:
            st.unifications += 1
```

```
            th1 = unify(g, f, th, self.oc)
            if th1 is not None: st.facts_matched += 1; yield th1, ('fact', f)
        for rule in (self.kb.rules_by_pred.get(g[0], []) if self.index else [r for r in self.kb.rules if r[0][0] == g[0]]):
            st.rule_tries += 1; head, body = rename_rule(rule); st.unifications += 1
            th1 = unify(g, head, th, self.oc)
            if th1 is None: continue
            for th2, subs in self.prove_all(body, th1, depth + 1, anc): yield th2, ('rule', rule, subs)
    def ask(self, goal, limit=None):
        t0 = time.perf_counter(); out = []; seen = set()
        for th, proof in self.prove(goal, {}, 0, frozenset()):
            ans = subst(goal, th)
            if ans not in seen: seen.add(ans); out.append((ans, proof))
            if limit and len(out) >= limit: break
        self.stat.time = time.perf_counter() - t0; self.stat.solutions = len(out); return out


def proof_size(p):
    if p is None: return 0
    if p[0] in ('fact', 'naf'): return 1
    return 1 + sum(proof_size(x) for x in p[2])
def proof_lines(p, indent=0, out=None):
    out = [] if out is None else out
    if p[0] == 'fact': out.append('  ' * indent + 'fact   ' + tstr(p[1]))
    elif p[0] == 'naf': out.append('  ' * indent + 'not    ' + tstr(p[1]) + '  (no proof found: negation as failure)')
    else:
        h, b = p[1]; out.append('  ' * indent + 'rule   ' + tstr(h) + ' :- ' + ', '.join(tstr(x) if x[0] != 'not' else 'not ' + tstr(x[1])
for x in b))
        for s in p[2]: proof_lines(s, indent + 1, out)
    return out


# ---------------------------------------------------------------- forward chaining
class FCStat:
    def __init__(self): self.iterations = 0; self.firings = 0; self.new_facts = 0; self.redundant = 0; self.unifications = 0; self.time =
0.0; self.initial = 0
def match_body(kb, body, th, stat, delta=None, used_delta=False):
    if not body:
        if delta is None or used_delta: yield th
        return
    lit, rest = body[0], body[1:]
    if lit[0] == 'not':
        g = subst(lit[1], th)
        if g not in kb.facts: yield from match_body(kb, rest, th, stat, delta, used_delta)
        return
    g = subst(lit, th)
    if g[0] == 'Ge':                                          # built-in comparison, both arguments must be bound numbers
        if isinstance(g[1], (int, float)) and isinstance(g[2], (int, float)) and g[1] >= g[2]:
            yield from match_body(kb, rest, th, stat, delta, used_delta)
        return
    for f in kb.candidates(g):
        stat.unifications += 1
        th1 = unify(g, f, th, False)
        if th1 is not None: yield from match_body(kb, rest, th1, stat, delta, used_delta or (delta is not None and f in delta))
def forward_chain(kb, seminaive=True, max_iter=100000):
    """Applies all rules until no new fact is produced.  Facts are added to kb.  semi-naive: a rule instance must use a fact derived in the
previous iteration."""
    st = FCStat(); t0 = time.perf_counter(); st.initial = len(kb.facts); delta = None
    while st.iterations < max_iter:
        st.iterations += 1; new = []
        for rule in kb.rules:
            head, body = rule
            for th in match_body(kb, body, {}, st, delta if seminaive else None):
                st.firings += 1; h = subst(head, th)
                if h in kb.facts or h in new: st.redundant += 1
                else: new.append(h)
        if not new: break
        for f in new: kb.add_fact(f)
        st.new_facts += len(new); delta = set(new)
    st.time = time.perf_counter() - t0; return st


# ---------------------------------------------------------------- propositionalisation (instantiate every rule with all ground terms)
def ground_terms(constants, functions, depth):
    """functions = {name: arity}.  returns list of ground terms of nesting depth <= depth"""
    terms = list(constants); level = list(constants)
    for _ in range(depth):
        new = []
        for fname, ar in functions.items():
            for args in itertools.product(terms, repeat=ar):
                t = (fname,) + args
                if t not in terms and t not in new: new.append(t)
        terms += new
    return terms
def propositionalise(rules, terms, facts=()):
    """ground instances of every rule (variables replaced by all combinations of terms).  returns list of (head, body) ground clauses"""
    out = []
    for head, body in rules:
        vs = []
        for a in [head] + [b if b[0] != 'not' else b[1] for b in body]: variables(a, vs)
        for combo in itertools.product(terms, repeat=len(vs)):
            th = dict(zip(vs, combo)); out.append((subst(head, th), [subst(b, th) if b[0] != 'not' else ('not', subst(b[1], th)) for b in
body]))
    return out
```

A.5 Explanation-based learning: domain theory, prover, regression (Task 5.3) (ebl.py)

```
"""Explanation-based learning on a Horn-clause domain theory (own small prover for ground facts + generalisation of the explanation)"""
from fol import unify, subst, tstr, rename_rule
THEORY = [
 (('Hazardous', '?s'), [('FireHazard', '?s')]),                                          # R1
 (('Hazardous', '?s'), [('Overheating', '?s'), ('NearResidential', '?s')]),              # R2  (a second, different route)
 (('FireHazard', '?s'), [('ArcRisk', '?s'), ('OilFilled', '?s')]),                       # R3
 (('ArcRisk', '?s'), [('InsulationStress', '?s'), ('Humid', '?s')]),                     # R4
 (('InsulationStress', '?s'), [('Overheating', '?s'), ('AgedInsulation', '?s')]),        # R5
 (('Overheating', '?s'), [('HighLoad', '?s'), ('CoolingFault', '?s')]),                  # R6
 (('Overheating', '?s'), [('HighTemp', '?s'), ('PoorVentilation', '?s')]),               # R7
```

```
 (('CoolingFault', '?s'), [('FanFailed', '?s')]),                                        # R8
 (('CoolingFault', '?s'), [('PumpFailed', '?s')]),                                       # R9
]
OPERATIONAL = {'FanFailed', 'PumpFailed', 'HighLoad', 'HighTemp', 'PoorVentilation', 'AgedInsulation', 'Humid', 'OilFilled',
'NearResidential'}   # directly observable facts
class Stat:
    def __init__(s): s.steps = 0
def prove(goal, facts, stat, depth=0):
    """first proof (depth-first) of a ground goal from the domain theory; returns an explanation tree or None.  steps = rule heads tried +
facts looked up"""
    stat.steps += 1
    if goal in facts: return ('fact', goal)
    if depth > 12: return None
    for i, (h, body) in enumerate(THEORY):
        if h[0] != goal[0]: continue
        stat.steps += 1                                                    # head unification attempt
        th = unify(h, goal, {}, True)
        if th is None: continue
        kids = []
        for b in body:
            k = prove(subst(b, th), facts, stat, depth + 1)
            if k is None: break
            kids.append(k)
        else: return ('rule', i, goal, kids)
    return None
def explanation_lines(t, ind=0, out=None):
    out = [] if out is None else out
    if t[0] == 'fact': out.append('  ' * ind + 'fact  ' + tstr(t[1]))
    else:
        out.append('  ' * ind + f'R{t[1] + 1}    ' + tstr(t[2]));
        for k in t[3]: explanation_lines(k, ind + 1, out)
    return out
_cnt = [0]
def _gen(t, goal, th):
    if t[0] == 'fact': return [goal], th
    _, i, _, kids = t; h, body = THEORY[i]; _cnt[0] += 1; sfx = f'_{_cnt[0]}'
    ren = lambda a: tuple(x + sfx if isinstance(x, str) and x.startswith('?') else x for x in a)
    h2 = ren(h); body2 = [ren(b) for b in body]; th = unify(goal, h2, th, True); leaves = []
    for b, k in zip(body2, kids):
        l, th = _gen(k, b, th); leaves += l
    return leaves, th
def  resolve(l, th):
    for _ in range(30):
        n = subst(l, th)
        if n == l: break
        l = n
    return l
def generalise(t, goal, th):
    leaves, th = _gen(t, goal, th); out = [_resolve(l, th) for l in leaves]
    names = {}
    for l in out:
        v = l[1]
        names.setdefault(v, '?x')
    return [(l[0], '?x') for l in out] if len(names) == 1 else out
def learn_rule(example_goal, facts):
    st = Stat(); t = prove(example_goal, facts, st); _cnt[0] = 0; leaves = generalise(t, ('Hazardous', '?x'), {})
    return t, leaves
def apply_rule(leaves, head, facts, stat):
    """evaluate the learned rule on a set of ground facts for a substation constant s: every body literal is looked up"""
    var = leaves[0][1]
    for s in sorted({f[1] for f in facts}):
        ok = True
        for l in leaves:
            stat.steps += 1
            if (l[0], s) not in facts: ok = False; break
        if ok: yield s
```

A.6 Smart-AC simulator, Q-learning, SARSA, baseline, evaluation (Task 6) (rl.py)

```
"""Smart-AC simulator (15-minute steps, one day = 96 steps) and tabular learners"""
import numpy as np
ACT_KW = np.array([0.0, 1.0, 2.0]); ACT_COOL = np.array([0.0, 0.45, 0.9])    # Off / Low / High : electrical kW, degrees removed per step
LO, HI = 22.0, 26.0; LAMBDA = 2.0                                           # comfort band when occupied; penalty per occupied step outside
the band
EDGES = [22, 23, 24, 25, 26, 27, 28, 30]; NB = len(EDGES) + 1
class ACEnv:
    def __init__(self, seed, heat=0.0, peak=(68, 84), early_arrival=None):
        r = np.random.RandomState(seed); self.r = r; self.heat = heat; self.peak = peak
        self.arrive = (early_arrival if early_arrival is not None else 72) + r.randint(-4, 5); self.leave = 32 + r.randint(-3, 4)
        self.base = r.uniform(28, 34) + heat; self.T = r.uniform(25, 29); self.t = 0
    def tout(self, t): return self.base + 5 * np.sin(2 * np.pi * (t - 40) / 96)
    def occ(self, t): return int(t < self.leave or t >= self.arrive)
    def price(self, t): return 3.0 if self.peak[0] <= t < self.peak[1] else 1.0
    def state(self): tt = min(self.t, 95); return (tt // 8, int(np.searchsorted(EDGES, self.T)), self.occ(tt))
    def step(self, a):
        t = self.t; kwh = ACT_KW[a] * 0.25; cost = kwh * self.price(t)
        self.T += 0.08 * (self.tout(t) - self.T) - ACT_COOL[a] + self.r.normal(0, 0.05)
        viol = int(self.occ(t + 1 if t < 95 else 95) and (self.T < LO or self.T > HI)) if t < 95 else int(self.occ(95) and (self.T < LO or
self.T > HI))
        self.t += 1; return -(cost + LAMBDA * viol), kwh, cost, viol, self.t >= 96
def sid(s): return s[0] * NB * 2 + s[1] * 2 + s[2]
NS = 12 * NB * 2
def baseline_action(env):
    if env.occ(env.t): return 2 if env.T > 26 else (1 if env.T > 24.5 else 0)
    return 1 if env.T > 30 else 0
def run_day(env, policy):
    R = K = C = V = 0
    while True:
        a = policy(env); r, kwh, cost, viol, done = env.step(a); R += r; K += kwh; C += cost; V += viol
        if done: return R, K, C, V
def greedy_policy(Q): return lambda env: int(np.argmax(Q[sid(env.state())]))
def train(seed, episodes=400, alpha=0.1, gamma=0.99, algo='q', eval_every=50, n_eval=30, eps_decay=0.995, eps_min=0.05):
    r = np.random.RandomState(seed); Q = np.zeros((NS, 3)); curve = []; train_R = []
    for ep in range(episodes):
        eps = max(eps_min, eps_decay ** ep); env = ACEnv(1000 * seed + ep); s = sid(env.state()); a = r.randint(3) if r.rand() < eps else
int(np.argmax(Q[s])); R = 0
```

```
        while True:
            rew, _, _, _, done = env.step(a); s2 = sid(env.state()); R += rew
            a2 = r.randint(3) if r.rand() < eps else int(np.argmax(Q[s2]))
            tgt = rew + (0 if done else gamma * (Q[s2, a2] if algo == 'sarsa' else Q[s2].max()))
            Q[s, a] += alpha * (tgt - Q[s, a]); s, a = s2, a2
            if done: break
        train_R.append(R)
        if (ep + 1) % eval_every == 0: curve.append((ep + 1, evaluate(Q, n_eval)[0]))
    return Q, curve, train_R
def evaluate(Q, n=40, **kw):
    pol = greedy_policy(Q); m = np.array([run_day(ACEnv(900000 + i, **kw), pol) for i in range(n)]); return m.mean(0)
def evaluate_baseline(n=40, **kw):
    m = np.array([run_day(ACEnv(900000 + i, **kw), baseline_action) for i in range(n)]); return m.mean(0)
```

A.7 Task 1 experiment driver (t1_run.py)

```
import numpy as np, warnings; warnings.filterwarnings('ignore')
from sklearn.datasets import make_classification, load_breast_cancer
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score, GridSearchCV, cross_validate
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.dummy import DummyClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, balanced_accuracy_score, confusion_matrix
import json
out = {}
print("TASK 1 - Diagnosing and repairing the learning protocol\n")
def at_risk_data(seed, n=600):
    """synthetic student at-risk data: 12 features (6 informative), 10 % label noise, 15 % at-risk (imbalanced)"""
    X, y = make_classification(n_samples=n, n_features=12, n_informative=6, n_redundant=2, weights=[0.85, 0.15], flip_y=0.10,
class_sep=1.0, random_state=seed)
    return X, y
print("A  THE DEFECT: accuracy on the training data versus 5-fold cross-validation and a held-out test set (3 models x 5 data seeds)")
print("   protocol of the junior engineer: fit on all data, score on the same data.  CV = stratified 5-fold on the training part;  test =
untouched 25 % held-out.")
print(f"{'model':>22} {'seed':>4} | {'train acc':>9} {'CV acc':>7} {'test acc':>8} | {'optimism (train - test)':>23} | {'majority baseline
acc':>21} {'test recall (at-risk)':>21}")
models = {'unpruned decision tree': lambda s: DecisionTreeClassifier(random_state=s), '1-nearest-neighbour': lambda s:
KNeighborsClassifier(1), 'random forest (100 trees)': lambda s: RandomForestClassifier(100, random_state=s)}
rows = []
for name, mk in models.items():
    for seed in range(5):
        X, y = at_risk_data(seed); Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=seed)
        m = mk(seed).fit(Xtr, ytr); tr = accuracy_score(ytr, m.predict(Xtr)); te = accuracy_score(yte, m.predict(Xte))
        cv = cross_val_score(mk(seed), Xtr, ytr, cv=StratifiedKFold(5, shuffle=True, random_state=seed)).mean()
        base = accuracy_score(yte, DummyClassifier(strategy='most_frequent').fit(Xtr, ytr).predict(Xte)); rec = recall_score(yte,
m.predict(Xte))
        rows.append((name, seed, tr, cv, te, base, rec)); print(f"{name:>22} {seed:>4} | {tr:>9.3f} {cv:>7.3f} {te:>8.3f} | {tr -
te:>23.3f} | {base:>21.3f} {rec:>21.3f}")
print("\n   Mean over the 5 seeds:")
print(f"{'model':>22} | {'train acc':>9} {'CV acc':>7} {'test acc':>8} {'optimism':>9} {'test recall':>11}")
S = []
for name in models:
    r = [x for x in rows if x[0] == name]; a = np.mean([x[2] for x in r]), np.mean([x[3] for x in r]), np.mean([x[4] for x in r]),
np.mean([x[6] for x in r])
    print(f"{name:>22} | {a[0]:>9.3f} {a[1]:>7.3f} {a[2]:>8.3f} {a[0]-a[2]:>9.3f} {a[3]:>11.3f}"); S.append(dict(model=name, train=a[0],
cv=a[1], test=a[2], recall=a[3]))
out['defect'] = S
print("\n   Second dataset (breast cancer, 569 cases) for the unpruned tree:")
Xb, yb = load_breast_cancer(return_X_y=True); Xtr, Xte, ytr, yte = train_test_split(Xb, yb, test_size=0.25, stratify=yb, random_state=0)
m = DecisionTreeClassifier(random_state=0).fit(Xtr, ytr); cv = cross_val_score(DecisionTreeClassifier(random_state=0), Xtr, ytr,
cv=StratifiedKFold(5, shuffle=True, random_state=0)).mean()
print(f"   train {accuracy_score(ytr, m.predict(Xtr)):.3f}   5-fold CV {cv:.3f}   test {accuracy_score(yte, m.predict(Xte)):.3f}")
print("\nB  OTHER DEFICIENCIES MADE VISIBLE")
X, y = at_risk_data(0); Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=0)
dm = DummyClassifier(strategy='most_frequent').fit(Xtr, ytr); p = dm.predict(Xte)
print(f"   (a) accuracy alone on imbalanced data: a model that always says 'not at risk' has accuracy {accuracy_score(yte, p):.3f} but
recall {recall_score(yte, p):.3f}, F1 {f1_score(yte, p):.3f}  (at-risk share in the test set: {yte.mean():.3f})")
print("   (b) no control of randomness: the same unpruned tree trained with 5 random seeds gives test accuracies",
[round(accuracy_score(yte, DecisionTreeClassifier(max_features=6, random_state=s).fit(Xtr, ytr).predict(Xte)), 3) for s in range(5)])
print("   (c) hyper-parameters chosen on the test set (data snooping): for each of 5 splits pick the tree depth with the BEST TEST accuracy
and report it:")
sn = []; hon = []
for seed in range(5):
    X, y = at_risk_data(seed + 10); Xa, Xte, ya, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=seed); Xtr, Xva,
ytr, yva = train_test_split(Xa, ya, test_size=0.33, stratify=ya, random_state=seed)
    ds = [1, 2, 3, 4, 5, 6, 8, 10, None]; te = [accuracy_score(yte, DecisionTreeClassifier(max_depth=d, random_state=seed).fit(Xtr,
ytr).predict(Xte)) for d in ds]; va = [accuracy_score(yva, DecisionTreeClassifier(max_depth=d, random_state=seed).fit(Xtr,
ytr).predict(Xva)) for d in ds]
    sn.append(max(te)); hon.append(te[int(np.argmax(va))])
print(f"       snooped (max over depths of test accuracy): mean {np.mean(sn):.3f};   honest (depth chosen on a validation set, then tested
once): mean {np.mean(hon):.3f};   optimism {np.mean(sn) - np.mean(hon):+.3f}")
out['snoop'] = dict(snooped=float(np.mean(sn)), honest=float(np.mean(hon)))
print("\nC  THE REPAIRED PROTOCOL EXECUTED (stratified split, 5-fold CV model selection, test set used once, majority baseline)")
print("   candidates: DecisionTree with max_depth in {1,2,3,4,5,8} x min_samples_leaf in {1,5,10}; selection metric = F1 of the at-risk
class")
R = []
for seed in range(5):
    X, y = at_risk_data(seed); Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, stratify=y, random_state=seed)
    gs = GridSearchCV(DecisionTreeClassifier(random_state=seed), {'max_depth': [1, 2, 3, 4, 5, 8], 'min_samples_leaf': [1, 5, 10]},
scoring='f1', cv=StratifiedKFold(5, shuffle=True, random_state=seed)).fit(Xtr, ytr)
    pt = gs.predict(Xte); pb = DummyClassifier(strategy='most_frequent').fit(Xtr, ytr).predict(Xte)
    R.append(dict(seed=seed, params=gs.best_params_, cv_f1=gs.best_score_, acc=accuracy_score(yte, pt), prec=precision_score(yte, pt,
zero_division=0), rec=recall_score(yte, pt), f1=f1_score(yte, pt), bacc=balanced_accuracy_score(yte, pt), base_acc=accuracy_score(yte, pb),
base_f1=f1_score(yte, pb, zero_division=0), cm=confusion_matrix(yte, pt).tolist()))
    r = R[-1]; print(f"   seed {seed}: best {r['params']}  CV F1 {r['cv_f1']:.3f} | TEST acc {r['acc']:.3f} prec {r['prec']:.3f} recall
{r['rec']:.3f} F1 {r['f1']:.3f} bal.acc {r['bacc']:.3f} | baseline acc {r['base_acc']:.3f} F1 {r['base_f1']:.3f} | confusion {r['cm']}")
print(f"   mean test F1 {np.mean([r['f1'] for r in R]):.3f} (CV estimate {np.mean([r['cv_f1'] for r in R]):.3f}); majority baseline
accuracy {np.mean([r['base_acc'] for r in R]):.3f} with F1 0")
out['repaired'] = R
json.dump(out, open('t1 results.json', 'w'), indent=1, default=str)
```

A.8 Task 2 experiment driver (t2_run.py)

```
import numpy as np, json, time, warnings; warnings.filterwarnings('ignore')
from dt import DecisionTree, entropy_from_counts
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
out = {}
print("TASK 2 - Decision-tree learning: desk-check, implementation, verification, pruning, challenge\n")
NAMES = ['Attendance', 'Assignments', 'Internal']
T = [('High', 'Done', 'Good', 'Pass'), ('High', 'Done', 'Poor', 'Pass'), ('High', 'Missed', 'Good', 'Pass'), ('High', 'Missed', 'Poor',
'Fail'),
     ('Low', 'Done', 'Good', 'Pass'), ('Low', 'Done', 'Poor', 'Fail'), ('Low', 'Missed', 'Good', 'Fail'), ('Low', 'Missed', 'Poor',
'Fail')]
def run(table, names, label, crit='entropy', show=True):
    X = [r[:-1] for r in table]; y = [r[-1] for r in table]; t = DecisionTree(crit, types=['cat'] * len(X[0]), names=names,
verbose=True).fit(X, y)
    if show:
        print(f"{label}   (criterion = {crit})")
        for depth, n, cnt, H, gains, best in t.log:
            print(f"   {'  ' * depth}node depth {depth}: n = {n}, class counts [Fail, Pass] = {cnt}, entropy = {H:.4f}")
            for nm, g, sc in gains: print(f"   {'  ' * depth}   gain({nm}) = {g:.4f}" + (f"   score(ratio) = {sc:.4f}" if crit ==
'gainratio' else ''))
            print(f"   {'  ' * depth}   -> split on {best}")
        print(t.text());
    return t
print("A  DESK-CHECK TABLE - the algorithm's calculations (program output; the same numbers were derived by hand in Section 2.4)")
t = run(T, NAMES, "Original 8-row table")
print(f"   leaves {t.n_leaves()}, depth {t.depth()}; training accuracy {np.mean(t.predict([r[:-1] for r in T]) == np.array([r[-1] for r in
T])):.3f}")
print("   Observation: Pass is exactly 'at least two of the three signals are good' (the majority of three).  The minimal tree needs 6
leaves; all 3 root gains tie at 0.1887, so the root depends on the tie-break rule (first attribute listed).")
out['desk'] = dict(leaves=t.n_leaves(), depth=t.depth())
print("\nB  CHALLENGE 1 - change ONE value: row 4 (High, Missed, Poor) Fail -> Pass.   PREDICTION WRITTEN BEFORE THE RUN (from hand
calculation):")
print("   H(S) = 0.9544 (5 Pass, 3 Fail).  Attendance: High 4P/0F entropy 0, Low 1P/3F 0.8113 -> gain 0.5488.  Assignments and Internal:
gain 0.0488.  => the root changes from a 3-way tie to a CLEAR winner: Attendance.")
T2 = [r for r in T]; T2[3] = ('High', 'Missed', 'Poor', 'Pass')
t2 = run(T2, NAMES, "Modified table")
print(f"   verified: root = {NAMES[t2.root.attr]}; leaves {t2.n_leaves()}")
print("\nC  CHALLENGE 2 - an identifier-like attribute (unique value per row) on the 8-row table")
T3 = [(f'S{i+1}',) + r for i, r in enumerate(T)]; N3 = ['StudentID'] + NAMES
t3 = run(T3, N3, "With StudentID, information gain", 'entropy')
print(f"   -> root = {N3[t3.root.attr]}, gain {t3.log[0][4][0][1]:.4f} = H(S), leaves {t3.n_leaves()} (one per row): a memorising table.")
t4 = run(T3, N3, "With StudentID, gain ratio (C4.5 repair)", 'gainratio')
print(f"   -> root = {N3[t4.root.attr]}")
print("   On a larger synthetic table (300 train / 200 test rows, label = majority of three signals with 10 % noise) with an added unique
ID column:")
rng = np.random.RandomState(4); R = []
for seed in range(5):
    rng = np.random.RandomState(seed); n = 500
    A = rng.choice(['High', 'Low'], n); B = rng.choice(['Done', 'Missed'], n); C = rng.choice(['Good', 'Poor'], n)
    good = (A == 'High').astype(int) + (B == 'Done') + (C == 'Good'); lab = np.where(good >= 2, 'Pass', 'Fail'); flip = rng.rand(n) < 0.10;
lab = np.where(flip, np.where(lab == 'Pass', 'Fail', 'Pass'), lab)
    ids = np.array([f'ID{i}' for i in range(n)]); X = np.stack([ids, A, B, C], axis=1); Xtr, Xte, ytr, yte = X[:300], X[300:], lab[:300],
lab[300:]
    res = {}
    for nm, X1, X2, ty, na in (('no ID', Xtr[:, 1:], Xte[:, 1:], ['cat'] * 3, NAMES), ('ID, info gain', Xtr, Xte, ['cat'] * 4, N3), ('ID,
gain ratio', Xtr, Xte, ['cat'] * 4, N3)):
        crit = 'gainratio' if nm.endswith('ratio') else 'entropy'; tt = DecisionTree(crit, types=ty, names=na).fit(X1, ytr)
        res[nm] = (np.mean(tt.predict(X1) == ytr), np.mean(tt.predict(X2) == yte), tt.n_leaves(), na[tt.root.attr])
    R.append(res)
print(f"   {'variant':>16} | {'train acc':>9} {'test acc':>8} {'leaves':>6} {'root (seed 0)':>14}")
for nm in R[0]: print(f"   {nm:>16} | {np.mean([r[nm][0] for r in R]):>9.3f} {np.mean([r[nm][1] for r in R]):>8.3f} {np.mean([r[nm][2] for
r in R]):>6.0f} {R[0][nm][3]:>14}")
out['id'] = {nm: dict(train=float(np.mean([r[nm][0] for r in R])), test=float(np.mean([r[nm][1] for r in R])),
leaves=float(np.mean([r[nm][2] for r in R]))) for nm in R[0]}
print("   (mean over 5 seeds)")
# ------------------------------------------------------------------ public dataset
print("\nD  IMPLEMENT AND VERIFY ON A PUBLIC DATASET: Wisconsin breast cancer (569 cases, 30 numeric features), stratified 60/20/20 split,
seed 0")
data = load_breast_cancer(); X, y = data.data, data.target
Xtv, Xte, ytv, yte = train_test_split(X, y, test_size=0.2, stratify=y, random_state=0); Xtr, Xva, ytr, yva = train_test_split(Xtv, ytv,
test_size=0.25, stratify=ytv, random_state=0)
print(f"   sizes: train {len(ytr)}, validation {len(yva)}, test {len(yte)};  malignant(0) / benign(1) in test:
{np.bincount(yte).tolist()}")
def metrics(yt, yp): return dict(acc=accuracy_score(yt, yp), prec=precision_score(yt, yp), rec=recall_score(yt, yp), f1=f1_score(yt, yp),
cm=confusion_matrix(yt, yp).tolist())
D = {}
print(f"   {'model':>28} | {'train':>6} {'test acc':>8} {'precision':>9} {'recall':>7} {'F1':>6} | {'leaves':>6} {'depth':>5} | confusion
[[TN,FP],[FN,TP]]   | {'fit s':>6}")
for crit in ('entropy', 'gini'):
    t0 = time.perf_counter(); own = DecisionTree(crit, types=['num'] * 30, names=list(data.feature_names)).fit(Xtr, ytr); tf =
time.perf_counter() - t0
    sk = DecisionTreeClassifier(criterion=crit, random_state=0).fit(Xtr, ytr)
    po, ps = own.predict(Xte), sk.predict(Xte)
    for nm, mdl, pr in ((f'own tree ({crit})', own, po), (f'scikit-learn ({crit})', sk, ps)):
        m = metrics(yte, pr); trn = accuracy_score(ytr, mdl.predict(Xtr)); lv = mdl.n_leaves() if hasattr(mdl, 'n_leaves') and not
callable(getattr(mdl, 'n_leaves', None)) else mdl.get_n_leaves(); dp = mdl.depth() if isinstance(mdl, DecisionTree) else
mdl.get_depth()
        print(f"   {nm:>28} | {trn:>6.3f} {m['acc']:>8.3f} {m['prec']:>9.3f} {m['rec']:>7.3f} {m['f1']:>6.3f} | {lv:>6} {dp:>5} | {m['cm']}
| {tf if nm.startswith('own') else 0:>6.2f}")
        D[nm] = dict(m, train=trn, leaves=lv, depth=dp)
    print(f"   {'':>28}   agreement of own and sklearn predictions on the test set ({crit}): {np.mean(po == ps):.4f};  on the training set:
{np.mean(own.predict(Xtr) == sk.predict(Xtr)):.4f};  root feature: own '{data.feature_names[own.root.attr]}' (thr {own.root.thr:.4g}),
sklearn '{data.feature_names[sk.tree_.feature[0]]}' (thr {sk.tree_.threshold[0]:.4g})")
out['verify'] = D
print("\n   Boundary tests of the own implementation: (1) a single class -> one leaf; (2) one example; (3) constant features -> majority
leaf; (4) empty partition (unseen categorical value) -> parent majority")
tt = DecisionTree('entropy', types=['num']).fit([[1], [2], [3]], [1, 1, 1]); print("      single class:", tt.n_leaves(), "leaf, predicts",
tt.predict([[7]]).tolist())
tt = DecisionTree('entropy', types=['num']).fit([[5]], [0]); print("      one example:", tt.n_leaves(), "leaf, predicts",
tt.predict([[5]]).tolist())
tt = DecisionTree('entropy', types=['num', 'num']).fit([[1, 1], [1, 1], [1, 1], [1, 1]], [0, 1, 1, 1]); print("      constant features:
leaves", tt.n_leaves(), "-> majority", tt.predict([[1, 1]]).tolist())
tt = DecisionTree('entropy', types=['cat']).fit([['a'], ['a'], ['b'], ['b']], ['x', 'x', 'y', 'y']); print("      unseen category 'c' ->
node default:", tt.predict([['c']]).tolist())
```

```
# ------------------------------------------------------------------ pruning
print("\nE  OVERFITTING AND PRUNING (own tree, entropy; train / validation / test accuracy)")
print(f"   {'setting':>26} | {'depth':>5} {'leaves':>6} | {'train':>6} {'validation':>10} {'test':>6}")
P = []
def row(nm, t):
    a = [accuracy_score(yy, t.predict(xx)) for xx, yy in ((Xtr, ytr), (Xva, yva), (Xte, yte))]
    print(f"   {nm:>26} | {t.depth():>5} {t.n_leaves():>6} | {a[0]:>6.3f} {a[1]:>10.3f} {a[2]:>6.3f}"); P.append(dict(setting=nm,
depth=t.depth(), leaves=t.n_leaves(), train=a[0], val=a[1], test=a[2]))
full = DecisionTree('entropy', types=['num'] * 30).fit(Xtr, ytr); row('full tree (no limit)', full)
for d in (1, 2, 3, 4, 5, 6): row(f'max_depth = {d}', DecisionTree('entropy', max_depth=d, types=['num'] * 30).fit(Xtr, ytr))
for m in (2, 5, 10, 20): row(f'min_samples_leaf = {m}', DecisionTree('entropy', min_samples_leaf=m, types=['num'] * 30).fit(Xtr, ytr))
rep = DecisionTree('entropy', types=['num'] * 30).fit(Xtr, ytr).reduced_error_prune(Xva, yva); row('reduced-error pruning', rep)
out['prune'] = P
print("\n   Repeat over 10 random splits (mean test accuracy): full tree versus max_depth = 3 versus reduced-error pruning")
acc = {'full': [], 'depth3': [], 'rep': []}
for s in range(10):
    a, b, c, d = train_test_split(X, y, test_size=0.2, stratify=y, random_state=s); a1, a2, c1, c2 = train_test_split(a, c, test_size=0.25,
stratify=c, random_state=s)
    f = DecisionTree('entropy', types=['num'] * 30).fit(a1, c1); acc['full'].append(accuracy_score(d, f.predict(b)))
    acc['depth3'].append(accuracy_score(d, DecisionTree('entropy', max_depth=3, types=['num'] * 30).fit(a1, c1).predict(b)))
    acc['rep'].append(accuracy_score(d, DecisionTree('entropy', types=['num'] * 30).fit(a1, c1).reduced_error_prune(a2, c2).predict(b)))
print("   " + ", ".join(f"{k}: {np.mean(v):.3f} +- {np.std(v):.3f}" for k, v in acc.items())); out['prune10'] = {k: (float(np.mean(v)),
float(np.std(v))) for k, v in acc.items()}
print("\n   Learned tree (max_depth = 3, entropy):")
t3_ = DecisionTree('entropy', max_depth=3, types=['num'] * 30, names=list(data.feature_names)).fit(Xtr, ytr); print(t3_.text())
json.dump(out, open('t2 results.json', 'w'), indent=1, default=str)
```

A.9 Task 3 experiment driver (t3_run.py)

```
import numpy as np, json, warnings; warnings.filterwarnings('ignore')
import matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
from em import em, loglik, normal
from sklearn.mixture import GaussianMixture
out = {}
print("TASK 3 - Hidden variables and the EM algorithm: the missing normalisation\n")
print("A  FAILING INPUT AND ONE-ITERATION TRACE (designed by hand): X = [0, 1, 4, 5], K = 2, pi = (0.5, 0.5), mu = (1, 4), var = (1, 1)")
X = np.array([0., 1, 4, 5]); h = em(X, [.5, .5], [1., 4.], [1., 1.], 1, False, trace=True); g = em(X, [.5, .5], [1., 4.], [1., 1.], 1,
True, trace=True)
print(f"   {'i':>2} {'x':>3} | {'r[i][1]':>9} {'r[i][2]':>9} {'row sum (faulty)':>16} | {'repaired r[i][1]':>16} {'r[i][2]':>9} {'row
sum':>8}")
for i in range(4): print(f"   {i+1:>2} {X[i]:>3.0f} | {h['log'][0]['r'][i,0]:>9.5f} {h['log'][0]['r'][i,1]:>9.5f}
{h['log'][0]['r'][i].sum():>16.5f} | {g['log'][0]['r'][i,0]:>16.5f} {g['log'][0]['r'][i,1]:>9.5f} {g['log'][0]['r'][i].sum():>8.5f}")
for nm, hh in (('faulty', h), ('repaired', g)):
    L = hh['log'][0]; print(f"   {nm:>8}: N = {np.round(L['Nk'], 5).tolist()} (sum {L['Nk'].sum():.5f}, must equal n = 4);  mu =
{np.round(L['mu'], 5).tolist()};  var = {np.round(L['var'], 5).tolist()};  pi = {np.round(L['pi'], 5).tolist()} (sum {L['pi'].sum():.5f},
must be 1)")
print("   first incorrect value: the responsibilities r[i][k] (row sums 0.12 to 0.20 instead of 1). Everything computed from them inherits
the error: N_k no longer counts points, pi_k no longer sums to 1.")
out['trace'] = dict(faulty_pi_sum=float(h['log'][0]['pi'].sum()), repaired_pi_sum=float(g['log'][0]['pi'].sum()))
print("\nB  20+ RANDOM INITIALISATIONS: data = 200 points from 0.4 N(-2,1) + 0.6 N(3,1.5^2), seed 1; K = 2; T = 60; 30 random
initialisations (random distinct data points as means, variance = data variance, Dirichlet weights)")
rng = np.random.RandomState(1); z = rng.rand(200) < 0.4; X = np.where(z, rng.normal(-2, 1, 200), rng.normal(3, 1.5, 200)); n = len(X)
gm = GaussianMixture(2, n_init=10, random_state=0, tol=1e-10, max_iter=1000).fit(X[:, None]); ll_ref = gm.score(X[:, None]) * n
print(f"   reference: scikit-learn GaussianMixture (10 restarts) log-likelihood {ll_ref:.4f}, weights {np.round(gm.weights_, 3).tolist()},
means {np.round(gm.means_.ravel(), 3).tolist()}")
H = {'faulty': [], 'repaired': []}
for s in range(30):
    r = np.random.RandomState(100 + s); mu0 = r.choice(X, 2, replace=False); pi0 = r.dirichlet([2, 2]); var0 = [X.var(), X.var()]
    for nm, rep in (('faulty', False), ('repaired', True)): H[nm].append(em(X, pi0, mu0, var0, 60, rep))
fig, ax = plt.subplots(1, 3, figsize=(12, 3.6))
for nm, col in (('faulty', '#C0504D'), ('repaired', '#2E8B57')):
    for hh in H[nm]:
        ax[0].plot(hh['ll'], color=col, alpha=.35, lw=1); ax[1].plot(hh['spi'], color=col, alpha=.35, lw=1)
ax[0].axhline(ll_ref, color='k', ls='--', lw=1, label='scikit-learn optimum'); ax[0].set_ylim(-700, -350); ax[0].set_title('Log-likelihood
per iteration (30 inits)'); ax[0].set_xlabel('iteration'); ax[0].legend(fontsize=8)
ax[1].axhline(1, color='k', ls='--', lw=1); ax[1].set_yscale('log'); ax[1].set_title('Sum of mixing weights (red faulty, green repaired)');
ax[1].set xlabel('iteration')
print(f"   {'version':>9} | {'runs with a decrease of LL':>26} {'largest decrease':>16} | {'sum(pi) final: min':>18} {'max':>8} {'|sum-1|
max':>11} | {'final LL: min':>13} {'median':>9} {'max':>9} | runs within 0.01 of optimum")
S = {}
for nm in H:
    dec = []; big = []
    for hh in H[nm]:
        d = np.diff(hh['ll']); dec.append(bool((d < -1e-9).any())); big.append(float(-d.min()) if (d < -1e-9).any() else 0.0)
    fin = np.array([hh['ll'][-1] for hh in H[nm]]); sp = np.array([hh['spi'][-1] for hh in H[nm]]); sp_all = np.array([x for hh in H[nm]
for x in hh['spi']])
    near = int((np.abs(fin - ll_ref) < 0.01).sum())
    print(f"   {nm:>9} | {sum(dec):>20} of 30 {max(big):>16.4f} | {sp.min():>18.4g} {sp.max():>8.4g} {np.abs(sp all - 1).max():>11.3g} |
{fin.min():>13.2f} {np.median(fin):>9.2f} {fin.max():>9.2f} | {near} of 30")
    S[nm] = dict(runs_with_decrease=int(sum(dec)), largest_decrease=max(big), sum_pi_min=float(sp.min()), sum_pi_max=float(sp.max()),
final_ll_median=float(np.median(fin)), near_opt=near)
out['inits'] = S
print("   Repaired version, local optima: the final log-likelihoods of the 30 runs (rounded):", sorted(set(np.round([hh['ll'][-1] for hh in
H['repaired']], 2).tolist())))
rep_params = np.array([[*hh['final'][1], *hh['final'][0]] for hh in H['repaired']]); print(f"   repaired final weights for the best run:
{np.round(max(H['repaired'], key=lambda q: q['ll'][-1])['final'][0], 3).tolist()}, means {np.round(max(H['repaired'], key=lambda q:
q['ll'][-1])['final'][1], 3).tolist()}, variances {np.round(max(H['repaired'], key=lambda q: q['ll'][-1])['final'][2], 3).tolist()}")
# histogram panel
ax[2].hist(X, bins=30, density=True, color='#BBBBBB'); xs = np.linspace(-6, 9, 300); best = max(H['repaired'], key=lambda q: q['ll'][-1]);
fp, fm, fv = best['final']
ax[2].plot(xs, sum(fp[k] * normal(xs, fm[k], fv[k]) for k in range(2)), color='#2E8B57', label='repaired fit'); badj =
H['faulty'][0]['final']
ax[2].plot(xs, sum(badj[0][k] * normal(xs, badj[1][k], badj[2][k]) for k in range(2)), color='#C0504D', ls='--', label='faulty fit (init
0)'); ax[2].legend(fontsize=8); ax[2].set_title('Data and fitted densities')
plt.tight_layout(); plt.savefig('fig/t3_em.png', dpi=200); plt.close()
print("\nC  WHY THE FAULTY VERSION BEHAVES AS IT DOES (one run, init 0): first 6 iterations")
print(f"   {'iter':>4} | {'faulty LL':>10} {'sum pi':>8} {'mu':>20} | {'repaired LL':>11} {'sum pi':>7} {'mu':>20}")
for t in range(0, 7): print(f"   {t:>4} | {H['faulty'][0]['ll'][t]:>10.2f} {H['faulty'][0]['spi'][t]:>8.4f}
{str(np.round(H['faulty'][0]['mu'][t], 3).tolist()):>20} | {H['repaired'][0]['ll'][t]:>11.2f} {H['repaired'][0]['spi'][t]:>7.4f}
{str(np.round(H['repaired'][0]['mu'][t], 3).tolist()):>20}")
print("\nD  REGRESSION CHECK - an input on which the ORIGINAL ALREADY WORKED")
print("   K = 1, two points symmetric about the mean, and parameters chosen so that every point has density exactly 1 (then the
unnormalised r[i][1] equals 1 = the correct responsibility).")
from scipy.optimize import brentq
var0 = 0.1; a = brentq(lambda a: normal(a, 0, var0) - 1, 0.01, 0.5); Xr = np.array([-a, a]); print(f"   var = {var0}, x = (-{a:.5f},
+{a:.5f}), density at both points = {normal(a, 0, var0):.6f}")
```

```
for nm, rep in (('original', False), ('repaired', True)):
    hh = em(Xr, [1.], [0.], [var0], 1, rep, trace=True); L = hh['log'][0]
    print(f"   {nm:>8}: r = {np.round(L['r'].ravel(), 6).tolist()}, N = {L['Nk'][0]:.6f}, mu = {L['mu'][0]:.6f}, var = {L['var'][0]:.6f},
pi = {L['pi'][0]:.6f}   (correct MLE: mu = 0, var = a^2 = {a*a:.6f}, pi = 1)")
Xr2 = np.array([-2., -1.9, -2.1, -1.8, 3., 3.2, 2.9, 3.1, 3.3, 2.8])
print("   Contrast input: K = 2 with well separated, tight clusters, started near the truth. Here the ORIGINAL FAILS (density is not 1 at
the points) and only the repaired version reaches the maximum-likelihood fit")
for nm, rep in (('original', False), ('repaired', True)):
    hh = em(Xr2, [.5, .5], [-2., 3.], [.05, .05], 30, rep); p, m, v = hh['final']; print(f"   {nm:>8}: pi = {np.round(p, 4).tolist()} (sum
{p.sum():.4f}), mu = {np.round(m, 4).tolist()}, var = {np.round(v, 4).tolist()}, LL = {hh['ll'][-1]:.4f}")
gm2 = GaussianMixture(2, means_init=[[-2], [3]], random_state=0).fit(Xr2[:, None]); print(f"   scikit-learn:  pi = {np.round(gm2.weights_,
4).tolist()}, mu = {np.round(gm2.means_.ravel(), 4).tolist()}, var = {np.round(gm2.covariances_.ravel(), 4).tolist()}, LL =
{gm2.score(Xr2[:, None]) * len(Xr2):.4f}")
json.dump(out, open('t3_results.json', 'w'), indent=1, default=str)
```

A.10 Task 4 experiment driver (t4_run.py)

```
import numpy as np, json, time, warnings; warnings.filterwarnings('ignore')
import matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
from nn import *
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix
out = {}
np.set_printoptions(precision=5, suppress=True)
print("TASK 4 - Back-propagation: desk-check, gradient check, XOR, digits experiments\n")
print("A  DESK-CHECK TRACE (Task 4.4) reproduced in code: 2-2-1 sigmoid network, x = (1, 0), y = 1, squared-error loss L = 1/2 (a - y)^2,
learning rate 0.5, all biases 0")
print("   interpretation: hidden unit 1 has incoming weights (0.5, -0.4), hidden unit 2 has (0.3, 0.8); the output unit has weights (0.6, -
0.2)")
net = MLP([2, 2, 1], 'sigmoid', 'sigmoid', 'mse'); net.W[0] = np.array([[0.5, -0.4], [0.3, 0.8]]); net.W[1] = np.array([[0.6, -0.2]])
x = np.array([[1.], [0.]]); y = np.array([[1.]]); z, a = net.forward(x)
print(f"   forward : z1 = {z[1].ravel()}  a1 = {a[1].ravel()}   z2 = {z[2].ravel()}  a2 (output) = {a[2].ravel()}   loss =
{net.loss_value(a[2], y):.6f}")
dW, db, delta = net.backward(z, a, y)
print(f"   backward: delta2 = (a - y) a (1 - a) = {delta[2].ravel()}   delta1 = (W2^T delta2) . a1(1 - a1) = {delta[1].ravel()}")
print(f"   gradients: dL/dW2 = delta2 a1^T = {dW[1]}   dL/dW1 = delta1 x^T = {dW[0].tolist()}   dL/db2 = {db[1].ravel()}  dL/db1 =
{db[0].ravel()}")
W1n = net.W[0] - 0.5 * dW[0]; W2n = net.W[1] - 0.5 * dW[1]; print(f"   update (eta = 0.5): W1 = {W1n.tolist()}   W2 = {W2n.tolist()}")
net.W[0], net.W[1] = W1n, W2n; z2, a2l = net.forward(x); a2 = a2l[-1]; print(f"   after the update (weights only; biases would also move by
-0.5 dL/db): output = {a2.ravel()}, loss = {net.loss_value(a2, y):.6f}   (before: {0.5*(a[2]-y)[0,0]**2:.6f}) -> the loss decreased")
out['trace'] = dict(output=float(a[2][0, 0]), loss=float(net.loss_value(a[2], y)), loss_after=float(net.loss_value(a2, y)))
print("\nB  GRADIENT CHECK: analytic back-propagation versus central finite differences (eps = 1e-6), double precision")
print(f"   {'network':>40} {'parameters':>10} {'max relative error':>19} {'max abs. difference':>20}")
G = []
for nm, sizes, h, o, l in (('3-4-2 sigmoid, MSE', [3, 4, 2], 'sigmoid', 'sigmoid', 'mse'), ('3-5-4-3 tanh hidden, softmax CE', [3, 5, 4,
3], 'tanh', 'softmax', 'ce'), ('3-6-3 ReLU hidden, softmax CE', [3, 6, 3], 'relu', 'softmax', 'ce'), ('4-5-5-5-2 sigmoid, MSE (deep)', [4,
5, 5, 5, 2], 'sigmoid', 'sigmoid', 'mse')):
    rng = np.random.RandomState(1); n = MLP(sizes, h, o, l, seed=2); X = rng.randn(sizes[0], 7)
    Y = np.eye(sizes[-1])[:, rng.randint(0, sizes[-1], 7)] if o == 'softmax' else rng.rand(sizes[-1], 7)
    z, a = n.forward(X); dW, db, _ = n.backward(z, a, Y); ga = grad_vector(dW, db); gn = numerical_gradient(n, X, Y)
    rel = np.abs(ga - gn) / np.maximum(np.abs(ga) + np.abs(gn), 1e-12); print(f"   {nm:>40} {len(ga):>10} {rel.max():>19.3e} {np.abs(ga-
gn).max():>20.3e}"); G.append(dict(net=nm, params=len(ga), rel=float(rel.max())))
out['gradcheck'] = G
print("   (a relative error below 1e-6 means the analytic gradients are correct; a deliberate bug - forgetting the f'(z) factor in one
layer - gives:)")
n = MLP([3, 4, 2], 'sigmoid', 'sigmoid', 'mse', seed=2); rng = np.random.RandomState(1); X = rng.randn(3, 7); Y = rng.rand(2, 7); z, a =
n.forward(X); dW, db, d = n.backward(z, a, Y)
d1_bug = n.W[1].T @ d[2]; dW_bug = [d1_bug @ X.T / 7] + dW[1:]; ga = grad_vector(dW_bug, db); gn = numerical_gradient(n, X, Y); print(f"
max relative error with the bug: {(np.abs(ga-gn)/np.maximum(np.abs(ga)+np.abs(gn),1e-12)).max():.3e}")
out['bug_rel'] = float((np.abs(ga - gn) / np.maximum(np.abs(ga) + np.abs(gn), 1e-12)).max())
print("\nC  XOR: single-layer perceptron versus the network")
XX = np.array([[0, 0, 1, 1], [0, 1, 0, 1]], float); YY = np.array([[0, 1, 1, 0]], float)
accs = []
for s in range(20):
    rng = np.random.RandomState(s); w = rng.randn(2) * .5; b = 0.0
    for ep in range(1000):
        for i in rng.permutation(4):
            p = 1.0 if XX[:, i] @ w + b > 0 else 0.0; w += 0.1 * (YY[0, i] - p) * XX[:, i]; b += 0.1 * (YY[0, i] - p)
    accs.append(np.mean([(1.0 if XX[:, i] @ w + b > 0 else 0.0) == YY[0, i] for i in range(4)]))
print(f"   perceptron learning rule, 1000 epochs, 20 random starts: accuracy values {sorted(set(float(x) for x in accs))}  (best
{max(accs):.2f}); it never classifies all four points: XOR is not linearly separable")
accs2 = []; loss2 = []
for s in range(20):
    n = MLP([2, 3, 1], 'sigmoid', 'sigmoid', 'mse', seed=s); hst = train(n, XX, YY, 6000, 1.0, batch=4, seed=s);
accs2.append(float(((n.predict(XX) > .5) == (YY > .5)).mean())); loss2.append(hst['loss'][-1])
print(f"   2-3-1 sigmoid network, back-propagation, 6000 epochs, lr 1.0, 20 random starts: all-four-correct in {sum(a == 1.0 for a in
accs2)} of 20 runs; median final loss {np.median(loss2):.5f}")
n = MLP([2, 3, 1], 'sigmoid', 'sigmoid', 'mse', seed=0); train(n, XX, YY, 6000, 1.0, batch=4, seed=0); print(f"   outputs of the run with
seed 0 for (0,0), (0,1), (1,0), (1,1): {n.predict(XX).ravel()}")
out['xor'] = dict(perceptron_best=float(max(accs)), net_success=int(sum(a == 1.0 for a in accs2)), median_loss=float(np.median(loss2)))
print("\nD  8x8 DIGITS: stress test.  Pixels scaled to [0, 1]; stratified 60 / 20 / 20 split (seed 0); mini-batch 32; 60 epochs; softmax
cross-entropy; Xavier init (sigmoid) or He init (ReLU)")
print("   PREDICTIONS WRITTEN BEFORE THE RUNS:")
print("     P1  hidden = 2 limits accuracy to roughly 0.5 - 0.75 for every setting (a 2-unit bottleneck cannot separate 10 classes).")
print("     P2  lr = 0.001 underfits: validation accuracy stays below 0.5 after 60 epochs for sigmoid (slow), better for ReLU.")
print("     P3  lr = 1.0 with ReLU is unstable (loss spikes or NaN); lr = 1.0 with sigmoid still trains because the sigmoid saturates.")
print("     P4  the best cell is hidden 64 or 32 with lr 0.1 (ReLU or sigmoid), test accuracy above 0.95.")
D = load_digits(); Xall = D.data / 16.0; yall = D.target
Xtv, Xte, ytv, yte = train_test_split(Xall, yall, test_size=0.2, stratify=yall, random_state=0); Xtr, Xva, ytr, yva = train_test_split(Xtv,
ytv, test_size=0.25, stratify=ytv, random_state=0)
oh = lambda y: np.eye(10)[:, y]; XtrT, XvaT, XteT = Xtr.T, Xva.T, Xte.T
print(f"   sizes: train {len(ytr)}, validation {len(yva)}, test {len(yte)}")
print(f"   {'hidden':>6} {'lr':>6} {'act':>8} | {'final train loss':>16} {'val acc':>8} {'test acc':>8} {'macro-F1':>8} | {'status':>14}")
RES = {}; curves = {}
for act in ('sigmoid', 'relu'):
    for hid in (2, 8, 32, 64):
        for lr in (0.001, 0.01, 0.1, 1.0):
            n = MLP([64, hid, 10], act, 'softmax', 'ce', seed=0); hst = train(n, XtrT, oh(ytr), 60, lr, 32, seed=0, Xv=XvaT, Yv=oh(yva))
            pte = n.predict(XteT).argmax(0) if np.isfinite(n.W[0]).all() else np.zeros(len(yte), int)
            ok = np.isfinite(hst['loss'][-1]); st = 'diverged (NaN)' if not ok else ('unstable' if max(hst['loss']) > 2 * hst['loss'][0]
and hst['loss'][-1] > 1.0 else 'ok')
            RES[(act, hid, lr)] = dict(loss=hst['loss'][-1], val=hst['val_acc'][-1] if hst['val_acc'] else 0, test=accuracy_score(yte,
pte), f1=f1_score(yte, pte, average='macro'), status=st, cm=confusion_matrix(yte, pte, labels=range(10)).tolist()); curves[(act, hid, lr)]
= hst['loss']
```

```
            r = RES[(act, hid, lr)]; print(f"   {hid:>6} {lr:>6} {act:>8} | {r['loss']:>16.4f} {r['val']:>8.3f} {r['test']:>8.3f}
{r['f1']:>8.3f} | {st:>14}")
out['grid'] = {f"{k[0]}|{k[1]}|{k[2]}": {kk: vv for kk, vv in v.items() if kk != 'cm'} for k, v in RES.items()}
best = max(RES, key=lambda k: RES[k]['val']); print(f"\n   best by VALIDATION accuracy: {best}: test accuracy {RES[best]['test']:.3f},
macro-F1 {RES[best]['f1']:.3f}")
print("   confusion matrix on the test set (rows = true digit, columns = predicted):")
cm = np.array(RES[best]['cm']); print("        " + " ".join(f"{j:>3}" for j in range(10)))
for i in range(10): print(f"     {i:>2} " + " ".join(f"{v:>3}" for v in cm[i]))
out['best'] = dict(cfg=str(best), test=RES[best]['test'], f1=RES[best]['f1'])
print("\n   Prediction check:")
sub = lambda a, h: [RES[(a, h, l)]['test'] for l in (0.001, 0.01, 0.1, 1.0)]
print(f"     P1: hidden = 2 test accuracies (sigmoid, lr .001 .01 .1 1): {np.round(sub('sigmoid', 2), 3).tolist()};  (ReLU):
{np.round(sub('relu', 2), 3).tolist()}")
print(f"     P2: lr = 0.001, hidden 64: sigmoid val acc {RES[('sigmoid', 64, 0.001)]['val']:.3f}, ReLU val acc {RES[('relu', 64,
0.001)]['val']:.3f}")
print(f"     P3: lr = 1.0, hidden 64: ReLU status {RES[('relu', 64, 1.0)]['status']} (test acc {RES[('relu', 64, 1.0)]['test']:.3f}, loss
{RES[('relu', 64, 1.0)]['loss']:.3f});  sigmoid test acc {RES[('sigmoid', 64, 1.0)]['test']:.3f}")
print(f"     P4: best cell {best} with test accuracy {RES[best]['test']:.3f}")
fig, ax = plt.subplots(1, 3, figsize=(12, 3.5))
for hid, col in zip((2, 8, 32, 64), ('#C0504D', '#E08A1E', '#5A9BD5', '#1F3864')):
    ax[0].plot(curves[('relu', hid, 0.1)], color=col, label=f'hidden {hid}'); ax[1].plot(curves[('sigmoid', hid, 0.1)], color=col,
label=f'hidden {hid}')
ax[0].set_title('ReLU, lr 0.1: training loss'); ax[1].set_title('sigmoid, lr 0.1: training loss'); ax[0].legend(fontsize=8);
ax[0].set_xlabel('epoch'); ax[1].set_xlabel('epoch')
for lr, col in zip((0.001, 0.01, 0.1, 1.0), ('#C0504D', '#E08A1E', '#2E8B57', '#1F3864')): ax[2].plot(curves[('relu', 64, lr)], color=col,
label=f'lr {lr}')
ax[2].set_yscale('log'); ax[2].set_title('ReLU, hidden 64: learning rates'); ax[2].legend(fontsize=8); ax[2].set_xlabel('epoch');
plt.tight_layout(); plt.savefig('fig/t4_curves.png', dpi=200); plt.close()
# ---------------------------------------------------------------------- failure provocation
print("\nE  PROVOKED FAILURES AND DIAGNOSIS WITH PER-LAYER GRADIENT NORMS")
print("   PREDICTION BEFORE THE RUN: a 6-hidden-layer sigmoid network (32 units each, lr 0.1) will not learn in 60 epochs; the gradient
norm of the first layers will be orders of magnitude smaller than the last layer's (each backward step multiplies by W^T and by f'(z) <=
0.25).")
fl = {}
for nm, act, init in (('deep sigmoid (6 hidden layers), Xavier', 'sigmoid', None), ('deep ReLU (6 hidden layers), He', 'relu', None)):
    n = MLP([64] + [32] * 6 + [10], act, 'softmax', 'ce', seed=0, init=init); hst = train(n, XtrT, oh(ytr), 60, 0.1, 32, seed=0, Xv=XvaT,
Yv=oh(yva), record_grads=(0, 9, 59))
    acc = accuracy_score(yte, n.predict(XteT).argmax(0)); fl[nm] = dict(acc=acc, g=hst['gnorm'], loss=hst['loss'])
    print(f"   {nm}: final loss {hst['loss'][-1]:.4f}, test accuracy {acc:.3f}")
    for ep, g in hst['gnorm'].items(): print(f"      epoch {ep + 1:>2}: ||dL/dW|| per layer 1..7 = " + ", ".join(f"{v:.2e}" for v in g) +
f"   ratio last/first = {g[-1] / g[0]:.1f}")
n = MLP([64, 64, 10], 'relu', 'softmax', 'ce', seed=0); hst = train(n, XtrT, oh(ytr), 60, 1.0, 32, seed=0, Xv=XvaT, Yv=oh(yva),
record_grads=(0, 5, 20))
print(f"   divergence candidate (ReLU, hidden 64, lr 1.0): loss by epoch 1, 2, 3, 5, 10, 30, 60 = {[round(hst['loss'][i], 3) if
np.isfinite(hst['loss'][i]) else 'NaN' for i in (0, 1, 2, 4, 9, 29, 59) if i < len(hst['loss'])]}, test accuracy {accuracy_score(yte,
n.predict(XteT).argmax(0)):.3f}")
for ep, g in hst['gnorm'].items(): print(f"      epoch {ep + 1:>2}: ||dL/dW|| per layer = " + ", ".join(f"{v:.2e}" for v in g))
n = MLP([64, 64, 10], 'sigmoid', 'softmax', 'ce', seed=0); hst2 = train(n, XtrT, oh(ytr), 60, 50.0, 32, seed=0, Xv=XvaT, Yv=oh(yva),
record_grads=(0, 5))
print(f"   a clear divergence (sigmoid, hidden 64, lr 50): loss by epoch 1, 2, 3, 5 = {[round(hst2['loss'][i], 2) if
np.isfinite(hst2['loss'][i]) else 'NaN' for i in (0, 1, 2, 4) if i < len(hst2['loss'])]}, test accuracy {accuracy_score(yte,
n.predict(XteT).argmax(0)):.3f}")
n = MLP([64, 64, 10], 'relu', 'softmax', 'ce', seed=0); hst3 = train(n, XtrT, oh(ytr), 60, 5.0, 32, seed=0, Xv=XvaT, Yv=oh(yva),
record_grads=(0, 5)); a_ = n.forward(XtrT)[1][1]; dead = float((a_.max(axis=1) == 0).mean())
print(f"   ReLU, hidden 64, lr 5.0: loss by epoch 1, 2, 3, 5, 60 = {[round(hst3['loss'][i], 2) if np.isfinite(hst3['loss'][i]) else 'NaN'
for i in (0, 1, 2, 4, len(hst3['loss'])-1)]}, test accuracy {accuracy_score(yte, n.predict(XteT).argmax(0)):.3f}, dead hidden units (never
active on the training set): {dead:.0%}")
out['failure'] = {k: dict(acc=v['acc'], g={str(e): gg for e, gg in v['g'].items()}) for k, v in fl.items()}
fig, ax = plt.subplots(1, 2, figsize=(9, 3.4))
for (nm, v), col in zip(fl.items(), ('#C0504D', '#2E8B57')):
    g = v['g'][9]; ax[0].semilogy(range(1, 8), g, 'o-', color=col, label=nm.split(',')[0])
ax[0].set_xlabel('layer (1 = nearest the input)'); ax[0].set_ylabel('gradient norm (epoch 10)'); ax[0].set_title('Per-layer gradient
norms'); ax[0].legend(fontsize=8)
for (nm, v), col in zip(fl.items(), ('#C0504D', '#2E8B57')): ax[1].plot(v['loss'], color=col, label=nm.split(',')[0])
ax[1].set_title('Training loss, 6 hidden layers, lr 0.1'); ax[1].set_xlabel('epoch'); ax[1].legend(fontsize=8); plt.tight_layout();
plt.savefig('fig/t4_vanish.png', dpi=200); plt.close()
json.dump(out, open('t4_results.json', 'w'), indent=1, default=str)
```

A.11 Task 5 experiment driver (Naive Bayes, SVMs, EBL test) (t5_run.py)

```
import numpy as np, json, time, warnings; warnings.filterwarnings('ignore')
import matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, GridSearchCV, StratifiedKFold
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, log_loss
from dt import DecisionTree
import ebl
out = {}
print("TASK 5 - Same data, different learning reasoning\n")
FEAT = ['load ratio', 'transformer temp', 'voltage dev', 'thd', 'ambient', 'age years']
def grid_data(n, seed):
    """synthetic substation states: hazardous when load AND temperature are both inside an 'overload band' (an ellipse) - NOT linearly
separable; 4 % label noise; 4 weak or irrelevant features"""
    r = np.random.RandomState(seed); X = np.stack([r.uniform(0.3, 1.3, n), r.uniform(40, 130, n), r.normal(0, 4, n), r.uniform(0, 10, n),
r.normal(25, 8, n), r.uniform(0, 40, n)], 1)
    y = (((X[:, 0] - 0.85) / 0.28) ** 2 + ((X[:, 1] - 85) / 22) ** 2 < 1).astype(int); flip = r.rand(n) < 0.04; y = np.where(flip, 1 - y,
y); return X, y
class CatNB:
    """categorical naive Bayes by maximum-likelihood counts with Laplace smoothing alpha"""
    def __init__(self, alpha): self.alpha = alpha
    def fit(self, B, y, nbins):
        self.classes = np.array([0, 1]); self.nbins = nbins; self.prior_cnt = np.bincount(y, minlength=2).astype(float); self.cnt =
np.zeros((2, B.shape[1], nbins))
        for c in (0, 1):
            for j in range(B.shape[1]): self.cnt[c, j] = np.bincount(B[y == c, j], minlength=nbins)
        return self
    def log_joint(self, B):
        with np.errstate(divide='ignore'):
            lp = np.log((self.prior_cnt + self.alpha) / (self.prior_cnt.sum() + 2 * self.alpha)) if self.alpha > 0 else
np.log(self.prior_cnt / self.prior_cnt.sum())
            L = np.tile(lp, (len(B), 1))
            for c in (0, 1):
                tot = self.cnt[c].sum(axis=1)                                          # per feature: number of class-c rows
                for j in range(B.shape[1]):
```

```
                    p = (self.cnt[c, j, B[:, j]] + self.alpha) / (tot[j] + self.alpha * self.nbins); L[:, c] += np.log(p)
        return L
    def predict_proba(self, B):
        L = self.log_joint(B); bad = np.isneginf(L).all(axis=1)
        m = np.where(np.isneginf(L).all(axis=1, keepdims=True), 0, L.max(axis=1, keepdims=True)); P = np.exp(L - m); s = P.sum(axis=1,
keepdims=True)
        P = np.where(s > 0, P / np.where(s > 0, s, 1), np.array([[0.5, 0.5]])); self.n_undefined = int(bad.sum()); return P
    def predict(self, B):
        P = self.predict_proba(B); return P.argmax(1)
def bins_from(Xtr, nb=8):
    edges = [np.unique(np.quantile(Xtr[:, j], np.linspace(0, 1, nb + 1)[1:-1])) for j in range(Xtr.shape[1])]
    return edges, lambda X: np.stack([np.searchsorted(edges[j], X[:, j]) for j in range(X.shape[1])], 1)
print("A  NAIVE BAYES (Task 5.1)")
print("A1 Toy table: why a zero count breaks classification (three examples, two binary features, classes Hazard / Normal)")
toy = [('HighLoad', 'HighTemp', 'Hazard'), ('HighLoad', 'LowTemp', 'Normal'), ('LowLoad', 'LowTemp', 'Normal')]
print("   examples: (HighLoad, HighTemp) Hazard;  (HighLoad, LowTemp) Normal;  (LowLoad, LowTemp) Normal.   Query: (LowLoad, HighTemp)")
cnt = {('Hazard', 'load', 'LowLoad'): 0, ('Hazard', 'temp', 'HighTemp'): 1, ('Normal', 'load', 'LowLoad'): 1, ('Normal', 'temp',
'HighTemp'): 0}
print("   maximum-likelihood (alpha = 0): P(Hazard)=1/3, P(Normal)=2/3;  P(LowLoad|Hazard) = 0/1 = 0,  P(HighTemp|Normal) = 0/2 = 0")
print("      score(Hazard) = 1/3 x P(LowLoad|Hazard) x P(HighTemp|Hazard) = 1/3 x 0 x 1 = 0;     score(Normal) = 2/3 x P(LowLoad|Normal) x
P(HighTemp|Normal) = 2/3 x 1/2 x 0 = 0  -> both zero: classification undefined (one unseen value vetoes the class, whatever the other
evidence says)")
print("   Laplace alpha = 1: P(LowLoad|Hazard) = (0+1)/(1+2) = 1/3, P(HighTemp|Hazard) = (1+1)/(1+2) = 2/3 -> score(Hazard) = (1+1)/(3+2) x
1/3 x 2/3 = 0.0889;  P(LowLoad|Normal) = (1+1)/(2+2) = 1/2, P(HighTemp|Normal) = (0+1)/(2+2) = 1/4 -> score(Normal) = (2+1)/(3+2) x 1/2 x
1/4 = 0.0750.  -> Hazard wins, a defined answer.")
alphas = (0, 0.1, 1, 10)
print("\nA2 Implementation on the smart-grid data: 8 equal-frequency bins per feature (edges from the training part only); test set 400
states, seed 11")
print(f"   {'train size':>10} {'alpha':>6} | {'accuracy':>8} {'precision':>9} {'recall':>7} {'F1':>6} {'log-loss':>9} | {'test rows with no
defined class':>31}")
NB = []
Xte, yte = grid_data(400, 11)
for ntr in (60, 200, 1000):
    Xtr, ytr = grid_data(ntr, 5); edges, tf = bins_from(Xtr); Btr, Bte = tf(Xtr), tf(Xte); nb = 8
    for a in alphas:
        m = CatNB(a).fit(Btr, ytr, nb); P = m.predict_proba(Bte); pr = P.argmax(1); und = m.n_undefined
        ll = log_loss(yte, np.clip(P[:, 1], 1e-15, 1 - 1e-15), labels=[0, 1]); r = dict(n=ntr, alpha=a, acc=accuracy_score(yte, pr),
prec=precision_score(yte, pr, zero_division=0), rec=recall_score(yte, pr, zero_division=0), f1=f1_score(yte, pr, zero_division=0), ll=ll,
undefined=und); NB.append(r)
        print(f"   {ntr:>10} {a:>6} | {r['acc']:>8.3f} {r['prec']:>9.3f} {r['rec']:>7.3f} {r['f1']:>6.3f} {ll:>9.3f} | {und:>31}")
out['nb'] = NB
print("   (log-loss of the alpha = 0 model uses probabilities clipped at 1e-15; undefined rows are classified by the prior and given
probability 0.5)")
# ---------------------------------------------------------------------- kernels
print("\nB  KERNEL MACHINES (Task 5.2)")
print("   which assumption becomes unsafe: Naive Bayes assumes the features are conditionally independent given the class and models each
by a per-feature count; a linear SVM assumes a linear boundary. The hazard region is a band in (load, temperature) jointly: neither the
marginal of one feature nor any line separates it.")
print("   Experiment on 5 different splits (train 600, test 400 each), all 6 features standardised for the SVMs; 5-fold CV for hyper-
parameters: linear C in {0.01,0.1,1,10,100}; RBF C in {0.1,1,10,100} x gamma in {0.01,0.1,1,10}")
R = {}
for seed in range(5):
    Xtr, ytr = grid data(600, 100 + seed); Xte, yte = grid data(400, 200 + seed); cv = StratifiedKFold(5, shuffle=True, random state=seed)
    res = {}
    t0 = time.perf_counter(); gl = GridSearchCV(make_pipeline(StandardScaler(), SVC(kernel='linear')), {'svc__C': [0.01, 0.1, 1, 10, 100]},
cv=cv, scoring='f1').fit(Xtr, ytr); tg = time.perf_counter() - t0
    t0 = time.perf counter(); mm = make pipeline(StandardScaler(), SVC(kernel='linear', C=gl.best params ['svc  C'])).fit(Xtr, ytr); tf  =
time.perf_counter() - t0; p = mm.predict(Xte)
    res['linear SVM'] = dict(par=gl.best_params_, sv=int(mm[-1].n_support_.sum()), acc=accuracy_score(yte, p), f1=f1_score(yte, p),
fit=tf_, grid=tg)
    t0 = time.perf_counter(); gr = GridSearchCV(make_pipeline(StandardScaler(), SVC(kernel='rbf')), {'svc__C': [0.1, 1, 10, 100],
'svc__gamma': [0.01, 0.1, 1, 10]}, cv=cv, scoring='f1').fit(Xtr, ytr); tg = time.perf_counter() - t0
    t0 = time.perf_counter(); mm = make_pipeline(StandardScaler(), SVC(kernel='rbf', C=gr.best_params_['svc__C'],
gamma=gr.best params ['svc  gamma'])).fit(Xtr, ytr); tf  = time.perf counter() - t0; p = mm.predict(Xte)
    res['RBF SVM'] = dict(par=gr.best_params_, sv=int(mm[-1].n_support_.sum()), acc=accuracy_score(yte, p), f1=f1_score(yte, p), fit=tf_,
grid=tg)
    edges, tf = bins_from(Xtr); best_a = max((0.1, 1, 10), key=lambda a: np.mean([f1_score(ytr[v], CatNB(a).fit(tf(Xtr)[t], ytr[t],
8).predict(tf(Xtr)[v])) for t, v in cv.split(Xtr, ytr)]))
    t0 = time.perf_counter(); m = CatNB(best_a).fit(tf(Xtr), ytr, 8); tf_ = time.perf_counter() - t0; p = m.predict(tf(Xte)); res['Naive
Bayes (binned)'] = dict(par={'alpha': best_a}, sv=0, acc=accuracy_score(yte, p), f1=f1_score(yte, p), fit=tf_, grid=0)
    bd = max((2, 3, 4, 5, 6, 8), key=lambda d: np.mean([f1_score(ytr[v], DecisionTree('entropy', max_depth=d, types=['num'] *
6).fit(Xtr[t], ytr[t]).predict(Xtr[v])) for t, v in cv.split(Xtr, ytr)]))
    t0 = time.perf_counter(); m = DecisionTree('entropy', max_depth=bd, types=['num'] * 6).fit(Xtr, ytr); tf_ = time.perf_counter() - t0; p
= m.predict(Xte); res['decision tree (Task 2 learner)'] = dict(par={'max_depth': bd}, sv=0, acc=accuracy_score(yte, p), f1=f1_score(yte,
p), fit=tf_, grid=0)
    for k, v in res.items(): R.setdefault(k, []).append(v)
print(f"   {'model':>32} | {'accuracy (mean +- sd)':>22} {'F1 hazard (mean)':>16} | {'support vectors':>15} | {'fit time ms':>11} {'grid
search s':>13} | chosen hyper-parameters per split")
K = {}
for k, v in R.items():
    print(f"   {k:>32} | {np.mean([x['acc'] for x in v]):>12.3f} +- {np.std([x['acc'] for x in v]):.3f} {np.mean([x['f1'] for x in
v]):>16.3f} | {np.mean([x['sv'] for x in v]):>15.0f} | {np.mean([x['fit'] for x in v]) * 1000:>11.1f} {np.mean([x['grid'] for x in
v]):>13.2f} | {[list(x['par'].values()) for x in v]}")
    K[k] = dict(acc=float(np.mean([x['acc'] for x in v])), sd=float(np.std([x['acc'] for x in v])), f1=float(np.mean([x['f1'] for x in
v])), sv=float(np.mean([x['sv'] for x in v])), fit=float(np.mean([x['fit'] for x in v])))
out['models'] = K; print("   (identical train/test split for every model within a split; 5 splits)")
fig, ax = plt.subplots(1, 4, figsize=(14, 3.4)); Xtr, ytr = grid_data(600, 100); Xp = Xtr[:, :2]; gx, gy = np.meshgrid(np.linspace(0.3,
1.3, 220), np.linspace(40, 130, 220)); G = np.stack([gx.ravel(), gy.ravel()], 1)
def paint(a, pred, title, sv=None):
    a.contourf(gx, gy, pred.reshape(gx.shape), alpha=.25, cmap='coolwarm', levels=[-.5, .5, 1.5]); a.scatter(Xp[:, 0], Xp[:, 1], c=ytr,
cmap='coolwarm', s=6, edgecolor='none')
    if sv is not None: a.scatter(sv[:, 0], sv[:, 1], s=26, facecolors='none', edgecolors='k', linewidths=.6)
    a.set_title(title, fontsize=9); a.set_xlabel('load_ratio'); a.set_ylabel('temperature')
ls = make_pipeline(StandardScaler(), SVC(kernel='linear', C=1)).fit(Xp, ytr); paint(ax[0], ls.predict(G), f'linear SVM ({ls[-
1].n_support_.sum()} SVs)', Xp[ls[-1].support_])
rs = make_pipeline(StandardScaler(), SVC(kernel='rbf', C=10, gamma=1)).fit(Xp, ytr); paint(ax[1], rs.predict(G), f'RBF SVM C=10, g=1 ({rs[-
1].n_support_.sum()} SVs)', Xp[rs[-1].support_])
edges, tf = bins_from(Xp); nbm = CatNB(1).fit(tf(Xp), ytr, 8); paint(ax[2], nbm.predict(tf(G)), 'Naive Bayes (8 bins, alpha=1)')
tr = DecisionTree('entropy', max_depth=5, types=['num', 'num']).fit(Xp, ytr); paint(ax[3], tr.predict(G), 'decision tree (depth 5)');
plt.tight_layout(); plt.savefig('fig/t5_boundaries.png', dpi=200); plt.close()
print("   decision boundaries (two features only, for display) saved: linear SVM accuracy on the 600 training states: %.3f, RBF: %.3f" %
(ls.score(Xp, ytr), rs.score(Xp, ytr)))
# ---------------------------------------------------------------------- EBL
print("\nC  EXPLANATION-BASED LEARNING (Task 5.3)")
print("   Domain theory (9 Horn rules; operational predicates = directly observable facts: FanFailed, PumpFailed, HighLoad, HighTemp,
PoorVentilation, AgedInsulation, Humid, OilFilled, NearResidential):")
for i, (h, b) in enumerate(ebl.THEORY, 1): print(f"     R{i}  {ebl.tstr(h)} :- " + ', '.join(ebl.tstr(x) for x in b))
ex = {('FanFailed', 'T7'), ('HighLoad', 'T7'), ('AgedInsulation', 'T7'), ('Humid', 'T7'), ('OilFilled', 'T7'), ('Name', 'T7')}
tree, leaves = ebl.learn rule(('Hazardous', 'T7'), ex)
```

```
print("\n   Explanation tree of the single training example Hazardous(T7) with facts FanFailed, HighLoad, AgedInsulation, Humid,
OilFilled:")
for l in ebl.explanation_lines(tree): print('     ' + l)
rule_str = "Hazardous(?x) :- " + ', '.join(ebl.tstr(l) for l in leaves); print("\n   Generalised explanation (constants T7 replaced by
variables, following the same rules R1, R3, R4, R5, R6, R8):"); print("     LEARNED RULE:  " + rule_str)
import random
rnd = random.Random(3); atoms = ['FanFailed', 'PumpFailed', 'HighLoad', 'HighTemp', 'PoorVentilation', 'AgedInsulation', 'Humid',
'OilFilled', 'NearResidential']
cases = []
for i in range(20):
    s = f'S{i+1}'
    if i < 6: f = {('FanFailed', s), ('HighLoad', s), ('AgedInsulation', s), ('Humid', s), ('OilFilled', s)}                      # same
pattern as the training example
    elif i < 10: f = {('PumpFailed', s), ('HighLoad', s), ('AgedInsulation', s), ('Humid', s), ('OilFilled', s)}                 #
hazardous by theory, different cause (learned rule MISSES)
    elif i < 12: f = {('HighTemp', s), ('PoorVentilation', s), ('NearResidential', s)}                                          #
hazardous by R2
    elif i < 14: f = {('HighTemp', s), ('PoorVentilation', s), ('AgedInsulation', s), ('Humid', s), ('OilFilled', s)}           #
hazardous by theory via R7 (learned rule misses)
    else: f = {a for a in rnd.sample(atoms, rnd.randint(1, 4))}; f = {(a, s) for a in f}                                        # random
fact sets
    f = f | ({(a, s) for a, _ in f} if False else set()); cases.append(f if i < 14 else {(a[0], s) if isinstance(a, tuple) else (a, s) for
a in f})
print(f"\n   20 designed test cases: 6 repeat the training pattern, 4 use PumpFailed instead of FanFailed, 2 reach Hazardous by R2, 2 by R7
and the rest are random fact sets (some with unrelated hazards).")
print(f"   {'case':>5} | {'theory proves':>13} {'theory steps':>12} | {'learned rule fires':>18} {'rule steps':>10}")
rows = []; tot_t = tot_l = 0; agree = 0; miss = 0
for i, f in enumerate(cases):
    s = f'S{i+1}'; st = ebl.Stat(); p = ebl.prove(('Hazardous', s), f, st); st2 = ebl.Stat(); fired = any(True for _ in
ebl.apply_rule(leaves, None, f, st2))
    rows.append((s, p is not None, st.steps, fired, st2.steps)); tot_t += st.steps; tot_l += st2.steps; agree += (p is not None) == fired;
miss += (p is not None and not fired)
    print(f"   {s:>5} | {str(p is not None):>13} {st.steps:>12} | {str(fired):>18} {st2.steps:>10}")
print(f"   totals: theory {tot_t} steps, learned rule {tot_l} steps (speed-up {tot_t / tot_l:.1f}x);  agreement {agree}/20;  hazardous
cases missed by the learned rule: {miss} (false negatives); false positives: {sum(1 for r in rows if r[3] and not r[1])}")
pos = [r for r in rows if r[1] and r[3]]; print(f"   on the 6 cases that match the learned pattern: theory {sum(r[2] for r in rows[:6])}
steps vs rule {sum(r[4] for r in rows[:6])} steps")
out['ebl'] = dict(rule=rule_str, theory_steps=tot_t, rule_steps=tot_l, agree=int(agree), missed=int(miss))
json.dump(out, open('t5_results.json', 'w'), indent=1, default=str)
```

A.12 Task 6 experiment driver (20 seeds, adversarial test) (t6_run.py)

```
import numpy as np, json, time, warnings; warnings.filterwarnings('ignore')
import matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
from scipy import stats
import rl
out = {}
print("TASK 6 - Smart-AC controller (reinforcement learning)\n")
print("S1  SIMULATOR FACTS: 96 steps/day (15 min); actions Off/Low/High = 0/1/2 kW, removing 0/0.45/0.9 degC per step; indoor temperature
relaxes 8 % per step toward outdoor (28-34 degC base +/-5 degC daily swing) + N(0, 0.05); occupied until ~08:00 (+-45 min) and from ~18:00
(+-1 h); price 1.0 per kWh, 3.0 in the 17:00-21:00 peak; reward = -(price x kWh) - 2 x (occupied step with indoor temperature outside 22-26
degC)")
print("\nS2  DESK-CHECK of Q-learning, 3 states / 2 actions (alpha = 0.5, gamma = 0.9, all Q = 0, actions: 0 = Off, 1 = Cool)")
Q = np.zeros((3, 2)); a1, ga = 0.5, 0.9
seq = [(0, 1, -1.0, 1), (1, 1, -0.5, 2), (2, 0, 1.0, 2), (2, 1, -0.5, 2), (1, 0, -3.0, 0), (0, 1, -1.0, 1), (1, 1, -0.5, 2)]
print(f"   {'k':>2} {'s':>2} {'a':>2} {'r':>6} {'s2':>3} | {'old Q(s,a)':>10} {'max Q(s2,.)':>11} {'target r+g*max':>14} {'TD error':>9}
{'new Q(s,a)':>10}")
desk = []
for k, (s, a, r, s2) in enumerate(seq, 1):
    old = Q[s, a]; mx = Q[s2].max(); tgt = r + ga * mx; td = tgt - old; Q[s, a] = old + a1 * td; desk.append((k, s, a, r, s2, old, mx, tgt,
td, Q[s, a]))
    print(f"   {k:>2} {s:>2} {a:>2} {r:>6.1f} {s2:>3} | {old:>10.4f} {mx:>11.4f} {tgt:>14.4f} {td:>9.4f} {Q[s, a]:>10.4f}")
print("   final Q table (rows s0,s1,s2; columns Off, Cool):", np.round(Q, 4).tolist(), "-> greedy policy:", ['Off' if x == 0 else 'Cool'
for x in Q.argmax(1)])
out['desk'] = [list(map(float, d)) for d in desk]
print("   (hyper-parameters alpha 0.1, gamma 0.99, epsilon 0.995^episode with floor 0.05, 8-bin temperature grid and 1500 days were chosen
on three separate tuning seeds 100-102, not on the 20 evaluation seeds)")
print("\nS3  PREDICTIONS (written before the 20-seed run)")
print("   P1: after training the learner (Q-learning) will use less cost than the fixed-rule baseline by moving cooling away from the
17:00-21:00 peak (pre-cooling), at the price of a similar or slightly higher number of comfort violations. Expected cost saving 10-40 %.")
print("   P2: SARSA will converge to a similar level but a little lower reward than Q-learning (on-policy exploration noise is costly near
the comfort band).")
print("   P3 (adversarial): the policy keys on clock-time blocks. If the peak is moved to 13:00-17:00 the learner will keep cooling at the
new peak and will lose most of its cost advantage; with a +6 degC heat wave it enters bins (> 30 degC) seen rarely in training and its
comfort violations will rise by more than the baseline's.")
SEEDS = list(range(20)); EP = 1500; res = {'q': [], 'sarsa': []}; t0 = time.perf_counter()
base = rl.evaluate_baseline(40); print(f"\nS4  FIXED-RULE BASELINE on 40 held-out evaluation days: reward {base[0]:.2f}, kWh {base[1]:.2f},
cost {base[2]:.2f}, comfort violations {base[3]:.2f} steps/day")
Qs = {}
for algo in ('q', 'sarsa'):
    for sd in SEEDS:
        Q , curve, trR = rl.train(sd, EP, algo=algo); fin = rl.evaluate(Q , 40); res[algo].append(dict(seed=sd, curve=curve,
final=fin.tolist(), train=trR))
        Qs[(algo, sd)] = Q_
print(f"   (training time for 2 x 20 seeds: {time.perf_counter() - t0:.0f} s)")
def conv(curve, final):
    for ep, v in curve:
        if v >= final - 0.05 * abs(final): return ep
    return curve[-1][0]
print(f"\nS5  RESULTS over 20 seeds ({EP} training days each; evaluation = greedy policy on 40 held-out days; mean +- sd across seeds)")
print(f"   {'controller':>20} | {'reward/day':>16} {'kWh/day':>14} {'cost/day':>14} {'violations/day':>16} | {'episodes to converge
(median)':>29}")
summ = {}
for algo, nm in (('q', 'Q-learning'), ('sarsa', 'SARSA')):
    F = np.array([x['final'] for x in res[algo]]); cv = [conv(x['curve'], x['final'][0]) for x in res[algo]]; summ[algo] = F
    print(f"   {nm:>20} | {F[:,0].mean():>8.2f} +- {F[:,0].std():.2f} {F[:,1].mean():>8.2f} +- {F[:,1].std():.2f} {F[:,2].mean():>8.2f} +-
{F[:,2].std():.2f} {F[:,3].mean():>9.2f} +- {F[:,3].std():.2f} | {np.median(cv):>29.0f}")
print(f"   {'fixed-rule baseline':>20} | {base[0]:>16.2f} {base[1]:>14.2f} {base[2]:>14.2f} {base[3]:>16.2f} | {'-':>29}")
q = summ['q']; s_ = summ['sarsa']
t, p = stats.ttest_1samp(q[:, 0], base[0]); print(f"   Q-learning vs baseline reward: mean diff {q[:,0].mean() - base[0]:+.2f}, one-sample
t-test over 20 seeds p = {p:.2e}; seeds beating baseline reward: {int((q[:,0] > base[0]).sum())}/20")
t, p = stats.ttest_rel(q[:, 0], s_[:, 0]); print(f"   Q-learning vs SARSA reward (paired by seed): mean diff {q[:,0].mean() -
s_[:,0].mean():+.2f}, p = {p:.3f}")
print(f"   cost change vs baseline: {100 * (q[:,2].mean() / base[2] - 1):+.1f} %, kWh change: {100 * (q[:,1].mean() / base[1] - 1):+.1f} %,
violation change: {q[:,3].mean() - base[3]:+.2f} steps/day")
```

```
out['summary'] = dict(base=base.tolist(), q=q.mean(0).tolist(), qsd=q.std(0).tolist(), sarsa=s_.mean(0).tolist(), n_beat=int((q[:, 0] >
base[0]).sum()))
# adversarial
print("\nS6  ADVERSARIAL TEST (designed by the student; predictions P3 above) - same trained Q-learning policies (20 seeds), changed world,
40 days each")
scen = {'normal': {}, 'heat wave +6 degC': dict(heat=6.0), 'peak moved to 13:00-17:00': dict(peak=(52, 68)), 'both': dict(heat=6.0,
peak=(52, 68))}
print(f"   {'scenario':>28} | {'baseline cost':>13} {'learner cost':>12} | {'baseline viol':>13} {'learner viol':>12} | {'baseline
reward':>15} {'learner reward':>14} | seeds learner reward > baseline")
adv = {}
rng_models = {}
for sd in SEEDS: rng_models[sd] = Qs[('q', sd)]
for nm, kw in scen.items():
    b = rl.evaluate_baseline(40, **kw); L = np.array([rl.evaluate(rng_models[sd], 40, **kw) for sd in SEEDS]); adv[nm] =
dict(base=b.tolist(), learner=L.mean(0).tolist(), n=int((L[:, 0] > b[0]).sum()))
    print(f"   {nm:>28} | {b[2]:>13.2f} {L[:,2].mean():>12.2f} | {b[3]:>13.2f} {L[:,3].mean():>12.2f} | {b[0]:>15.2f}
{L[:,0].mean():>14.2f} | {adv[nm]['n']}/20")
out['adv'] = adv
# ---- figures
fig, ax = plt.subplots(1, 3, figsize=(14, 3.6))
for algo, c, nm in (('q', 'C0', 'Q-learning'), ('sarsa', 'C1', 'SARSA')):
    C = np.array([[v for _, v in x['curve']] for x in res[algo]]); eps_ = [e for e, _ in res[algo][0]['curve']]; m = C.mean(0); s =
C.std(0)
    ax[0].plot(eps_, m, c, label=nm); ax[0].fill_between(eps_, m - s, m + s, color=c, alpha=.2)
ax[0].axhline(base[0], color='k', ls='--', label='fixed-rule baseline'); ax[0].set_xlabel('training days'); ax[0].set_ylabel('reward per
day (held-out days)'); ax[0].legend(fontsize=8); ax[0].set_title('learning curve (mean +- sd, 20 seeds)', fontsize=9)
names = ['baseline', 'Q-learning', 'SARSA']; cost = [base[2], q[:, 2].mean(), s_[:, 2].mean()]; viol = [base[3], q[:, 3].mean(), s_[:,
3].mean()]
ax[1].bar(names, cost, color=['gray', 'C0', 'C1']); ax[1].set_ylabel('cost per day'); ax[1].set_title('energy cost', fontsize=9)
ax[2].bar(names, viol, color=['gray', 'C0', 'C1']); ax[2].set_ylabel('violations per day (15-min steps)'); ax[2].set_title('comfort
violations', fontsize=9); plt.tight_layout(); plt.savefig('fig/t6_curves.png', dpi=200); plt.close()
# one day trace
Qd = Qs[('q', 0)]; fig, ax = plt.subplots(2, 1, figsize=(10, 5), sharex=True)
for nm, pol, c in (('baseline', rl.baseline_action, 'gray'), ('learned', rl.greedy_policy(Qd), 'C0')):
    env = rl.ACEnv(900001); T = []; A = []
    for _ in range(96): a = pol(env); A.append(a); env.step(a); T.append(env.T)
    ax[0].plot(T, c, label=nm); ax[1].step(range(96), np.array(A) + (0.06 if nm == 'learned' else 0), c, label=nm)
ax[0].axhspan(22, 26, color='g', alpha=.1); ax[0].axvspan(68, 84, color='r', alpha=.1); ax[1].axvspan(68, 84, color='r', alpha=.1);
ax[0].set_ylabel('indoor degC'); ax[1].set_ylabel('action (0 Off,1 Low,2 High)'); ax[1].set_xlabel('15-minute step of the day (red = peak
price, green = comfort band)'); ax[0].legend(); plt.tight_layout(); plt.savefig('fig/t6_day.png', dpi=200); plt.close()
json.dump(out, open('t6_results.json', 'w'), indent=1, default=str)
```

A.13 Figures for Tasks 1 and 2 (make_figs4.py)

```
import numpy as np, matplotlib; matplotlib.use('Agg'); import matplotlib.pyplot as plt
# Task 1: optimism (values copied from t1_output.txt, means over 5 seeds)
m = ['unpruned\ndecision tree', '1-nearest\nneighbour', 'random forest\n(100 trees)']; tr = [1, 1, 1]; cv = [.798, .831, .863]; te = [.793,
.851, .871]; bl = .811
x = np.arange(3); w = .26; fig, ax = plt.subplots(figsize=(7.5, 3.6))
ax.bar(x - w, tr, w, label='training accuracy (junior protocol)', color='#C0392B'); ax.bar(x, cv, w, label='5-fold CV accuracy',
color='#2F5597'); ax.bar(x + w, te, w, label='held-out test accuracy', color='#5DA271')
ax.axhline(bl, color='k', ls='--', lw=1, label='majority-class baseline (0.811)'); ax.set_xticks(x); ax.set_xticklabels(m); ax.set_ylim(.6,
1.03); ax.set_ylabel('accuracy'); ax.legend(fontsize=7.5, loc='lower right'); ax.set_title('Training accuracy overstates real accuracy
(mean of 5 seeds)', fontsize=10)
for i in range(3): ax.text(x[i] - w, tr[i] + .006, '1.000', ha='center', fontsize=7)
plt.tight_layout(); plt.savefig('fig/t1_optimism.png', dpi=200); plt.close()
# Task 2: pruning curves
d = [1, 2, 3, 4, 5, 6]; trn = [.918, .927, .977, .988, .997, 1.0]; val = [.886, .877, .956, .939, .947, .947]; tst = [.851, .842, .912,
.895, .912, .912]
fig, ax = plt.subplots(1, 2, figsize=(10, 3.4)); ax[0].plot(d, trn, 'o-', label='train', color='#C0392B'); ax[0].plot(d, val, 's-',
label='validation', color='#2F5597'); ax[0].plot(d, tst, '^-', label='test', color='#5DA271'); ax[0].set_xlabel('max_depth');
ax[0].set_ylabel('accuracy'); ax[0].legend(fontsize=8); ax[0].set_title('depth limit (own tree, breast cancer)', fontsize=9)
ml = [1, 2, 5, 10, 20]; leaves = [10, 10, 9, 8, 6]; trl = [1.0, .994, .982, .971, .962]; vl = [.947, .947, .956, .956, .965]; tl = [.912,
.904, .939, .921, .921]
ax[1].plot(ml, trl, 'o-', color='#C0392B', label='train'); ax[1].plot(ml, vl, 's-', color='#2F5597', label='validation'); ax[1].plot(ml,
tl, '^-', color='#5DA271', label='test'); ax[1].set_xlabel('min_samples_leaf'); ax[1].set_title('minimum leaf size', fontsize=9);
ax[1].legend(fontsize=8)
for i, l in enumerate(leaves): ax[1].annotate(f'{l} leaves', (ml[i], trl[i]), textcoords='offset points', xytext=(0, 6), fontsize=7,
ha='center')
plt.tight_layout(); plt.savefig('fig/t2_pruning.png', dpi=200); plt.close()
# Task 2: desk-check tree drawing (8-row table)
fig, ax = plt.subplots(figsize=(10, 4.4)); ax.axis('off')
def box(x, y, t, c='#EAF1FB'): ax.text(x, y, t, ha='center', va='center', fontsize=8, bbox=dict(boxstyle='round,pad=.3', fc=c,
ec='#2F5597'))
def ln(x1, y1, x2, y2, t): ax.plot([x1, x2], [y1, y2], color='#555', lw=.8); ax.text((x1 + x2) / 2, (y1 + y2) / 2 + .02, t, fontsize=7,
ha='center', color='#7B2D00')
box(.5, .93, 'Attendance?\nH(S)=1.0000, gain 0.1887 (3-way tie, first listed)')
box(.25, .62, 'Assignments?\nH=0.8113, gain 0.3113'); box(.75, .62, 'Assignments?\nH=0.8113, gain 0.3113'); ln(.5, .88, .25, .67, 'High');
ln(.5, .88, .75, .67, 'Low')
box(.10, .28, 'Pass [0,2]', '#DDF2E3'); box(.38, .34, 'Internal?\nH=1.0, gain 1.0'); ln(.25, .57, .10, .32, 'Done'); ln(.25, .57, .38, .39,
'Missed')
box(.30, .04, 'Good → Pass', '#DDF2E3'); box(.46, .04, 'Poor → Fail', '#F8D7DA'); ln(.38, .29, .30, .08, ''); ln(.38, .29, .46, .08, '')
box(.62, .34, 'Internal?\nH=1.0, gain 1.0'); box(.90, .28, 'Fail [2,0]', '#F8D7DA'); ln(.75, .57, .62, .39, 'Done'); ln(.75, .57, .90, .32,
'Missed')
box(.54, .04, 'Good → Pass', '#DDF2E3'); box(.70, .04, 'Poor → Fail', '#F8D7DA'); ln(.62, .29, .54, .08, ''); ln(.62, .29, .70, .08, '')
ax.set_xlim(0, 1); ax.set_ylim(0, 1); plt.tight_layout(); plt.savefig('fig/t2_tree.png', dpi=200); plt.close()
```

Appendix B – Complete Console Output of Each Task

B.1 Task 1 (t1_output.txt)

```
TASK 1 - Diagnosing and repairing the learning protocol


A  THE DEFECT: accuracy on the training data versus 5-fold cross-validation and a held-out test set (3 models x 5 data seeds)
   protocol of the junior engineer: fit on all data, score on the same data.  CV = stratified 5-fold on the training part;  test = untouched 25 % held-
out.
                model seed | train acc  CV acc test acc | optimism (train - test) | majority baseline acc test recall (at-risk)
unpruned decision tree    0 |     1.000   0.816    0.787 |                   0.213 |                 0.820                 0.519
unpruned decision tree    1 |     1.000   0.824    0.827 |                   0.173 |                 0.800                 0.600
unpruned decision tree    2 |     1.000   0.749    0.813 |                   0.187 |                 0.813                 0.607
unpruned decision tree    3 |     1.000   0.813    0.773 |                   0.227 |                 0.813                 0.429
unpruned decision tree    4 |     1.000   0.789    0.767 |                   0.233 |                 0.807                 0.448
   1-nearest-neighbour    0 |     1.000   0.867    0.813 |                   0.187 |                 0.820                 0.333
   1-nearest-neighbour    1 |     1.000   0.827    0.880 |                   0.120 |                 0.800                 0.600
   1-nearest-neighbour    2 |     1.000   0.800    0.840 |                   0.160 |                 0.813                 0.643
   1-nearest-neighbour    3 |     1.000   0.824    0.880 |                   0.120 |                 0.813                 0.643
   1-nearest-neighbour    4 |     1.000   0.836    0.840 |                   0.160 |                 0.807                 0.448
random forest (100 trees)    0 |     1.000   0.887    0.853 |                   0.147 |                 0.820                 0.222
random forest (100 trees)    1 |     1.000   0.884    0.887 |                   0.113 |                 0.800                 0.567
random forest (100 trees)    2 |     1.000   0.831    0.887 |                   0.113 |                 0.813                 0.464
random forest (100 trees)    3 |     1.000   0.860    0.860 |                   0.140 |                 0.813                 0.464
random forest (100 trees)    4 |     1.000   0.851    0.867 |                   0.133 |                 0.807                 0.414


   Mean over the 5 seeds:
                model | train acc  CV acc test acc  optimism test recall
unpruned decision tree |     1.000   0.798    0.793     0.207       0.521
   1-nearest-neighbour |     1.000   0.831    0.851     0.149       0.533
random forest (100 trees) |     1.000   0.863    0.871     0.129       0.426


   Second dataset (breast cancer, 569 cases) for the unpruned tree:
   train 1.000   5-fold CV 0.927   test 0.902


B  OTHER DEFICIENCIES MADE VISIBLE
   (a) accuracy alone on imbalanced data: a model that always says 'not at risk' has accuracy 0.820 but recall 0.000, F1 0.000  (at-risk share in the
test set: 0.180)
   (b) no control of randomness: the same unpruned tree trained with 5 random seeds gives test accuracies [0.78, 0.693, 0.727, 0.767, 0.773]
   (c) hyper-parameters chosen on the test set (data snooping): for each of 5 splits pick the tree depth with the BEST TEST accuracy and report it:
       snooped (max over depths of test accuracy): mean 0.845;   honest (depth chosen on a validation set, then tested once): mean 0.836;   optimism
+0.009


C  THE REPAIRED PROTOCOL EXECUTED (stratified split, 5-fold CV model selection, test set used once, majority baseline)
   candidates: DecisionTree with max_depth in {1,2,3,4,5,8} x min_samples_leaf in {1,5,10}; selection metric = F1 of the at-risk class
   seed 0: best {'max_depth': 8, 'min_samples_leaf': 5}  CV F1 0.639 | TEST acc 0.833 prec 0.550 recall 0.407 F1 0.468 bal.acc 0.667 | baseline acc
0.820 F1 0.000 | confusion [[114, 9], [16, 11]]
   seed 1: best {'max_depth': 3, 'min_samples_leaf': 10}  CV F1 0.662 | TEST acc 0.873 prec 0.789 recall 0.500 F1 0.612 bal.acc 0.733 | baseline acc
0.800 F1 0.000 | confusion [[116, 4], [15, 15]]
   seed 2: best {'max_depth': 8, 'min_samples_leaf': 10}  CV F1 0.449 | TEST acc 0.887 prec 0.789 recall 0.536 F1 0.638 bal.acc 0.751 | baseline acc
0.813 F1 0.000 | confusion [[118, 4], [13, 15]]
   seed 3: best {'max_depth': 8, 'min_samples_leaf': 5}  CV F1 0.427 | TEST acc 0.800 prec 0.462 recall 0.429 F1 0.444 bal.acc 0.657 | baseline acc
0.813 F1 0.000 | confusion [[108, 14], [16, 12]]
   seed 4: best {'max_depth': 8, 'min_samples_leaf': 10}  CV F1 0.487 | TEST acc 0.833 prec 0.591 recall 0.448 F1 0.510 bal.acc 0.687 | baseline acc
0.807 F1 0.000 | confusion [[112, 9], [16, 13]]
   mean test F1 0.535 (CV estimate 0.533); majority baseline accuracy 0.811 with F1 0
```

B.2 Task 2 (t2_output.txt)

```
TASK 2 - Decision-tree learning: desk-check, implementation, verification, pruning, challenge


A  DESK-CHECK TABLE - the algorithm's calculations (program output; the same numbers were derived by hand in Section 2.4)
Original 8-row table   (criterion = entropy)
   node depth 0: n = 8, class counts [Fail, Pass] = [4, 4], entropy = 1.0000
      gain(Attendance) = 0.1887
      gain(Assignments) = 0.1887
      gain(Internal) = 0.1887
      -> split on Attendance
     node depth 1: n = 4, class counts [Fail, Pass] = [1, 3], entropy = 0.8113
        gain(Assignments) = 0.3113
        gain(Internal) = 0.3113
        -> split on Assignments
       node depth 2: n = 2, class counts [Fail, Pass] = [1, 1], entropy = 1.0000
          gain(Internal) = 1.0000
          -> split on Internal
     node depth 1: n = 4, class counts [Fail, Pass] = [3, 1], entropy = 0.8113
        gain(Assignments) = 0.3113
        gain(Internal) = 0.3113
        -> split on Assignments
       node depth 2: n = 2, class counts [Fail, Pass] = [1, 1], entropy = 1.0000
          gain(Internal) = 1.0000
          -> split on Internal
[Attendance = High]
    [Assignments = Done]
        -> Pass  [0, 2]
    [Assignments = Missed]
        [Internal = Good]
            -> Pass  [0, 1]
        [Internal = Poor]
            -> Fail  [1, 0]
[Attendance = Low]
    [Assignments = Done]
        [Internal = Good]
            -> Pass  [0, 1]
        [Internal = Poor]
            -> Fail  [1, 0]
    [Assignments = Missed]
        -> Fail  [2, 0]


   leaves 6, depth 3; training accuracy 1.000
   Observation: Pass is exactly 'at least two of the three signals are good' (the majority of three).  The minimal tree needs 6 leaves; all 3 root
gains tie at 0.1887, so the root depends on the tie-break rule (first attribute listed).
```

```
B  CHALLENGE 1 - change ONE value: row 4 (High, Missed, Poor) Fail -> Pass.   PREDICTION WRITTEN BEFORE THE RUN (from hand calculation):
   H(S) = 0.9544 (5 Pass, 3 Fail).  Attendance: High 4P/0F entropy 0, Low 1P/3F 0.8113 -> gain 0.5488.  Assignments and Internal: gain 0.0488.  => the
root changes from a 3-way tie to a CLEAR winner: Attendance.
Modified table   (criterion = entropy)
   node depth 0: n = 8, class counts [Fail, Pass] = [3, 5], entropy = 0.9544
      gain(Attendance) = 0.5488
      gain(Assignments) = 0.0488
      gain(Internal) = 0.0488
      -> split on Attendance
     node depth 1: n = 4, class counts [Fail, Pass] = [3, 1], entropy = 0.8113
        gain(Assignments) = 0.3113
        gain(Internal) = 0.3113
        -> split on Assignments
       node depth 2: n = 2, class counts [Fail, Pass] = [1, 1], entropy = 1.0000
          gain(Internal) = 1.0000
          -> split on Internal
[Attendance = High]
    -> Pass  [0, 4]
[Attendance = Low]
    [Assignments = Done]
        [Internal = Good]
            -> Pass  [0, 1]
        [Internal = Poor]
            -> Fail  [1, 0]
    [Assignments = Missed]
        -> Fail  [2, 0]


   verified: root = Attendance; leaves 4


C  CHALLENGE 2 - an identifier-like attribute (unique value per row) on the 8-row table
With StudentID, information gain   (criterion = entropy)
   node depth 0: n = 8, class counts [Fail, Pass] = [4, 4], entropy = 1.0000
      gain(StudentID) = 1.0000
      gain(Attendance) = 0.1887
      gain(Assignments) = 0.1887
      gain(Internal) = 0.1887
      -> split on StudentID
[StudentID = S1]
    -> Pass  [0, 1]
[StudentID = S2]
    -> Pass  [0, 1]
[StudentID = S3]
    -> Pass  [0, 1]
[StudentID = S4]
    -> Fail  [1, 0]
[StudentID = S5]
    -> Pass  [0, 1]
[StudentID = S6]
    -> Fail  [1, 0]
[StudentID = S7]
    -> Fail  [1, 0]
[StudentID = S8]
    -> Fail  [1, 0]


   -> root = StudentID, gain 1.0000 = H(S), leaves 8 (one per row): a memorising table.
With StudentID, gain ratio (C4.5 repair)   (criterion = gainratio)
   node depth 0: n = 8, class counts [Fail, Pass] = [4, 4], entropy = 1.0000
      gain(StudentID) = 1.0000   score(ratio) = 0.3333
      gain(Attendance) = 0.1887   score(ratio) = 0.1887
      gain(Assignments) = 0.1887   score(ratio) = 0.1887
      gain(Internal) = 0.1887   score(ratio) = 0.1887
      -> split on StudentID
[StudentID = S1]
    -> Pass  [0, 1]
[StudentID = S2]
    -> Pass  [0, 1]
[StudentID = S3]
    -> Pass  [0, 1]
[StudentID = S4]
    -> Fail  [1, 0]
[StudentID = S5]
    -> Pass  [0, 1]
[StudentID = S6]
    -> Fail  [1, 0]
[StudentID = S7]
    -> Fail  [1, 0]
[StudentID = S8]
    -> Fail  [1, 0]


   -> root = StudentID
   On a larger synthetic table (300 train / 200 test rows, label = majority of three signals with 10 % noise) with an added unique ID column:
           variant | train acc test acc leaves  root (seed 0)
             no ID |     0.912    0.905      8       Internal
     ID, info gain |     1.000    0.501    300      StudentID
    ID, gain ratio |     1.000    0.862    281       Internal
   (mean over 5 seeds)


D  IMPLEMENT AND VERIFY ON A PUBLIC DATASET: Wisconsin breast cancer (569 cases, 30 numeric features), stratified 60/20/20 split, seed 0
   sizes: train 341, validation 114, test 114;  malignant(0) / benign(1) in test: [42, 72]
                         model |  train test acc precision  recall     F1 | leaves depth | confusion [[TN,FP],[FN,TP]]   |  fit s
            own tree (entropy) |  1.000    0.912     0.956   0.903  0.929 |     10     6 | [[39, 3], [7, 65]] |   0.02
        scikit-learn (entropy) |  1.000    0.912     0.956   0.903  0.929 |     10     6 | [[39, 3], [7, 65]] |   0.00
                                agreement of own and sklearn predictions on the test set (entropy): 1.0000;  on the training set: 1.0000;  root
feature: own 'worst perimeter' (thr 101.6), sklearn 'worst perimeter' (thr 101.5)
              own tree (gini) |  1.000    0.912     0.943   0.917  0.930 |     13     6 | [[38, 4], [6, 66]] |   0.02
           scikit-learn (gini) |  1.000    0.921     0.957   0.917  0.936 |     13     6 | [[39, 3], [6, 66]] |   0.00
                               agreement of own and sklearn predictions on the test set (gini): 0.9912;  on the training set: 1.0000;  root feature:
own 'mean concave points' (thr 0.04923), sklearn 'mean concave points' (thr 0.04923)


   Boundary tests of the own implementation: (1) a single class -> one leaf; (2) one example; (3) constant features -> majority leaf; (4) empty
partition (unseen categorical value) -> parent majority
      single class: 1 leaf, predicts [1]
      one example: 1 leaf, predicts [0]
      constant features: leaves 1 -> majority [1]
      unseen category 'c' -> node default: ['x']


E  OVERFITTING AND PRUNING (own tree, entropy; train / validation / test accuracy)
                    setting | depth leaves |  train validation   test
        full tree (no limit) |     6     10 |  1.000      0.947  0.912
               max_depth = 1 |     1      2 |  0.918      0.886  0.851
```

```
               max_depth = 2 |     2      4 |  0.927      0.877  0.842
               max_depth = 3 |     3      6 |  0.977      0.956  0.912
               max_depth = 4 |     4      8 |  0.988      0.939  0.895
               max_depth = 5 |     5      9 |  0.997      0.947  0.912
               max_depth = 6 |     6     10 |  1.000      0.947  0.912
        min_samples_leaf = 2 |     6     10 |  0.994      0.947  0.904
        min_samples_leaf = 5 |     5      9 |  0.982      0.956  0.939
       min_samples_leaf = 10 |     4      8 |  0.971      0.956  0.921
       min_samples_leaf = 20 |     3      6 |  0.962      0.965  0.921
       reduced-error pruning |     3      4 |  0.962      0.974  0.939


   Repeat over 10 random splits (mean test accuracy): full tree versus max_depth = 3 versus reduced-error pruning
   full: 0.931 +- 0.023, depth3: 0.938 +- 0.019, rep: 0.936 +- 0.014


   Learned tree (max_depth = 3, entropy):
[worst perimeter <= 101.6]
    [worst smoothness <= 0.188]
        [area error <= 47.03]
            -> 1  [0, 191]
        [area error > 47.03]
            -> 0  [2, 0]
    [worst smoothness > 0.188]
        -> 0  [3, 0]
[worst perimeter > 101.6]
    [worst perimeter <= 117.5]
        [worst smoothness <= 0.1367]
            -> 1  [7, 22]
        [worst smoothness > 0.1367]
            -> 0  [17, 1]
    [worst perimeter > 117.5]
        -> 0  [98, 0]
```

B.3 Task 3 (t3_output.txt)

```
TASK 3 - Hidden variables and the EM algorithm: the missing normalisation


A  FAILING INPUT AND ONE-ITERATION TRACE (designed by hand): X = [0, 1, 4, 5], K = 2, pi = (0.5, 0.5), mu = (1, 4), var = (1, 1)
    i   x |   r[i][1]   r[i][2] row sum (faulty) | repaired r[i][1]   r[i][2]  row sum
    1   0 |   0.12099   0.00007          0.12105 |          0.99945   0.00055  1.00000
    2   1 |   0.19947   0.00222          0.20169 |          0.98901   0.01099  1.00000
    3   4 |   0.00222   0.19947          0.20169 |          0.01099   0.98901  1.00000
    4   5 |   0.00007   0.12099          0.12105 |          0.00055   0.99945  1.00000
     faulty: N = [0.32274, 0.32274] (sum 0.64548, must equal n = 4);  mu = [0.64656, 4.35344];  var = [0.31506, 0.31506];  pi = [0.08068, 0.08068] (sum
0.16137, must be 1)
   repaired: N = [2.0, 2.0] (sum 4.00000, must equal n = 4);  mu = [0.51786, 4.48214];  var = [0.32113, 0.32113];  pi = [0.5, 0.5] (sum 1.00000, must
be 1)
   first incorrect value: the responsibilities r[i][k] (row sums 0.12 to 0.20 instead of 1). Everything computed from them inherits the error: N_k no
longer counts points, pi_k no longer sums to 1.


B  20+ RANDOM INITIALISATIONS: data = 200 points from 0.4 N(-2,1) + 0.6 N(3,1.5^2), seed 1; K = 2; T = 60; 30 random initialisations (random distinct
data points as means, variance = data variance, Dirichlet weights)
   reference: scikit-learn GaussianMixture (10 restarts) log-likelihood -451.2079, weights [0.527, 0.473], means [3.178, -1.59]
     version | runs with a decrease of LL largest decrease | sum(pi) final: min      max |sum-1| max | final LL: min    median       max | runs within
0.01 of optimum
      faulty |                  30 of 30        9940.8684 |          4.772e-55 1.025e-19           1 |    -135339.31 -39685.45 -32131.60 | 0 of 30
    repaired |                   0 of 30           0.0000 |                   1        1    8.88e-16 |       -481.33   -451.21   -451.21 | 22 of 30
   Repaired version, local optima: the final log-likelihoods of the 30 runs (rounded): [-481.33, -481.32, -454.45, -451.21]
   repaired final weights for the best run: [0.527, 0.473], means [3.178, -1.59], variances [1.675, 1.397]


C  WHY THE FAULTY VERSION BEHAVES AS IT DOES (one run, init 0): first 6 iterations
   iter |  faulty LL   sum pi            mu | repaired LL  sum pi            mu
      0 |    -521.36   1.0000  [2.702, 2.202] |     -521.36  1.0000  [2.702, 2.202]
      1 |   -1005.34   0.0922  [2.122, 1.818] |     -481.33  1.0000  [1.006, 0.509]
      2 |   -1484.31   0.0094  [2.058, 1.771] |     -481.33  1.0000  [1.007, 0.507]
      3 |   -1964.77   0.0010  [2.116, 1.809] |     -481.33  1.0000  [1.007, 0.505]
      4 |   -2453.56   0.0001  [2.232, 1.886] |     -481.33  1.0000  [1.007, 0.503]
      5 |   -2957.36   0.0000  [2.385, 1.992] |     -481.33  1.0000  [1.008, 0.501]
      6 |   -3477.66   0.0000  [2.554, 2.124] |     -481.33  1.0000  [1.008, 0.498]


D  REGRESSION CHECK - an input on which the ORIGINAL ALREADY WORKED
   K = 1, two points symmetric about the mean, and parameters chosen so that every point has density exactly 1 (then the unnormalised r[i][1] equals 1
= the correct responsibility).
   var = 0.1, x = (-0.21557, +0.21557), density at both points = 1.000000
   original: r = [1.0, 1.0], N = 2.000000, mu = 0.000000, var = 0.046471, pi = 1.000000   (correct MLE: mu = 0, var = a^2 = 0.046471, pi = 1)
   repaired: r = [1.0, 1.0], N = 2.000000, mu = 0.000000, var = 0.046471, pi = 1.000000   (correct MLE: mu = 0, var = a^2 = 0.046471, pi = 1)
   Contrast input: K = 2 with well separated, tight clusters, started near the truth. Here the ORIGINAL FAILS (density is not 1 at the points) and only
the repaired version reaches the maximum-likelihood fit
   original: pi = [0.2684, 0.3554] (sum 0.6239), mu = [-1.9543, 3.0417], var = [0.0042, 0.008], LL = -12.1326
   repaired: pi = [0.4, 0.6] (sum 1.0000), mu = [-1.95, 3.05], var = [0.0125, 0.0292], LL = -1.5513
   scikit-learn:  pi = [0.4, 0.6], mu = [-1.95, 3.05], var = [0.0125, 0.0292], LL = -1.5513
```

B.4 Task 4 (t4_output.txt)

```
TASK 4 - Back-propagation: desk-check, gradient check, XOR, digits experiments


A  DESK-CHECK TRACE (Task 4.4) reproduced in code: 2-2-1 sigmoid network, x = (1, 0), y = 1, squared-error loss L = 1/2 (a - y)^2, learning rate 0.5,
all biases 0
   interpretation: hidden unit 1 has incoming weights (0.5, -0.4), hidden unit 2 has (0.3, 0.8); the output unit has weights (0.6, -0.2)
   forward : z1 = [0.5 0.3]  a1 = [0.62246 0.57444]   z2 = [0.25859]  a2 (output) = [0.56429]   loss = 0.094922
   backward: delta2 = (a - y) a (1 - a) = [-0.10713]   delta1 = (W2^T delta2) . a1(1 - a1) = [-0.01511  0.00524]
   gradients: dL/dW2 = delta2 a1^T = [[-0.06668 -0.06154]]   dL/dW1 = delta1 x^T = [[-0.01510513751074582, 0.0], [0.00523761430424534, 0.0]]   dL/db2 =
[-0.10713]  dL/db1 = [-0.01511  0.00524]
   update (eta = 0.5): W1 = [[0.5075525687553729, -0.4], [0.29738119284787734, 0.8]]   W2 = [[0.6333410825179691, -0.16923086474120758]]
   after the update (weights only; biases would also move by -0.5 dL/db): output = [0.57401], loss = 0.090732   (before: 0.094922) -> the loss
decreased


B  GRADIENT CHECK: analytic back-propagation versus central finite differences (eps = 1e-6), double precision
                          network parameters  max relative error  max abs. difference
                  3-4-2 sigmoid, MSE         26           3.124e-07            2.022e-11
          3-5-4-3 tanh hidden, softmax CE    59           2.113e-08            1.907e-10
            3-6-3 ReLU hidden, softmax CE    45           6.107e-07            4.162e-10
              4-5-5-5-2 sigmoid, MSE (deep)  97           5.297e-07            1.795e-11
   (a relative error below 1e-6 means the analytic gradients are correct; a deliberate bug - forgetting the f'(z) factor in one layer - gives:)
```

```
   max relative error with the bug: 1.000e+00


C  XOR: single-layer perceptron versus the network
   perceptron learning rule, 1000 epochs, 20 random starts: accuracy values [np.float64(0.5)]  (best 0.50); it never classifies all four points: XOR is
not linearly separable
   2-3-1 sigmoid network, back-propagation, 6000 epochs, lr 1.0, 20 random starts: all-four-correct in 20 of 20 runs; median final loss 0.00078
   outputs of the run with seed 0 for (0,0), (0,1), (1,0), (1,1): [0.02112 0.95778 0.95388 0.05397]


D  8x8 DIGITS: stress test.  Pixels scaled to [0, 1]; stratified 60 / 20 / 20 split (seed 0); mini-batch 32; 60 epochs; softmax cross-entropy; Xavier
init (sigmoid) or He init (ReLU)
   PREDICTIONS WRITTEN BEFORE THE RUNS:
     P1  hidden = 2 limits accuracy to roughly 0.5 - 0.75 for every setting (a 2-unit bottleneck cannot separate 10 classes).
     P2  lr = 0.001 underfits: validation accuracy stays below 0.5 after 60 epochs for sigmoid (slow), better for ReLU.
     P3  lr = 1.0 with ReLU is unstable (loss spikes or NaN); lr = 1.0 with sigmoid still trains because the sigmoid saturates.
     P4  the best cell is hidden 64 or 32 with lr 0.1 (ReLU or sigmoid), test accuracy above 0.95.
   sizes: train 1077, validation 360, test 360
   hidden     lr      act | final train loss  val acc test acc macro-F1 |         status
        2  0.001  sigmoid |           2.3115    0.097    0.097    0.018 |             ok
        2   0.01  sigmoid |           2.2458    0.175    0.189    0.148 |             ok
        2    0.1  sigmoid |           1.3723    0.444    0.425    0.343 |             ok
        2    1.0  sigmoid |           1.0302    0.547    0.575    0.522 |             ok
        8  0.001  sigmoid |           2.2818    0.100    0.081    0.058 |             ok
        8   0.01  sigmoid |           2.0022    0.581    0.542    0.477 |             ok
        8    0.1  sigmoid |           0.3825    0.925    0.908    0.907 |             ok
        8    1.0  sigmoid |           0.0362    0.956    0.958    0.958 |             ok
       32  0.001  sigmoid |           2.2634    0.164    0.161    0.155 |             ok
       32   0.01  sigmoid |           1.7656    0.783    0.775    0.748 |             ok
       32    0.1  sigmoid |           0.2018    0.950    0.950    0.950 |             ok
       32    1.0  sigmoid |           0.0187    0.978    0.972    0.972 |             ok
       64  0.001  sigmoid |           2.2656    0.214    0.222    0.172 |             ok
       64   0.01  sigmoid |           1.6450    0.833    0.822    0.805 |             ok
       64    0.1  sigmoid |           0.1759    0.950    0.947    0.947 |             ok
       64    1.0  sigmoid |           0.0183    0.972    0.972    0.972 |             ok
        2  0.001     relu |           2.2326    0.150    0.206    0.109 |             ok
        2   0.01     relu |           1.7967    0.217    0.250    0.173 |             ok
        2    0.1     relu |           0.8527    0.658    0.653    0.625 |             ok
        2    1.0     relu |           2.3103    0.100    0.100    0.018 |             ok
        8  0.001     relu |           2.1212    0.203    0.183    0.162 |             ok
        8   0.01     relu |           0.6007    0.886    0.844    0.843 |             ok
        8    0.1     relu |           0.0639    0.961    0.950    0.950 |             ok
        8    1.0     relu |           2.3103    0.100    0.100    0.018 |             ok
       32  0.001     relu |           1.9298    0.439    0.414    0.379 |             ok
       32   0.01     relu |           0.3338    0.933    0.925    0.925 |             ok
       32    0.1     relu |           0.0445    0.975    0.964    0.964 |             ok
       32    1.0     relu |           0.0017    0.967    0.969    0.970 |             ok
       64  0.001     relu |           1.8465    0.542    0.492    0.454 |             ok
       64   0.01     relu |           0.3063    0.925    0.922    0.923 |             ok
       64    0.1     relu |           0.0405    0.972    0.975    0.975 |             ok
       64    1.0     relu |           0.0014    0.975    0.969    0.970 |             ok


   best by VALIDATION accuracy: ('sigmoid', 32, 1.0): test accuracy 0.972, macro-F1 0.972
   confusion matrix on the test set (rows = true digit, columns = predicted):
        0   1   2   3   4   5   6   7   8   9
     0  35   0   0   0   1   0   0   0   0   0
     1   0  35   0   0   0   0   0   0   1   0
     2   0   1  34   0   0   0   0   0   0   0
     3   0   0   0  37   0   0   0   0   0   0
     4   0   1   0   0  34   0   0   0   0   1
     5   0   0   0   0   0  37   0   0   0   0
     6   0   1   0   0   0   0  35   0   0   0
     7   0   0   0   0   0   0   0  36   0   0
     8   0   2   0   0   0   1   0   0  32   0
     9   0   0   0   0   0   1   0   0   0  35


   Prediction check:
     P1: hidden = 2 test accuracies (sigmoid, lr .001 .01 .1 1): [0.097, 0.189, 0.425, 0.575];  (ReLU): [0.206, 0.25, 0.653, 0.1]
     P2: lr = 0.001, hidden 64: sigmoid val acc 0.214, ReLU val acc 0.542
     P3: lr = 1.0, hidden 64: ReLU status ok (test acc 0.969, loss 0.001);  sigmoid test acc 0.972
     P4: best cell ('sigmoid', 32, 1.0) with test accuracy 0.972


E  PROVOKED FAILURES AND DIAGNOSIS WITH PER-LAYER GRADIENT NORMS
   PREDICTION BEFORE THE RUN: a 6-hidden-layer sigmoid network (32 units each, lr 0.1) will not learn in 60 epochs; the gradient norm of the first
layers will be orders of magnitude smaller than the last layer's (each backward step multiplies by W^T and by f'(z) <= 0.25).
   deep sigmoid (6 hidden layers), Xavier: final loss 2.3087, test accuracy 0.100
      epoch  1: ||dL/dW|| per layer 1..7 = 1.37e-04, 4.04e-04, 1.48e-03, 6.46e-03, 2.93e-02, 1.36e-01, 5.59e-01   ratio last/first = 4079.6
      epoch 10: ||dL/dW|| per layer 1..7 = 1.53e-04, 5.11e-04, 1.93e-03, 7.66e-03, 4.93e-02, 2.11e-01, 6.08e-01   ratio last/first = 3977.9
      epoch 60: ||dL/dW|| per layer 1..7 = 1.29e-04, 3.38e-04, 1.26e-03, 5.90e-03, 2.51e-02, 1.14e-01, 3.58e-01   ratio last/first = 2776.2
   deep ReLU (6 hidden layers), He: final loss 0.0004, test accuracy 0.956
      epoch  1: ||dL/dW|| per layer 1..7 = 8.52e-01, 7.13e-01, 6.56e-01, 6.82e-01, 7.75e-01, 6.85e-01, 5.67e-01   ratio last/first = 0.7
      epoch 10: ||dL/dW|| per layer 1..7 = 3.38e+00, 1.80e+00, 1.33e+00, 1.13e+00, 1.08e+00, 1.16e+00, 1.12e+00   ratio last/first = 0.3
      epoch 60: ||dL/dW|| per layer 1..7 = 1.42e-02, 7.45e-03, 5.13e-03, 5.36e-03, 5.54e-03, 5.36e-03, 5.37e-03   ratio last/first = 0.4
   divergence candidate (ReLU, hidden 64, lr 1.0): loss by epoch 1, 2, 3, 5, 10, 30, 60 = [1.662, 0.525, 0.277, 0.126, 0.035, 0.003, 0.001], test
accuracy 0.969
      epoch  1: ||dL/dW|| per layer = 8.13e-01, 7.64e-01
      epoch  6: ||dL/dW|| per layer = 2.52e-01, 1.67e-01
      epoch 21: ||dL/dW|| per layer = 1.77e-02, 8.76e-03
   a clear divergence (sigmoid, hidden 64, lr 50): loss by epoch 1, 2, 3, 5 = [17.85, 15.8, 18.78, 21.47], test accuracy 0.097
   ReLU, hidden 64, lr 5.0: loss by epoch 1, 2, 3, 5, 60 = [5.09, 2.37, 2.36, 2.36, 2.35], test accuracy 0.103, dead hidden units (never active on the
training set): 100%
```

B.5 Task 5 (t5_output.txt)

```
TASK 5 - Same data, different learning reasoning


A  NAIVE BAYES (Task 5.1)
A1 Toy table: why a zero count breaks classification (three examples, two binary features, classes Hazard / Normal)
   examples: (HighLoad, HighTemp) Hazard;  (HighLoad, LowTemp) Normal;  (LowLoad, LowTemp) Normal.   Query: (LowLoad, HighTemp)
   maximum-likelihood (alpha = 0): P(Hazard)=1/3, P(Normal)=2/3;  P(LowLoad|Hazard) = 0/1 = 0,  P(HighTemp|Normal) = 0/2 = 0
      score(Hazard) = 1/3 x P(LowLoad|Hazard) x P(HighTemp|Hazard) = 1/3 x 0 x 1 = 0;     score(Normal) = 2/3 x P(LowLoad|Normal) x P(HighTemp|Normal)
= 2/3 x 1/2 x 0 = 0  -> both zero: classification undefined (one unseen value vetoes the class, whatever the other evidence says)
   Laplace alpha = 1: P(LowLoad|Hazard) = (0+1)/(1+2) = 1/3, P(HighTemp|Hazard) = (1+1)/(1+2) = 2/3 -> score(Hazard) = (1+1)/(3+2) x 1/3 x 2/3 =
0.0889;  P(LowLoad|Normal) = (1+1)/(2+2) = 1/2, P(HighTemp|Normal) = (0+1)/(2+2) = 1/4 -> score(Normal) = (2+1)/(3+2) x 1/2 x 1/4 = 0.0750.  -> Hazard
wins, a defined answer.


A2 Implementation on the smart-grid data: 8 equal-frequency bins per feature (edges from the training part only); test set 400 states, seed 11
   train size  alpha | accuracy precision  recall     F1  log-loss | test rows with no defined class
           60      0 |    0.802     0.639   0.520  0.573     2.652 |                               0
           60    0.1 |    0.797     0.627   0.510  0.562     0.518 |                               0
```

```
          60      1 |    0.790     0.598   0.539  0.567     0.449 |                            0
          60     10 |    0.795     0.750   0.294  0.423     0.501 |                            0
         200      0 |    0.853     0.821   0.539  0.651     0.885 |                            0
         200    0.1 |    0.853     0.821   0.539  0.651     0.389 |                            0
         200      1 |    0.853     0.821   0.539  0.651     0.365 |                            0
         200     10 |    0.833     0.949   0.363  0.525     0.424 |                            0
        1000      0 |    0.910     0.867   0.765  0.812     0.326 |                            0
        1000    0.1 |    0.910     0.867   0.765  0.812     0.326 |                            0
        1000      1 |    0.912     0.876   0.765  0.817     0.325 |                            0
        1000     10 |    0.905     0.890   0.716  0.793     0.344 |                            0
   (log-loss of the alpha = 0 model uses probabilities clipped at 1e-15; undefined rows are classified by the prior and given probability 0.5)


B  KERNEL MACHINES (Task 5.2)
   which assumption becomes unsafe: Naive Bayes assumes the features are conditionally independent given the class and models each by a per-feature
count; a linear SVM assumes a linear boundary. The hazard region is a band in (load, temperature) jointly: neither the marginal of one feature nor any
line separates it.
   Experiment on 5 different splits (train 600, test 400 each), all 6 features standardised for the SVMs; 5-fold CV for hyper-parameters: linear C in
{0.01,0.1,1,10,100}; RBF C in {0.1,1,10,100} x gamma in {0.01,0.1,1,10}
                              model |  accuracy (mean +- sd) F1 hazard (mean) | support vectors | fit time ms grid search s | chosen hyper-parameters
per split
                       linear SVM |        0.774 +- 0.016           0.000 |             302 |        4.8          0.39 | [[0.01], [0.01], [0.01],
[0.01], [0.01]]
                         RBF SVM |        0.892 +- 0.014           0.762 |             235 |        8.9          0.91 | [[100, 0.01], [10, 0.1],
[10, 0.1], [10, 0.1], [100, 0.1]]
               Naive Bayes (binned) |        0.894 +- 0.018           0.735 |               0 |        0.2          0.00 | [[0.1], [0.1], [0.1],
[0.1], [0.1]]
     decision tree (Task 2 learner) |        0.911 +- 0.015           0.803 |               0 |        7.7          0.00 | [[4], [4], [4], [4], [4]]
   (identical train/test split for every model within a split; 5 splits)
   decision boundaries (two features only, for display) saved: linear SVM accuracy on the 600 training states: 0.742, RBF: 0.948


C  EXPLANATION-BASED LEARNING (Task 5.3)
   Domain theory (9 Horn rules; operational predicates = directly observable facts: FanFailed, PumpFailed, HighLoad, HighTemp, PoorVentilation,
AgedInsulation, Humid, OilFilled, NearResidential):
     R1  Hazardous(?s) :- FireHazard(?s)
     R2  Hazardous(?s) :- Overheating(?s), NearResidential(?s)
     R3  FireHazard(?s) :- ArcRisk(?s), OilFilled(?s)
     R4  ArcRisk(?s) :- InsulationStress(?s), Humid(?s)
     R5  InsulationStress(?s) :- Overheating(?s), AgedInsulation(?s)
     R6  Overheating(?s) :- HighLoad(?s), CoolingFault(?s)
     R7  Overheating(?s) :- HighTemp(?s), PoorVentilation(?s)
     R8  CoolingFault(?s) :- FanFailed(?s)
     R9  CoolingFault(?s) :- PumpFailed(?s)


   Explanation tree of the single training example Hazardous(T7) with facts FanFailed, HighLoad, AgedInsulation, Humid, OilFilled:
     R1    Hazardous(T7)
       R3    FireHazard(T7)
         R4    ArcRisk(T7)
           R5    InsulationStress(T7)
             R6    Overheating(T7)
               fact  HighLoad(T7)
               R8    CoolingFault(T7)
                 fact  FanFailed(T7)
             fact  AgedInsulation(T7)
           fact  Humid(T7)
         fact  OilFilled(T7)


   Generalised explanation (constants T7 replaced by variables, following the same rules R1, R3, R4, R5, R6, R8):
     LEARNED RULE:  Hazardous(?x) :- HighLoad(?x), FanFailed(?x), AgedInsulation(?x), Humid(?x), OilFilled(?x)


   20 designed test cases: 6 repeat the training pattern, 4 use PumpFailed instead of FanFailed, 2 reach Hazardous by R2, 2 by R7 and the rest are
random fact sets (some with unrelated hazards).
    case | theory proves theory steps | learned rule fires rule steps
      S1 |          True           17 |              True          5
      S2 |          True           17 |              True          5
      S3 |          True           17 |              True          5
      S4 |          True           17 |              True          5
      S5 |          True           17 |              True          5
      S6 |          True           17 |              True          5
      S7 |          True           19 |             False          2
      S8 |          True           19 |             False          2
      S9 |          True           19 |             False          2
     S10 |          True           19 |             False          2
     S11 |          True           23 |             False          1
     S12 |          True           23 |             False          1
     S13 |          True           17 |             False          1
     S14 |          True           17 |             False          1
     S15 |         False           29 |             False          2
     S16 |         False           19 |             False          1
     S17 |         False           19 |             False          1
     S18 |         False           21 |             False          1
     S19 |         False           21 |             False          1
     S20 |         False           31 |             False          2
   totals: theory 398 steps, learned rule 50 steps (speed-up 8.0x);  agreement 12/20;  hazardous cases missed by the learned rule: 8 (false negatives);
false positives: 0
   on the 6 cases that match the learned pattern: theory 102 steps vs rule 30 steps
```

B.6 Task 6 (t6_output.txt)

```
TASK 6 - Smart-AC controller (reinforcement learning)


S1  SIMULATOR FACTS: 96 steps/day (15 min); actions Off/Low/High = 0/1/2 kW, removing 0/0.45/0.9 degC per step; indoor temperature relaxes 8 % per step
toward outdoor (28-34 degC base +/-5 degC daily swing) + N(0, 0.05); occupied until ~08:00 (+-45 min) and from ~18:00 (+-1 h); price 1.0 per kWh, 3.0
in the 17:00-21:00 peak; reward = -(price x kWh) - 2 x (occupied step with indoor temperature outside 22-26 degC)


S2  DESK-CHECK of Q-learning, 3 states / 2 actions (alpha = 0.5, gamma = 0.9, all Q = 0, actions: 0 = Off, 1 = Cool)
    k  s  a      r  s2 | old Q(s,a) max Q(s2,.) target r+g*max  TD error new Q(s,a)
    1  0  1   -1.0   1 |     0.0000      0.0000        -1.0000   -1.0000    -0.5000
    2  1  1   -0.5   2 |     0.0000      0.0000        -0.5000   -0.5000    -0.2500
    3  2  0    1.0   2 |     0.0000      0.0000         1.0000    1.0000     0.5000
    4  2  1   -0.5   2 |     0.0000      0.5000        -0.0500   -0.0500    -0.0250
    5  1  0   -3.0   0 |     0.0000      0.0000        -3.0000   -3.0000    -1.5000
    6  0  1   -1.0   1 |    -0.5000     -0.2500        -1.2250   -0.7250    -0.8625
    7  1  1   -0.5   2 |    -0.2500      0.5000        -0.0500    0.2000    -0.1500
   final Q table (rows s0,s1,s2; columns Off, Cool): [[0.0, -0.8625], [-1.5, -0.15], [0.5, -0.025]] -> greedy policy: ['Off', 'Cool', 'Off']
   (hyper-parameters alpha 0.1, gamma 0.99, epsilon 0.995^episode with floor 0.05, 8-bin temperature grid and 1500 days were chosen on three separate
tuning seeds 100-102, not on the 20 evaluation seeds)
```

```
S3  PREDICTIONS (written before the 20-seed run)
   P1: after training the learner (Q-learning) will use less cost than the fixed-rule baseline by moving cooling away from the 17:00-21:00 peak (pre-
cooling), at the price of a similar or slightly higher number of comfort violations. Expected cost saving 10-40 %.
   P2: SARSA will converge to a similar level but a little lower reward than Q-learning (on-policy exploration noise is costly near the comfort band).
   P3 (adversarial): the policy keys on clock-time blocks. If the peak is moved to 13:00-17:00 the learner will keep cooling at the new peak and will
lose most of its cost advantage; with a +6 degC heat wave it enters bins (> 30 degC) seen rarely in training and its comfort violations will rise by
more than the baseline's.


S4  FIXED-RULE BASELINE on 40 held-out evaluation days: reward -59.60, kWh 18.54, cost 31.50, comfort violations 14.05 steps/day
   (training time for 2 x 20 seeds: 81 s)


S5  RESULTS over 20 seeds (1500 training days each; evaluation = greedy policy on 40 held-out days; mean +- sd across seeds)
            controller |       reward/day        kWh/day       cost/day   violations/day | episodes to converge (median)
            Q-learning |   -58.31 +- 6.81    26.75 +- 4.73    37.79 +- 7.37     10.26 +- 5.36 |                          175
                 SARSA |   -57.00 +- 4.75    27.91 +- 2.39    39.73 +- 3.62      8.64 +- 3.01 |                          200
    fixed-rule baseline |          -59.60          18.54          31.50            14.05 |                            -
   Q-learning vs baseline reward: mean diff +1.29, one-sample t-test over 20 seeds p = 4.20e-01; seeds beating baseline reward: 14/20
   Q-learning vs SARSA reward (paired by seed): mean diff -1.31, p = 0.479
   cost change vs baseline: +20.0 %, kWh change: +44.3 %, violation change: -3.79 steps/day


S6  ADVERSARIAL TEST (designed by the student; predictions P3 above) - same trained Q-learning policies (20 seeds), changed world, 40 days each
                      scenario | baseline cost learner cost | baseline viol learner viol | baseline reward learner reward | seeds learner reward >
baseline
                        normal |         31.50        37.79 |         14.05        10.26 |          -59.60         -58.31 | 14/20
             heat wave +6 degC |         44.77        20.14 |         35.02        44.84 |         -114.83        -109.82 | 16/20
      peak moved to 13:00-17:00 |         25.01        40.65 |         14.05        10.26 |          -53.11         -61.18 | 1/20
                          both |         38.80        26.70 |         35.02        44.84 |         -108.85        -116.38 | 3/20
```